National University of Computer and Emerging Sciences
Lahore Campus

MLOps — Fall 2026

VisionGuard

AI vs Real Image Classification and Deployment

Assignment 2 — `23L-0764_A2_Ops`

Name	Anas Naveed Butt
Roll Number	23L-0764
Section	7-D
Instructor	Usman Anwer
Submission date	6 November 2026

Project Information

Links and graded version

GitHub repository	https://github.com/Anasnaveed69/23L-0764_A2_Ops
DagsHub project (code, data, experiments, registry)	https://dagshub.com/Anasnaveed69/23L-0764_A2_Ops
MLflow tracking server	https://dagshub.com/Anasnaveed69/23L-0764_A2_Ops.mlflow
Live model API (cloud)	https://20-219-10-43.sslip.io (`GET /health`, `POST /v1/predict`)
Web application	Flask on `http://localhost:5000` calling the API above (Task 8 design)
Commit hash of graded version	`9fbde8c`

System at a glance

VisionGuard is a web service that estimates whether an uploaded image was produced by a generative model or by a camera. The pipeline is: Kaggle dataset → audit and cleaning → DVC-versioned data on DagsHub → `tf.data` pipeline → two Keras models (a CNN from scratch and a fine-tuned MobileNetV2) → every run tracked in MLflow on DagsHub → the best model registered with alias `champion` → a model API container on an Azure VM behind HTTPS → a Flask web app in Docker Compose with a PostgreSQL audit trail, calling that API.

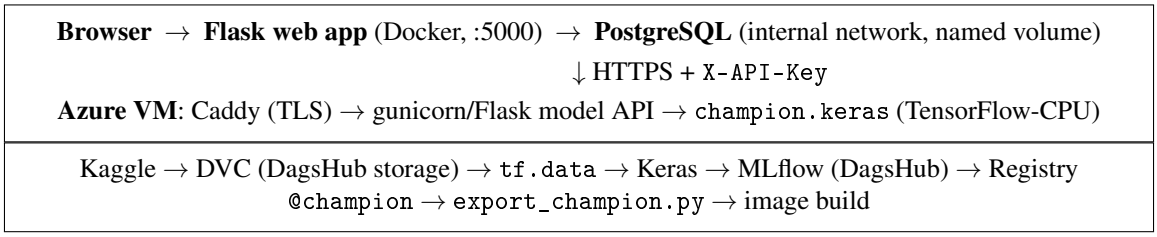


Figure 1: Serving path (top) and training path (bottom).

Disclosure of AI assistance and tutorials

I used an AI assistant, Claude (Anthropic), extensively in this assignment:

- **Code**: Claude wrote most of the code in the repository (data scripts, training and tracking code, Flask app, API, Dockerfiles, Compose files, CI workflow, Colab notebooks), which I reviewed.
- **Execution**: through the Claude desktop app, Claude operated my browser to run the Colab notebooks, create the GitHub/DagsHub repositories and set up the Azure VM via Cloud Shell, while I signed in to each service, created the access tokens and entered all secrets myself.
- **Report**: Claude drafted the explanations in this report; I checked them against my results and edited them. All numbers, tables and plots come from the actual runs (they are written by the project scripts

from MLflow and the evaluation code, not typed by hand).

Documentation consulted: TensorFlow/Keras guides (`tf.data`, transfer learning), MLflow, DagsHub, DVC, Docker/Compose, GitHub Actions and Azure documentation.

How to read this report

Each chapter is one task; each section is one sub-task ID from the assignment, and explains *what* I did, *why*, *how it works*, and *what went wrong*. Every command is explained where it is used. Tables marked as generated were written by the project scripts (`report/generated/`) so the numbers cannot drift from the actual runs.

Contents

Project Information i

1 Data Understanding and Versioning 1

1.1 Environment . . . 1
1.2 Dataset audit . . . 2
1.3 Visual EDA . . . 5
1.4 Data version control with DVC . . . 6

2 Model Development with TensorFlow/Keras 12

2.1 Input pipeline . . . 12
2.2 Baseline CNN from scratch . . . 13
2.3 Transfer learning (MobileNetV2) . . . 15
2.4 Training discipline . . . 17
2.5 Evaluation . . . 18
2.6 Error analysis . . . 19
2.7 Tool lab: Keras Tuner . . . 20

3 Experiment Tracking with MLflow and DagsHub 25

3.1 Tool lab: MLflow locally . . . 25
3.2 DagsHub setup . . . 27
3.3 Instrumenting VisionGuard training . . . 28
3.4 Artifacts . . . 31
3.5 Experiment comparison . . . 32
3.6 Model registry . . . 34

4 Flask Web Application 37

4.1 Inference module . . . 37
4.2 Routes and validation . . . 38
4.3 HTML and CSS . . . 39
4.4 Configuration and production server . . . 42

5 Database Trail 43

6 Containerization with Docker and Docker Compose 46

6.1 Docker warm-up . . . 46
6.2 Dockerfile for the web app . . . 49
6.3 Compose stack . . . 51
6.4 Image analysis . . . 54
6.5 Operations and troubleshooting . . . 56

6.6 Tool lab: GitHub Actions (CI) 59

7 API Deployment **63**

8 Integrating Flask and the Cloud API **71**

9 Conclusion and Limitations **75**

1 | Data Understanding and Versioning

1.1 Environment

What and why. Every part of the project runs in an isolated environment, so library versions cannot clash with anything else and anyone can rebuild the same set:

- **Training/data**: a Google Colab runtime, i.e. a fresh, disposable Linux VM per session with an NVIDIA T4 GPU. On top of Colab's own TensorFlow/Keras the notebooks install pinned versions of the tracking stack (MLflow 2.22.5, Keras Tuner 1.4.7, DVC 3.67.1) and my package `visionguard` in editable mode (`pip install -e .`).
- **Serving**: Docker images with pinned `requirements-serve.txt` and `requirements-tf.txt` (Chapter 6).
- **Local development**: `py -m venv .venv` creates a private interpreter with an empty `site-packages`; `pip install -r requirements.txt` installs the pinned set (Python 3.11/3.12).

`scripts/00_env_check.py` prints the date, user and host, the Python version, every library version, whether a virtualenv is active, and what TensorFlow can see with `tf.config.list_physical_devices("GPU")`. In the Colab runtime it reports TensorFlow 2.21.0 built with CUDA and **one visible GPU**. (It also reports "in virtualenv: False": Colab's isolation is the whole throw-away VM, not a venv inside it.)

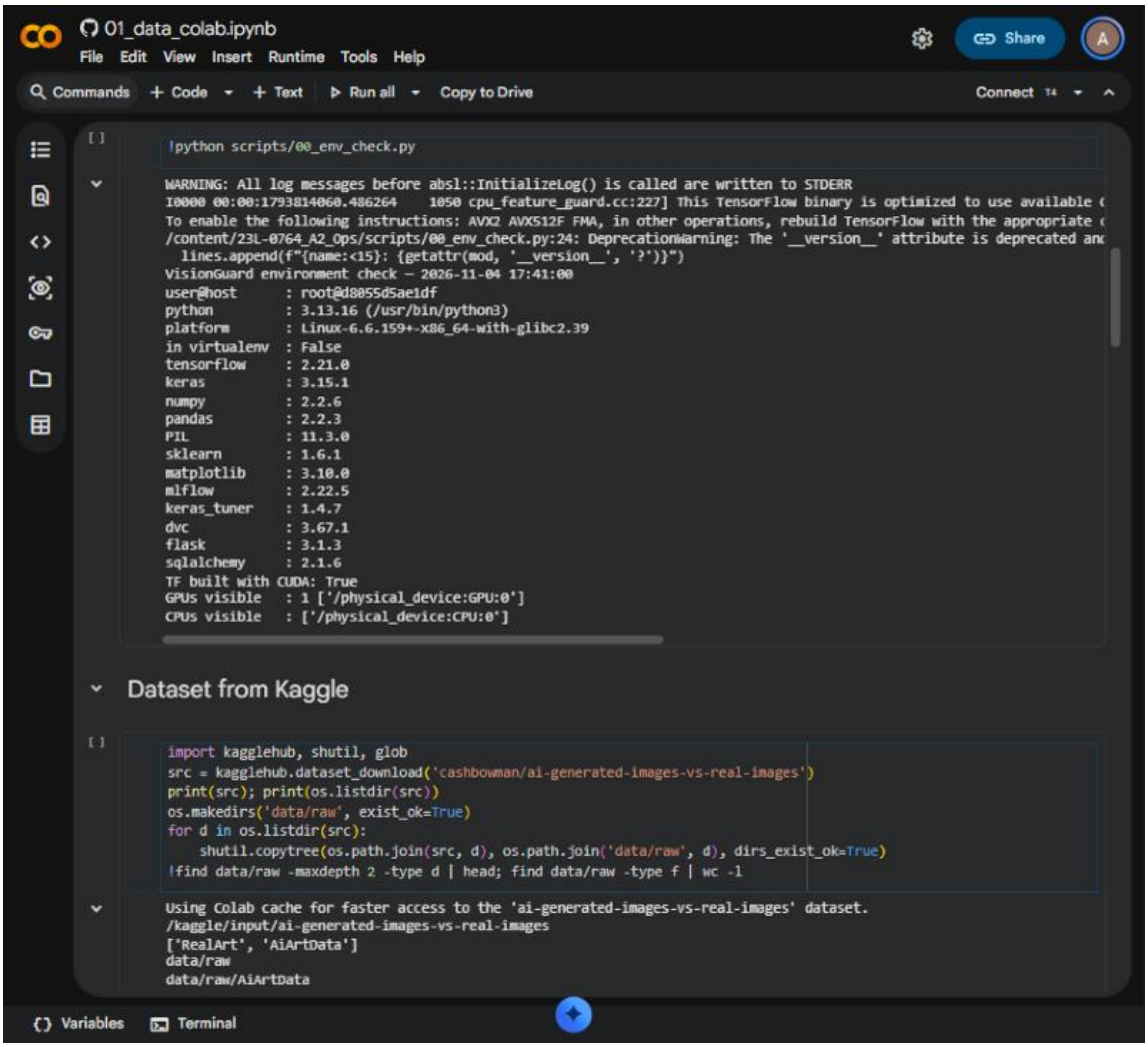


Figure 1.1: Exact library versions and GPU visibility (`scripts/00_env_check.py` in the Colab runtime).

Why MLflow 2.x and not 3.x? MLflow 3 logs models through a new "logged models" REST API. DagsHub's hosted tracking server implements the MLflow 2 API, so I pinned the client to the latest 2.x release to keep `log_model` and the Model Registry working against DagsHub.

1.2 Dataset audit

What. `scripts/01_audit.py` walks every file under `data/raw` and records, per image: class (from the top-level folder name), file size, width, height, colour mode, format, a SHA-256 hash of the bytes, whether the file can be fully decoded, and the error if not. The full table is saved to `reports/audit/audit.csv`; summaries are written straight into this report.

How each check works.

- *Readability*: `PIL.Image.verify()` catches broken headers and `load()` forces a full pixel decode, which catches truncated files that open fine but fail later. I deliberately use the *same* decode function as training and serving (`src/visionguard/imaging.py`), so "readable" in the audit means "usable by the model".
- *Duplicates*: two files are exact duplicates when their SHA-256 hashes are equal (same bytes). I keep the first copy only. Duplicates matter because a copy in the training set and another in the test set leaks the answer and inflates test accuracy.

- *Label conflicts*: a hash that appears under *both* classes is label noise; all copies are dropped.
- Images smaller than 32 pixels on a side are flagged as unusable.

Class	Files	Readable	Corrupt	Duplicates	Too small
AI-generated	538	538	0	1	0
Real	435	435	0	0	0
Total	973	973	0	1	0

Table 1.1: Files per class: total, readable, corrupt (cannot be decoded), exact duplicates and too-small images.

Class	Quantity	Min	Median	Max
AI-generated	Width (px)	225	1028	6000
AI-generated	Height (px)	135	800	4001
AI-generated	File size (KB)	11	195	5494
Real	Width (px)	183	940	8495
Real	Height (px)	158	720	7091
Real	File size (KB)	11	142	9956

Table 1.2: Minimum, median and maximum of width, height and file size per class (readable files only).

Class	Format	Colour mode	Files
AI-generated	JPEG	L	1
AI-generated	JPEG	RGB	427
AI-generated	PNG	P	14
AI-generated	PNG	RGB	61
AI-generated	PNG	RGBA	35
Real	JPEG	L	6
Real	JPEG	RGB	396
Real	PNG	P	9
Real	PNG	RGB	10
Real	PNG	RGBA	14

Table 1.3: Image formats and colour modes per class.

The dataset has 973 files: 538 AI-generated and 435 real. 0 files could not be decoded and 1 are exact duplicates (1 duplicate groups, 0 hashes appearing in both classes). After cleaning, 972 images remain and are split 70/15/15 with stratification (same AI/real ratio in every split), using a fixed seed:

Split	Images	AI-generated	Real	% AI
train	680	376	304	55.3
val	146	81	65	55.5
test	146	80	66	54.8

Table 1.4: Stratified train / validation / test split after removing corrupt files, duplicates and label conflicts.

File	Error
none	-

Table 1.5: Files that could not be decoded (first 15) and the decoder's error.

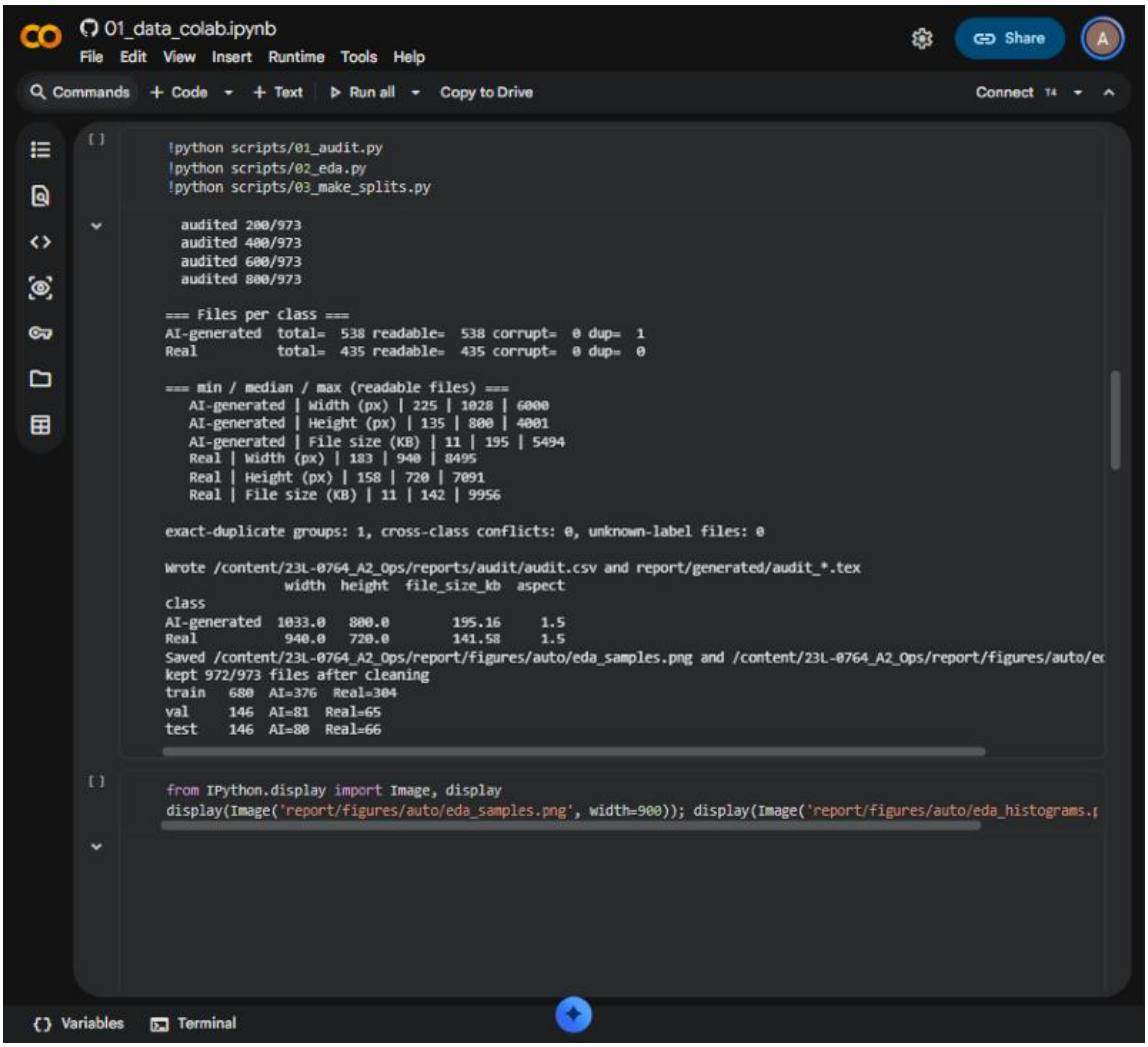


Figure 1.2: Terminal output of the audit script.

Problem I had to design around

The obvious decoder, `tf.io.decode_image`, cannot read WEBP files, CMYK JPEGs or palette PNGs with transparency, all of which occur in scraped data. Using it would have marked good images as "corrupt" and thrown them away, so the audit, training and serving all use one Pillow function that converts every image to RGB (compositing transparency onto white so transparent pixels do not turn black).

1.3 Visual EDA

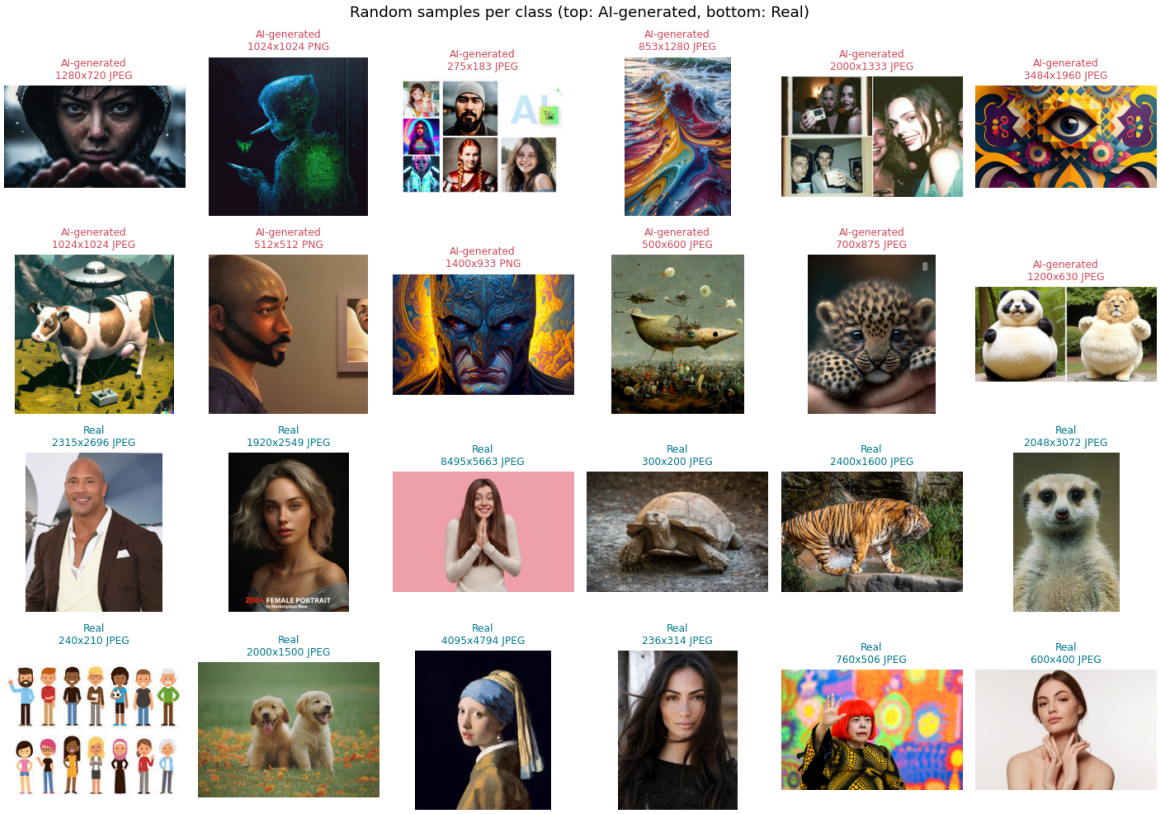


Figure 1.3: Random samples per class (top two rows AI-generated, bottom two rows real), with original size and format.

Interpretation. The AI-generated images are mostly fantasy and "digital art" compositions: saturated colours, glossy lighting, psychedelic patterns, impossible scenes (a cow under a UFO) and several collages of many small images. The "real" class is *not* only camera photographs: it is mainly professional stock photography and studio portraits, but it also contains classical paintings (a Vermeer portrait) and flat cartoon clip-art. So "real" here means "made without a generator", and both classes contain people, animals and landscapes; the model cannot solve the task by topic alone and has to use style and texture.

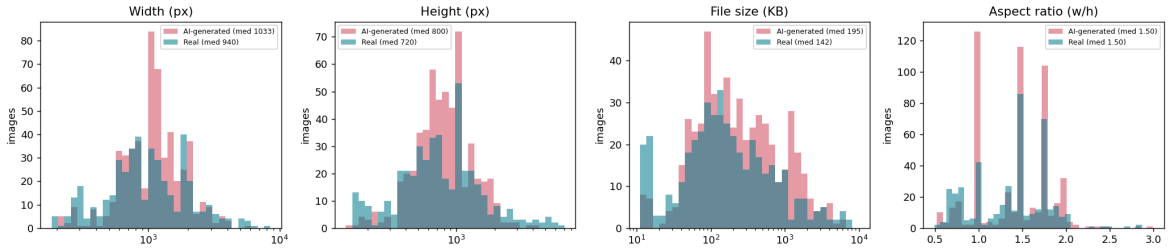


Figure 1.4: Width, height, file size (log scale) and aspect ratio per class.

Interpretation. The median AI image is 1033×800 pixels and 195 KB, versus 940×720 pixels and 141 KB for real images. 23% of AI images are exactly square against 9% of real ones, because generators output fixed square resolutions. These differences are a *shortcut risk*: a network could learn "square and small file ⇒ AI" instead of learning generation artefacts. Resizing every image to the same square input removes the size signal from the pixels, but compression strength still differs; I return to this in the error analysis (Section 2.6).

Check against the figures. The histograms confirm the direction of the size gap: AI images are larger (median 1033×800, 195 KB vs 940×720, 141 KB), and the square-image peak is clear but not dominant (23 % vs 9 %), so size alone could not separate the classes.

1.4 Data version control with DVC

Toy exercise

What and why. In an empty folder I created a 3-row `scores.csv`, versioned it with DVC, changed it, travelled back to the old version, and round-tripped it through remote storage on DagsHub (repository `23L-0764_A2_Ops-dvc-toy`). Commands, in order (`labs/dvc_toy.sh`):

1. `git init`, `dvc init`: Git tracks code and small text; DVC adds `.dvc/` (config + local cache) on top of Git.
2. `dvc add scores.csv`: moves the content into the content-addressed cache `.dvc/cache/<md5>` and writes the pointer `scores.csv.dvc`; it also adds `scores.csv` to `.gitignore` so Git never stores the data itself. I commit the pointer.
3. Append two rows, `dvc add` again, commit: the pointer now holds a different md5.
4. `git checkout HEAD~1 - scores.csv.dvc` restores the old pointer and `dvc checkout` replaces the workspace file with the cached content for that md5: the 3-row version is back. Git chooses *which* version; DVC supplies the *bytes*.
5. `dvc remote add -d origin s3://dvc` plus `endpointurl` points DVC at DagsHub's S3-compatible storage; the token is set with `-local`, which writes it to `.dvc/config.local` (git-ignored) instead of the committed `.dvc/config`.
6. `dvc push -all-commits` uploads the data for every committed version. I then deleted `scores.csv` *and* the local cache, and `dvc pull` downloaded it again.

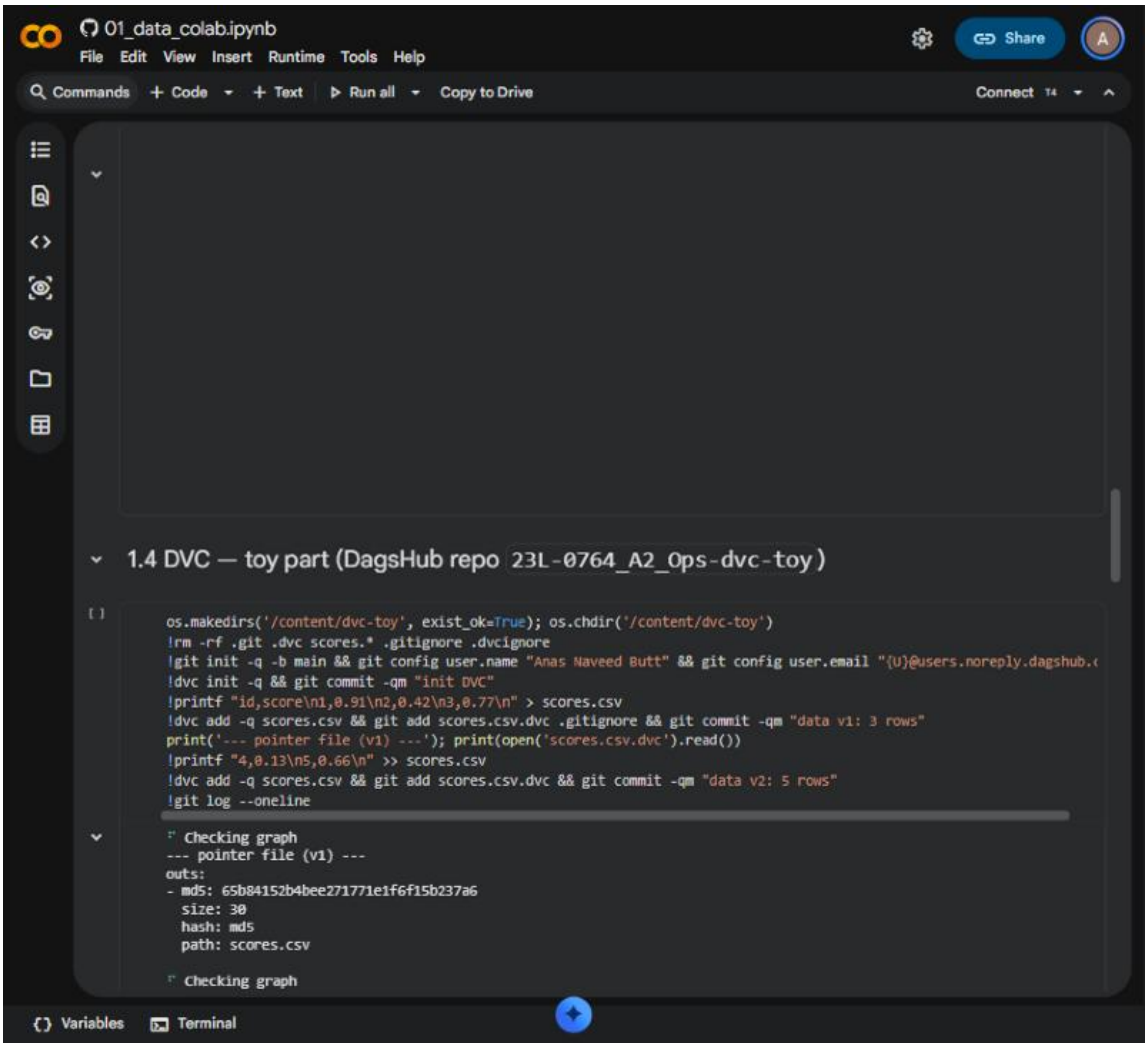


Figure 1.5: The pointer file and the two commits (v1, v2).

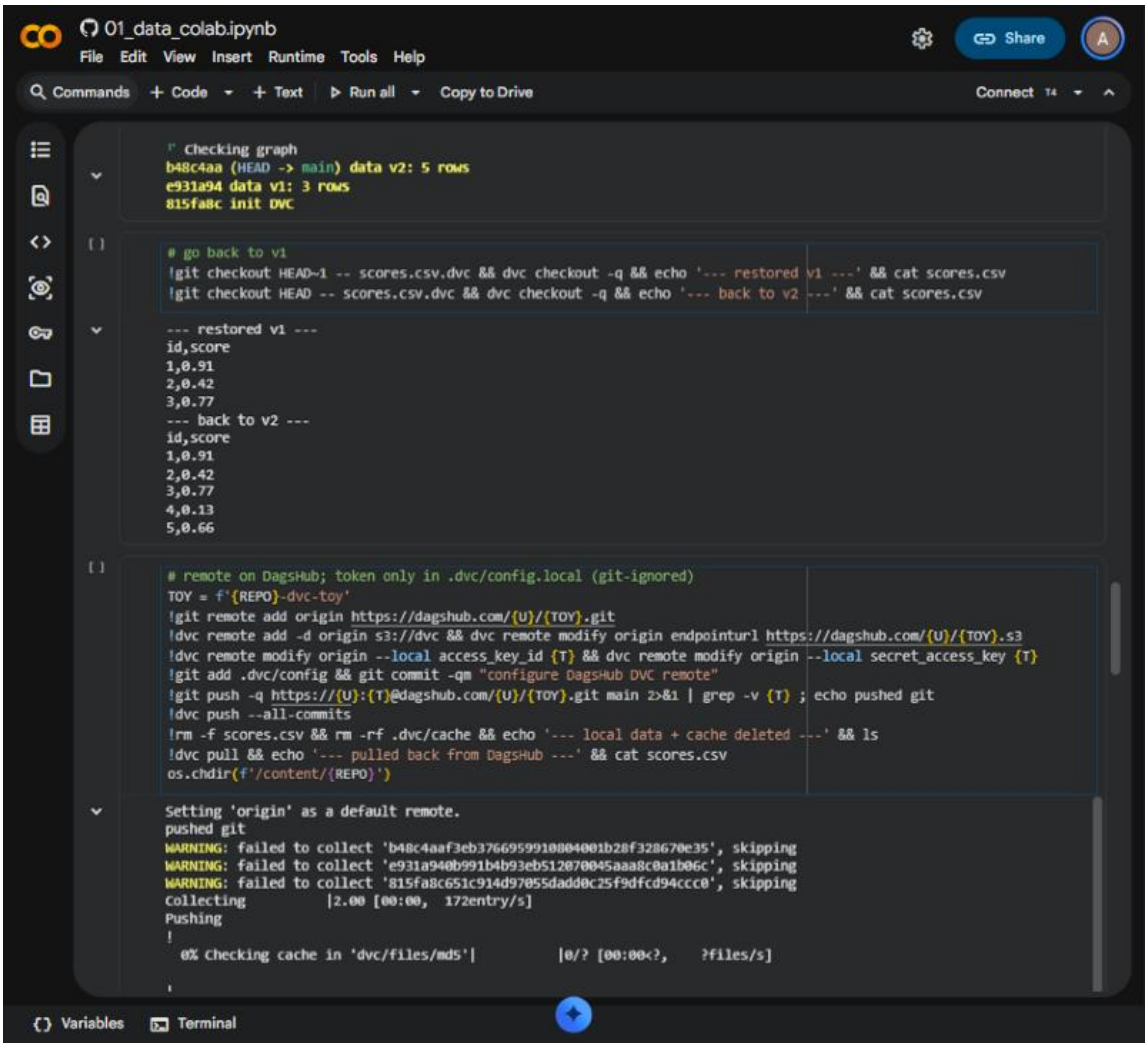


Figure 1.6: Returning to the older version with `git checkout` + `dvc checkout`.

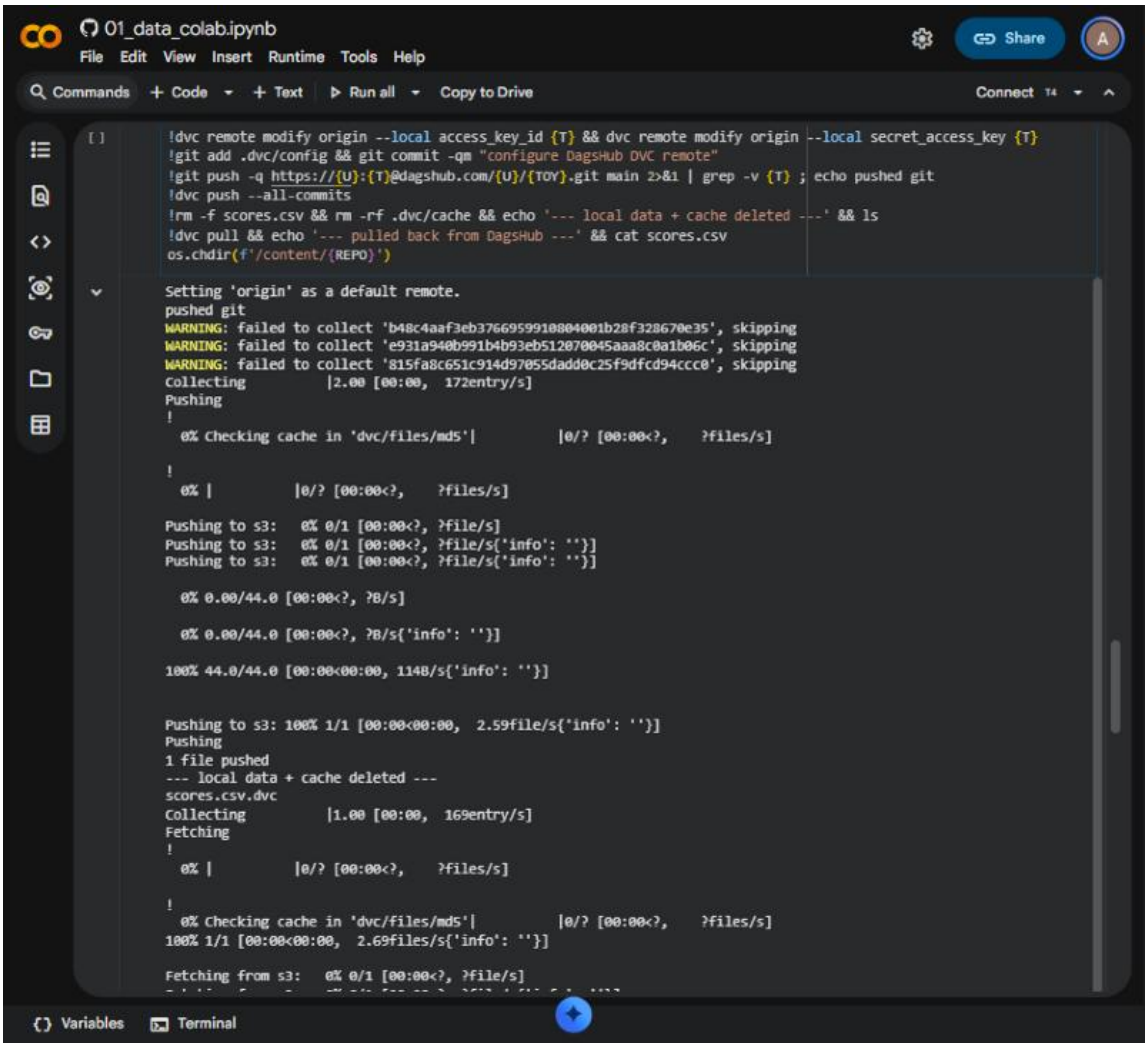


Figure 1.7: Push to DagsHub, delete local data and cache, pull it back.

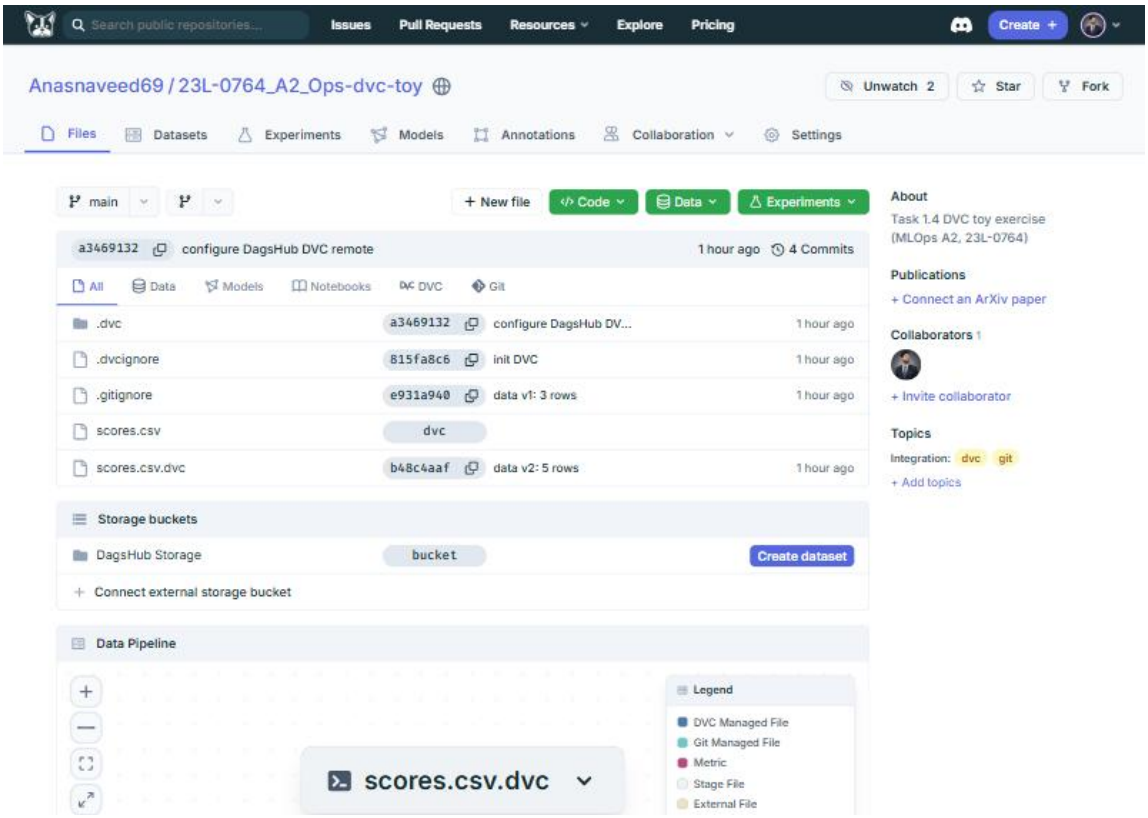


Figure 1.8: The toy repository on DagsHub with the DVC-tracked file.

What the pointer file holds, and why data does not belong in Git

The `.dvc` file is a few lines of YAML: the `md5` of the content (for a directory, the md5 of a manifest listing every file and its hash), the total `size`, the number of files and the `path`. It is a fingerprint, not data. Git is built for small text that changes line by line: every version of every file is stored forever in `.git`, so committing hundreds of megabytes of images makes every clone download all of it, binary diffs are meaningless, and GitHub rejects files over 100 MB. With DVC, Git history stays tiny, and the data version is the md5 in the pointer, which I also log to MLflow as the tag `dvc_data_version`, so every experiment says exactly which data it used.

Applying it to VisionGuard

`dvc add data/raw` tracks the whole dataset directory as one unit (`data/raw.dvc`), with DagsHub storage of the `23L-0764_A2_Ops` repository as the default remote. The split lists (`data/splits/*.csv`, a few KB) are committed to Git directly, because they are small text and must change together with the code that produced them.

```
Task 1.4: notebook 01 output (dvc add / push) + pointer commit
$ dvc remote add -d -f origin s3://dvc
$ dvc remote modify origin endpointurl https://dagshub.com/Anasnaveed69/23L-0764_A2_Ops.s3
$ dvc remote modify origin --local access_key_id ****   # token only in .dvc/config.local
$ dvc add data/raw
$ cat data/raw.dvc
outs:
- md5: b2cec67839f742df4653f66d4bcbbafb.dir
  size: 503783378
  nfiles: 975
  hash: md5
  path: raw
$ dvc push
Pushing
975 files pushed
$ git log --oneline -1 -- data/raw.dvc
4c3de23 Track dataset with DVC (DagsHub storage); audit, EDA and stratified spli
$ cat data/.gitignore
/raw
```

Figure 1.9: `dvc add data/raw`, commit of the pointer, and `dvc push` of the dataset.

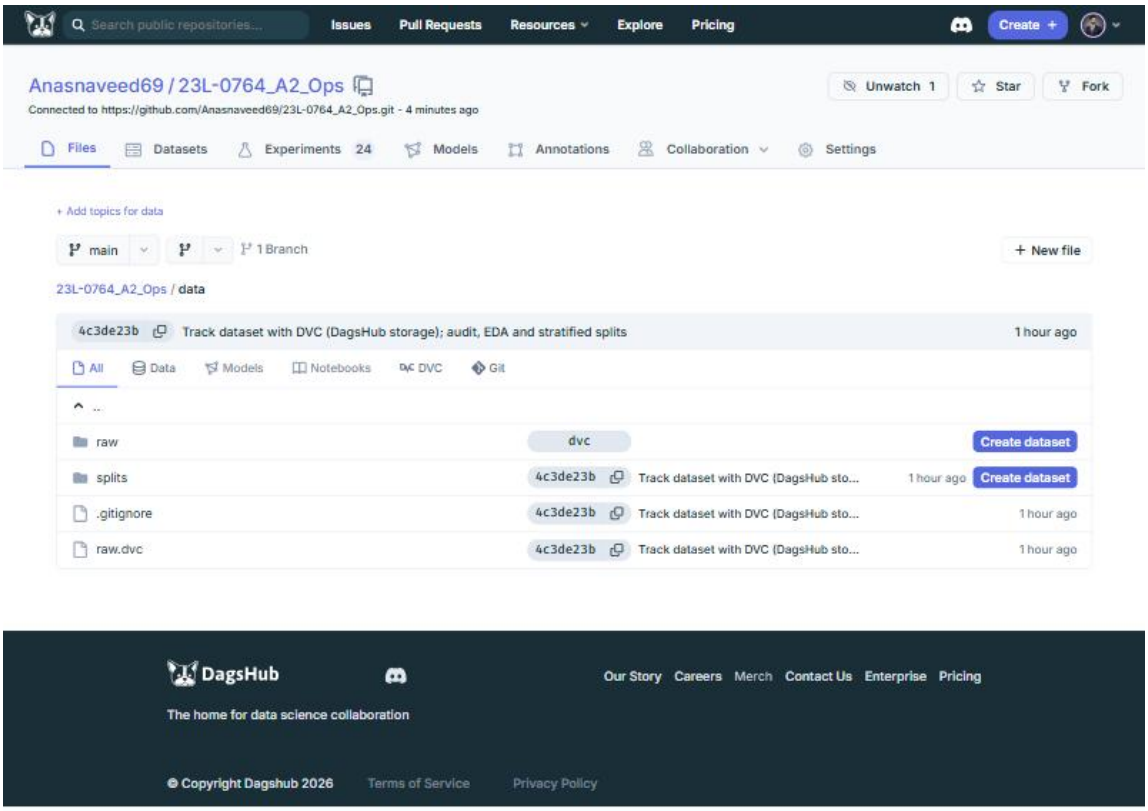


Figure 1.10: DagsHub repository page showing `data/raw` tracked by DVC.

Problems to watch for

A plain `dvc push` uploads only the data of the *current* commit, so after deleting the cache an older version could not be restored from the remote; `dvc push -all-commits` uploads every version referenced in Git history. The second trap is credentials: `dvc remote modify` without `-local` writes the token into `.dvc/config`, which is committed, so the token would be published.

2 | Model Development with TensorFlow/Keras

Where training ran

All model code lives in `src/visionguard/` and is driven by `scripts/train.py`; every setting comes from `params.yaml` and can be overridden per run (`-set train.lr_head=0.0001`). Everything ran on a Google Colab T4 GPU in two notebooks: `notebooks/01_data_colab.ipynb` (download from Kaggle, audit, EDA, splits, DVC push) and `notebooks/02_train_colab.ipynb` (training, tuning, tracking). The second one clones the repository from GitHub and pulls the exact data version with `dvc pull`, so a run is fully identified by the `git_commit` and `dvc_data_version` tags it logs. Colab already ships a matching TensorFlow/Keras pair, so only the tracking libraries are pinned there.

2.1 Input pipeline

Stages (`src/visionguard/data.py`, `make_dataset`):

1. **List**: `from_tensor_slices((paths, labels))` from the split CSV. Reading the split lists, not the folders, guarantees the same images land in the same split every time.
2. **Decode + resize** (parallel): `tf.numpy_function` calls the shared Pillow function `decode_and_resize` with `num_parallel_calls=AUTOTUNE`. The output is a $224 \times 224 \times 3$ tensor of raw 0–255 values.
3. **Cache**: the decoded images are cached in RAM after the first epoch as `uint8` ($4\times$ smaller than `float32`), so JPEG decoding happens once.
4. **Shuffle** (training only): buffer = whole training set, fixed seed, reshuffled every epoch.
5. **Batch** (32) and **augment** (training only, applied to whole batches): horizontal flip, rotation $\pm 5\%$, zoom $\pm 10\%$, translation $\pm 5\%$, contrast $\pm 10\%$.
6. **Prefetch**: the CPU prepares the next batch while the GPU trains on the current one.

Validation and test data go through the same stages without shuffle and augmentation, so they are evaluated exactly as they are.

Image size. 224 px is the resolution MobileNetV2 was pre-trained on, so its filters see objects at the scale they learned; it also keeps the batch cheap. One experiment uses 160 px to measure the effect (Section 3.5).

Aspect ratio. The default resize *squashes* every image to a square, so a 1000×500 photo is compressed horizontally by $2\times$ and its aspect ratio is lost. The alternative (`resize_mode: pad`) keeps the ratio and pads with black bars, but then the amount of padding itself tells the model the original shape, which is one of the shortcuts found in the EDA. I kept squashing for both classes so the shape signal is neutral.

Augmentation choices. The augmentations change geometry and global contrast but keep fine texture. I deliberately avoided blur, noise and JPEG re-compression: those destroy the high-frequency artefacts that distinguish generated images, and could turn an AI image into something that statistically looks like a camera image while keeping the "AI" label.

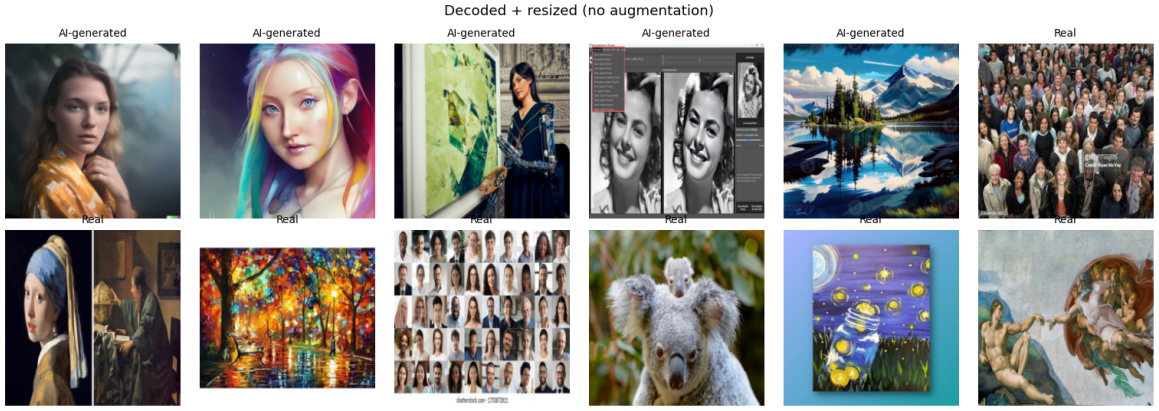


Figure 2.1: A training batch after decode + resize, before augmentation.

Figure 2.2: The same batch after the training-time augmentation.

2.2 Baseline CNN from scratch

Architecture (`models.py`, `build_baseline`). Every layer has a job:

- **Rescaling(1/255)**: maps pixels to $[0, 1]$ inside the model, so serving code only sends raw pixels and cannot forget the scaling.
- **4 blocks of Conv2D(3×3) → BatchNorm → ReLU → MaxPool(2)** with 32, 64, 128, 256 filters: convolutions learn local patterns (edges, then textures, then parts); doubling filters while halving resolution keeps compute balanced; BatchNorm normalises activations so training is stable with a higher learning rate (the conv has `use_bias=False` because BN's shift replaces the bias); ReLU adds non-linearity; pooling gives translation tolerance and grows the receptive field.
- **GlobalAveragePooling**: averages each feature map to one number, turning $14 \times 14 \times 256$ into 256 values. Far fewer parameters than `Flatten` + Dense, which matters with only a few hundred training images.
- **Dropout(0.4)**, **Dense(64, ReLU)**, **Dropout(0.2)**: a small classifier head; dropout randomly zeroes units during training to reduce over-fitting.
- **Dense(1, sigmoid)**: one output $p = P(\text{AI} \mid \text{image})$.

Loss, optimiser, metrics. Binary cross-entropy is the negative log-likelihood of a Bernoulli output, the natural loss for one sigmoid unit, and it rewards calibrated probabilities (which I need, because the threshold is chosen later). Adam with learning rate 10^{-3} adapts per-parameter step sizes and works well without

tuning on small CNNs. Metrics: accuracy (easy to read), ROC-AUC (threshold-free ranking quality), precision and recall of the AI class (the two error types of Section 2.5). Inverse-frequency class weights stop the model from ignoring the smaller class.

Layer	Type	Output shape	Params
image	InputLayer	(None, 224, 224, 3)	0
scale_0_1	Rescaling	(None, 224, 224, 3)	0
conv1	Conv2D	(None, 224, 224, 32)	864
bn1	BatchNormalization	(None, 224, 224, 32)	128
relu1	Activation	(None, 224, 224, 32)	0
pool1	MaxPooling2D	(None, 112, 112, 32)	0
conv2	Conv2D	(None, 112, 112, 64)	18,432
bn2	BatchNormalization	(None, 112, 112, 64)	256
relu2	Activation	(None, 112, 112, 64)	0
pool2	MaxPooling2D	(None, 56, 56, 64)	0
conv3	Conv2D	(None, 56, 56, 128)	73,728
bn3	BatchNormalization	(None, 56, 56, 128)	512
relu3	Activation	(None, 56, 56, 128)	0
pool3	MaxPooling2D	(None, 28, 28, 128)	0
conv4	Conv2D	(None, 28, 28, 256)	294,912
bn4	BatchNormalization	(None, 28, 28, 256)	1,024
relu4	Activation	(None, 28, 28, 256)	0
pool4	MaxPooling2D	(None, 14, 14, 256)	0
gap	GlobalAveragePooling2D	(None, 256)	0
dropout1	Dropout	(None, 256)	0
dense	Dense	(None, 64)	16,448
dropout2	Dropout	(None, 64)	0
p_ai	Dense	(None, 1)	65

Table 2.1: Baseline CNN: output shape and parameter count per layer (406,369 parameters).

baseline CNN — model.summary()

```
$ python -c "from visionguard.models import build_baseline; build_baseline(224).summary()"
Model: "baseline_cnn"
```

Layer (type)	Output Shape	Param #
image (InputLayer)	(None, 224, 224, 3)	0
scale_0_1 (Rescaling)	(None, 224, 224, 3)	0
conv1 (Conv2D)	(None, 224, 224, 32)	864
bn1 (BatchNormalization)	(None, 224, 224, 32)	128
relu1 (Activation)	(None, 224, 224, 32)	0
pool1 (MaxPooling2D)	(None, 112, 112, 32)	0
conv2 (Conv2D)	(None, 112, 112, 64)	18,432
bn2 (BatchNormalization)	(None, 112, 112, 64)	256
relu2 (Activation)	(None, 112, 112, 64)	0
pool2 (MaxPooling2D)	(None, 56, 56, 64)	0
conv3 (Conv2D)	(None, 56, 56, 128)	73,728
bn3 (BatchNormalization)	(None, 56, 56, 128)	512
relu3 (Activation)	(None, 56, 56, 128)	0
pool3 (MaxPooling2D)	(None, 28, 28, 128)	0
conv4 (Conv2D)	(None, 28, 28, 256)	294,912
bn4 (BatchNormalization)	(None, 28, 28, 256)	1,024
relu4 (Activation)	(None, 28, 28, 256)	0
pool4 (MaxPooling2D)	(None, 14, 14, 256)	0
gap (GlobalAveragePooling2D)	(None, 256)	0
dropout1 (Dropout)	(None, 256)	0
dense (Dense)	(None, 64)	16,448
dropout2 (Dropout)	(None, 64)	0
p_ai (Dense)	(None, 1)	65

```
Total params: 406,369 (1.55 MB)
Trainable params: 405,409 (1.55 MB)
Non-trainable params: 960 (3.75 KB)
```

Figure 2.3: `model.summary()` of the baseline CNN.

2.3 Transfer learning (MobileNetV2)

Why MobileNetV2. Its depthwise-separable convolutions give good ImageNet features with only about 2.3 M backbone parameters (14 MB on disk), so it fine-tunes quickly on a small dataset and runs fast on the CPU-only server. It is loaded with `keras.applications.MobileNetV2(include_top=False, weights="imagenet")`: no 1000-class classifier, just the convolutional feature extractor ending in a $7 \times 7 \times 1280$ map.

Preprocessing. MobileNetV2 was trained on pixels scaled to $[-1, 1]$ (`mobilenet_v2.preprocess_input` computes $x/127.5 - 1$). Feeding $[0, 1]$ or $[0, 255]$ would shift every activation away from what the pre-trained filters expect and destroy the value of the weights. I put this scaling inside the model as `Rescaling(1/127.5, offset=-1)`, so the saved model always applies it, in training and in serving. (EfficientNetB0, used in one experiment, contains its own normalisation and expects raw 0–255, so it gets an identity layer.)

Head. GlobalAveragePooling → Dropout(0.3) → Dense(1, sigmoid).

Stage 1 (frozen backbone). `base.trainable = False`; only the head (1 281 weights) learns, at learning rate 10^{-3}. Random head weights produce large gradients; if the backbone were trainable, those gradients would wreck the pre-trained filters.

Stage 2 (last block unfrozen). Only the layers of MobileNetV2's last inverted-residual block (`block_16`) and the final `Conv_1` are made trainable: `block_16_expand`, `block_16_expand_relu`, `block_16_depthwise`, `block_16_depthwise_relu`, `block_16_project`, `Conv_1`, `out_relu` . The model is recompiled (required for the change to take effect) with a learning rate $100\times$ smaller (10^{-5}), so the high-level features adapt to generator artefacts without forgetting ImageNet. BatchNorm layers stay

frozen and the backbone is called with `training=False`: otherwise BN would replace ImageNet statistics with statistics from batches of 32 images.

Stage	Trainable weight tensors	Backbone layers unfrozen
1 (head only)	2	0
2 (fine-tune)	6	7

Table 2.2: Trainable weights in each stage.

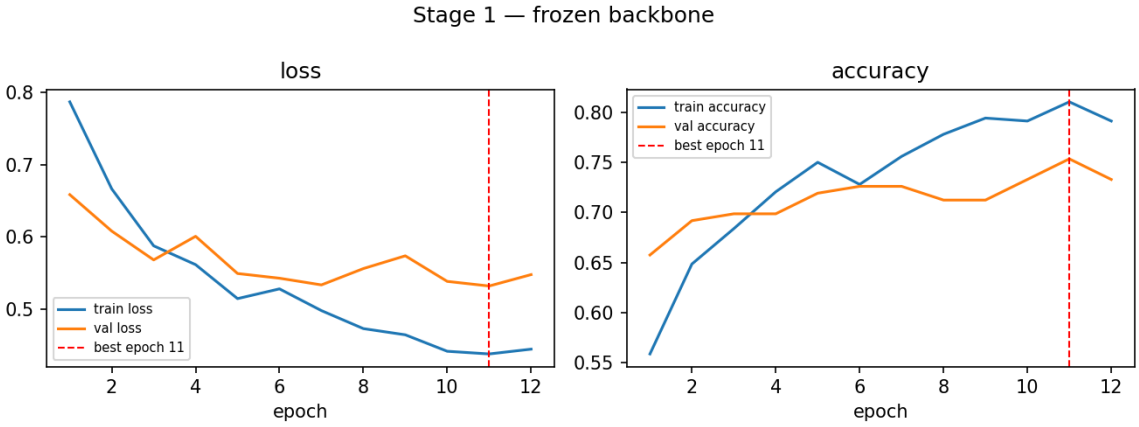


Figure 2.4: Stage 1 (frozen backbone): loss and accuracy.

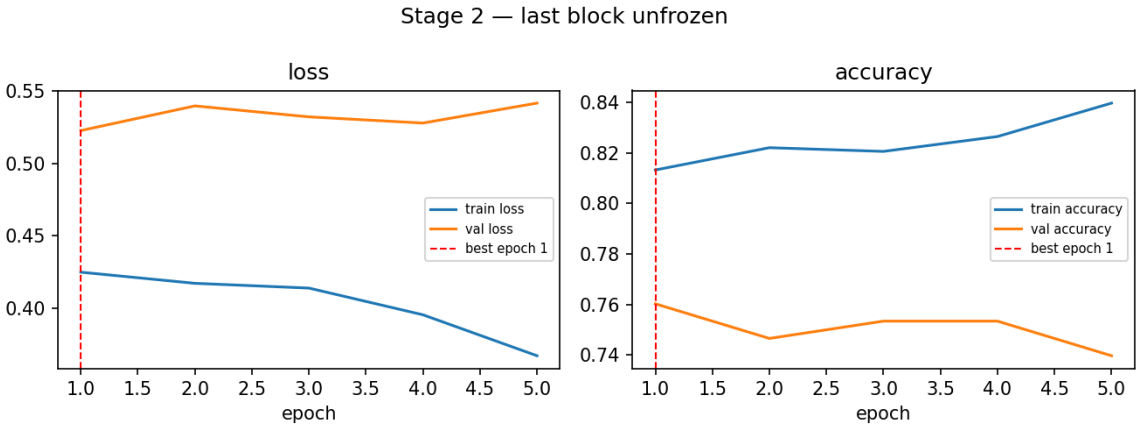


Figure 2.5: Stage 2 (last block unfrozen, learning rate 10^{-5}): loss and accuracy.

Layer	Type	Output shape	Params
image	InputLayer	(None, 224, 224, 3)	0
mobilenetv2_preprocess	Rescaling	(None, 224, 224, 3)	0
backbone	Functional	(None, 7, 7, 1280)	2,257,984
gap	GlobalAveragePooling2D	(None, 1280)	0
head_dropout	Dropout	(None, 1280)	0
p_ai	Dense	(None, 1)	1,281

Table 2.3: Transfer model, top-level layers (2,259,265 parameters in total, 880,321 trainable in stage 2).

What went wrong

In Keras 3, renaming the backbone with `base._name = "backbone"` silently had no effect, so `model.get_layer("backbone")` failed when unfreezing. Passing `name="backbone"` to the constructor fixed it.

2.4 Training discipline

- **Seeds**: `keras.utils.set_random_seed(42)` seeds Python, NumPy and TensorFlow together; `PYTHONHASHSEED` is fixed; the split, the shuffle and every augmentation layer get explicit seeds. (GPU kernels are not bit-exact, so reruns on a GPU can differ in the third decimal; the seed is logged as a tag on every run.)
- **EarlyStopping** on `val_loss`, patience 4, `restore_best_weights=True`.
- **ModelCheckpoint** on `val_loss` with `save_best_only`. One checkpoint object is shared by both transfer stages, so it keeps the best epoch *across* stages; after training I reload that file, so the evaluated model is the best one, not the last one.
- **ReduceLROnPlateau**: multiplies the learning rate by 0.3 after 2 epochs without improvement in `val_loss`.

Best epochs (lowest validation loss): baseline CNN **epoch 38**, transfer model **epoch 13** (counted across both stages).

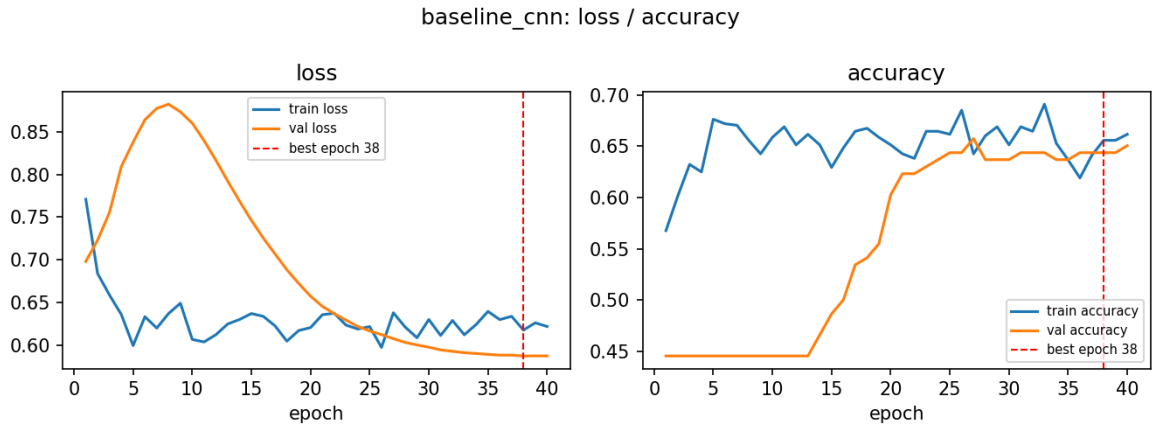


Figure 2.6: Baseline CNN: loss and accuracy, best epoch marked in red.

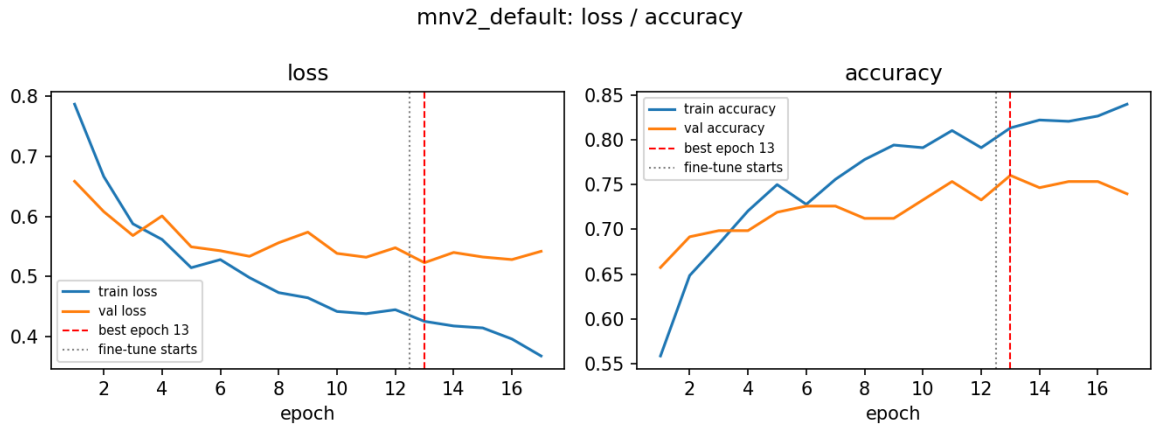


Figure 2.7: Transfer model, both stages: best epoch in red, start of fine-tuning dotted.

What the curves show. *Baseline:* the training loss drops quickly to about 0.62 and then stays flat, so the small CNN under-fits rather than over-fits. For the first 13 epochs the validation accuracy is stuck at 0.45, the fraction of real images, i.e. it predicts "real" for everything, and the validation loss even rises up to epoch 8: the BatchNorm running statistics used at validation time are still far from the training-batch statistics. Once they settle, the validation loss falls steadily to its minimum at epoch 38. This is why the baseline needed the lower learning rate and the warm-up before early stopping (Chapter 3 box). *Transfer:* the validation loss reaches about 0.53 within 5 epochs of the frozen stage and then oscillates, while the training loss keeps falling to 0.37, so the widening gap is over-fitting on ∼680 training images. Fine-tuning (dotted line) gave one more small improvement (best epoch 13, the first fine-tuning epoch) and then stopped helping, so early stopping ended the run 4 epochs later and the best weights were restored.

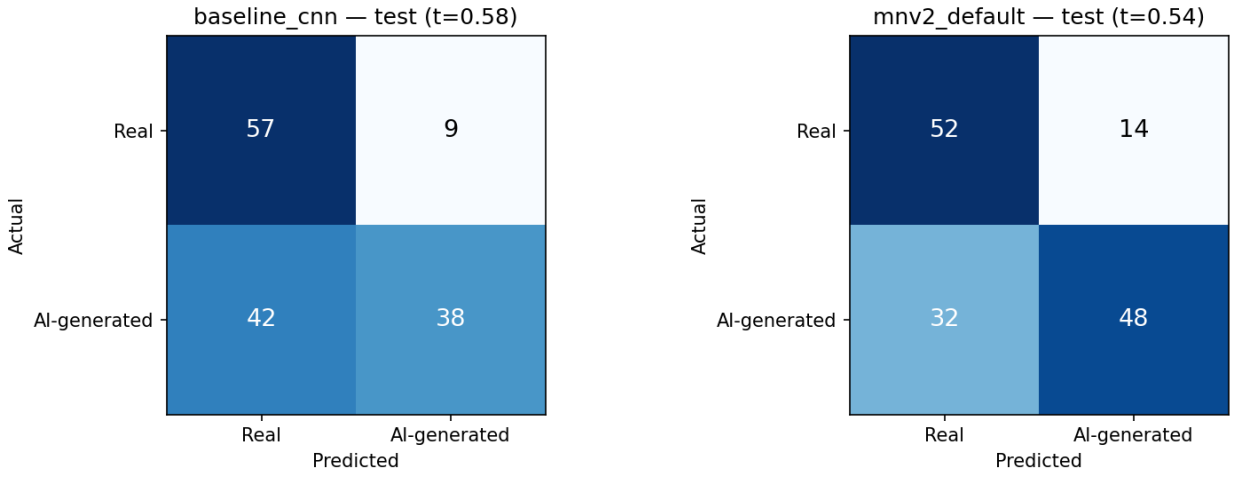
2.5 Evaluation

Which error is worse? A *false positive* (a real photo called AI-generated) is worse for this product. VisionGuard would be used to judge whether someone's photo is genuine; a false accusation harms a real person (a journalist, a seller, a student) and destroys trust in the tool. A false negative (an AI image passing as real) is bad but is the status quo without the tool. So I want high *precision* on the AI class, i.e. a low false-positive rate.

Threshold. The test set is used once, at the end. The threshold is chosen on the *validation* predictions: I scan $t \in [0.05, 0.95]$ and pick the value that maximises F_β with $\beta = 0.5$, which weights precision twice as much as recall and therefore encodes the choice above. Chosen thresholds: baseline 0.58, transfer 0.54. The threshold is saved with the model (`model_meta.json`) and used by the server.

Model	Thr.	Accuracy	Precision	Recall	F1	ROC-AUC	FPR
Baseline CNN	0.58	0.651	0.809	0.475	0.598	0.743	0.136
MobileNetV2 transfer	0.54	0.685	0.774	0.600	0.676	0.754	0.212

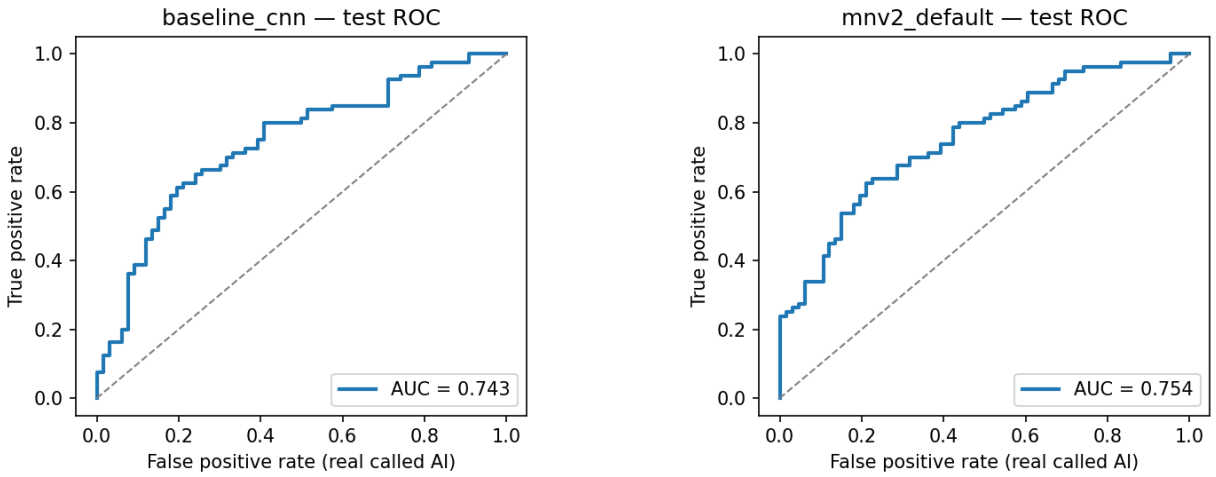
Table 2.4: Test-set metrics at the validation-chosen threshold (FPR = real photos called AI).



(a) Baseline CNN

(b) MobileNetV2 transfer

Figure 2.8: Test confusion matrices.



(a) Baseline CNN

(b) MobileNetV2 transfer

Figure 2.9: Test ROC curves.

The transfer model reaches test accuracy 0.685, F1 0.676 and ROC-AUC 0.754, with a false-positive rate of 0.212; the baseline reaches 0.651, 0.598 and 0.743 (FPR 0.136).

Comparison. With the reference settings the transfer model is only slightly better than the CNN trained from scratch (AUC 0.754 vs 0.743, F1 0.676 vs 0.598); most of the gain is in recall at the chosen threshold (0.60 vs 0.48), while the baseline is more conservative (lower FPR, 0.14 vs 0.21). The pre-trained features pay off once the transfer settings are tuned: the final champion `mnv2_tuned` (same architecture, tuner's learning rate, dropout and 60 unfrozen layers) reaches test AUC 0.844, accuracy 0.733 and F1 0.731 (Section 3.5).

2.6 Error analysis

`scripts/error_analysis.py` takes the champion's test predictions, keeps the ten most *confident* mistakes (the most informative ones), and measures simple properties of each: size, sharpness (variance of an edge filter), colour saturation, grey-scale, aspect ratio and format. These give each error a testable hypothesis instead of a guess.

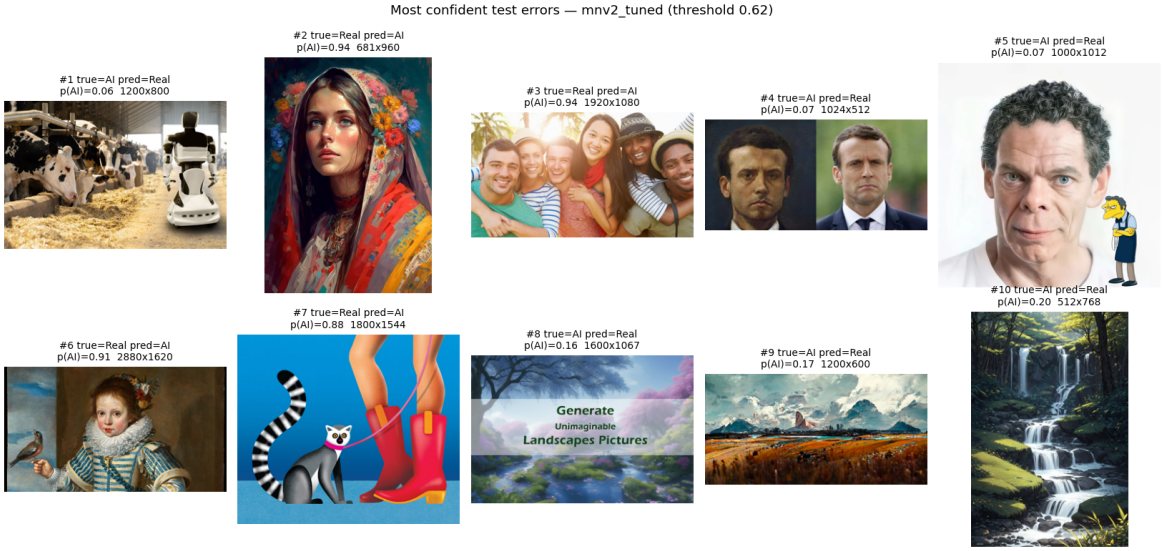


Figure 2.10: The ten most confident test errors (true label, prediction, $p(\text{AI})$, size).

\{}#	True	p(AI)	Size	Sharpness	Hypothesis (from measured cues)
1	AI	0.06	1200x800	2699	very sharp, heavy detail or noise looks 'camera-like'
2	Real	0.94	681x960	1517	very sharp, heavy detail or noise looks 'camera-like'; highly saturated, painted / illustration style
3	Real	0.94	1920x1080	2115	very sharp, heavy detail or noise looks 'camera-like'; extreme aspect ratio squashed by resizing
4	AI	0.07	1024x512	609	extreme aspect ratio squashed by resizing
5	AI	0.07	1000x1012	1447	very sharp, heavy detail or noise looks 'camera-like'
6	Real	0.91	2880x1620	2290	very sharp, heavy detail or noise looks 'camera-like'; extreme aspect ratio squashed by resizing
7	Real	0.88	1800x1544	770	highly saturated, painted / illustration style
8	AI	0.16	1600x1067	1543	very sharp, heavy detail or noise looks 'camera-like'
9	AI	0.17	1200x600	2290	very sharp, heavy detail or noise looks 'camera-like'; extreme aspect ratio squashed by resizing
10	AI	0.20	512x768	2886	very sharp, heavy detail or noise looks 'camera-like'

Table 2.5: Measured cues and hypothesis per error.

Checking the hypotheses by eye. The measured cues are only partly right; looking at the images shows a clearer pattern. *False positives* (#2, #6, #7, real → AI) are painted or illustrated "real" art: a richly coloured painted portrait, a 17th-century oil portrait and a flat vector illustration. The model has effectively learned "painterly, saturated ⇒ AI", which is mostly true in the training data but not always. #3 is a glossy, over-lit stock photo of smiling people, which looks like generator output. *False negatives* (AI → real) are photorealistic: #1 (cows with a robot in a real-looking barn), #5 (a photographic face with a cartoon pasted in) and #4 (a side-by-side composite of a painting and a press photo); #8 has large text overlaid on the picture, and #9/#10 are landscapes whose style is close to real landscape paintings. Two errors (#4, #9) are extreme aspect ratios squashed to a square. So the main weakness is the label definition itself: style (painting vs photo) is confounded with origin (generator vs human).

2.7 Tool lab: Keras Tuner

Toy part

`labs/tuner_toy.py` tunes a one-hidden-layer network on Fashion-MNIST with **Hyperband**: number of units $\in \{32, \ldots, 512\}$ and learning rate $\in \{10^{-2}, 10^{-3}, 10^{-4}\}$. Hyperband trains many configurations for a few epochs, keeps the best third, trains those longer, and repeats, so most of the budget goes to promising settings. `tuner.get_best_hyperparameters()` returns the winner.

Trial	Units	LR	Epochs	val accuracy
0005	448	0.001	8	0.8862
0006	224	0.001	8	0.8832
0004	448	0.001	3	0.8792
0001	224	0.001	3	0.8776
0003	512	0.001	3	0.8761
0009	256	0.0001	8	0.8748
0000	128	0.001	3	0.8637
0008	320	0.01	8	0.8467

Table 2.6: Toy tuner: best trials on Fashion-MNIST (validation accuracy).

```
!bash scripts/colab_regen.sh

== 2.7 toy tuner
Trial 0001 summary
Hyperparameters:
units: 224
lr: 0.001
tuner/epochs: 3
tuner/initial_epoch: 0
tuner/bracket: 1
tuner/round: 0
Score: 0.8775833249092102
Trial 0003 summary
Hyperparameters:
units: 512
lr: 0.001
tuner/epochs: 3
tuner/initial_epoch: 0
tuner/bracket: 1
tuner/round: 0
Score: 0.8760833144187927
BEST: units=448 lr=0.001
1500/1500 - 6s - 4ms/step - accuracy: 0.8210 - loss: 0.4968 - val_accuracy: 0.8490 - val_loss: 0.425
1500/1500 - 4s - 3ms/step - accuracy: 0.8646 - loss: 0.3706 - val_accuracy: 0.8680 - val_loss: 0.364
1500/1500 - 5s - 3ms/step - accuracy: 0.8788 - loss: 0.3284 - val_accuracy: 0.8771 - val_loss: 0.342
1500/1500 - 4s - 3ms/step - accuracy: 0.8879 - loss: 0.3022 - val_accuracy: 0.8798 - val_loss: 0.334
1500/1500 - 4s - 3ms/step - accuracy: 0.8954 - loss: 0.2808 - val_accuracy: 0.8834 - val_loss: 0.321
test accuracy with best settings: 0.8804
```

Figure 2.11: Toy tuner results summary.

Applied to VisionGuard

`scripts/tune.py` defines a `HyperModel` whose `fit()` reproduces the two-stage training inside each trial and searches the head learning rate (log-uniform 10^{-4}–$3 \cdot 10^{-3}$), dropout (0.1–0.5) and how many backbone layers to unfreeze $\in \{0, 11 \text{ (last block)}, 30, 60\}$ with **RandomSearch**, 10 trials, objective validation loss. Every trial is also logged to MLflow as a nested run.

Rank	lr\{}_head	Dropout	Unfrozen	val loss	val AUC
1	2.24e-03	0.1	60	0.4773	0.864
2	8.80e-04	0.1	11	0.5440	0.796
3	6.45e-04	0.1	11	0.5908	0.768
4	1.02e-03	0.5	11	0.5911	0.777
5	4.54e-04	0.3	11	0.6397	0.695
6	2.29e-04	0.5	30	0.6478	0.675
7	2.52e-04	0.5	11	0.6551	0.680
8	2.85e-04	0.3	60	0.6605	0.688
9	1.27e-04	0.1	30	0.6806	0.649
10	1.16e-04	0.3	0	0.8280	0.413

Table 2.7: VisionGuard tuner trials, best first.

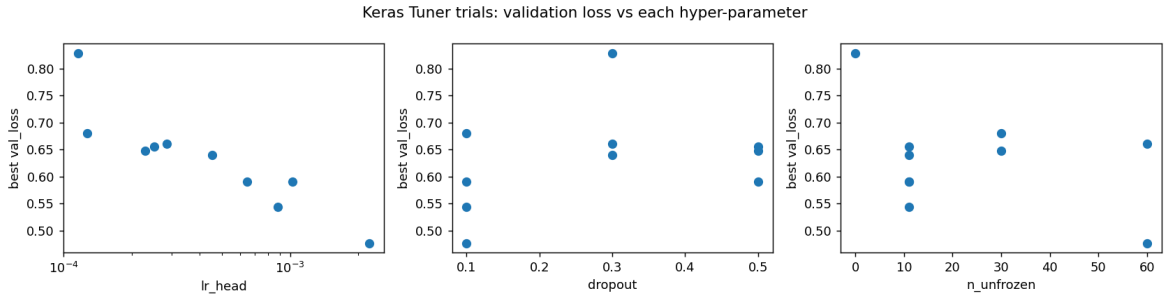


Figure 2.12: Validation loss of each trial against each hyper-parameter.

Hyper-parameter	Range of mean val loss across levels
unfrozen layers	0.2590
learning rate	0.1521
dropout	0.1362

Table 2.8: Effect size: spread of mean validation loss across the levels of each hyper-parameter.

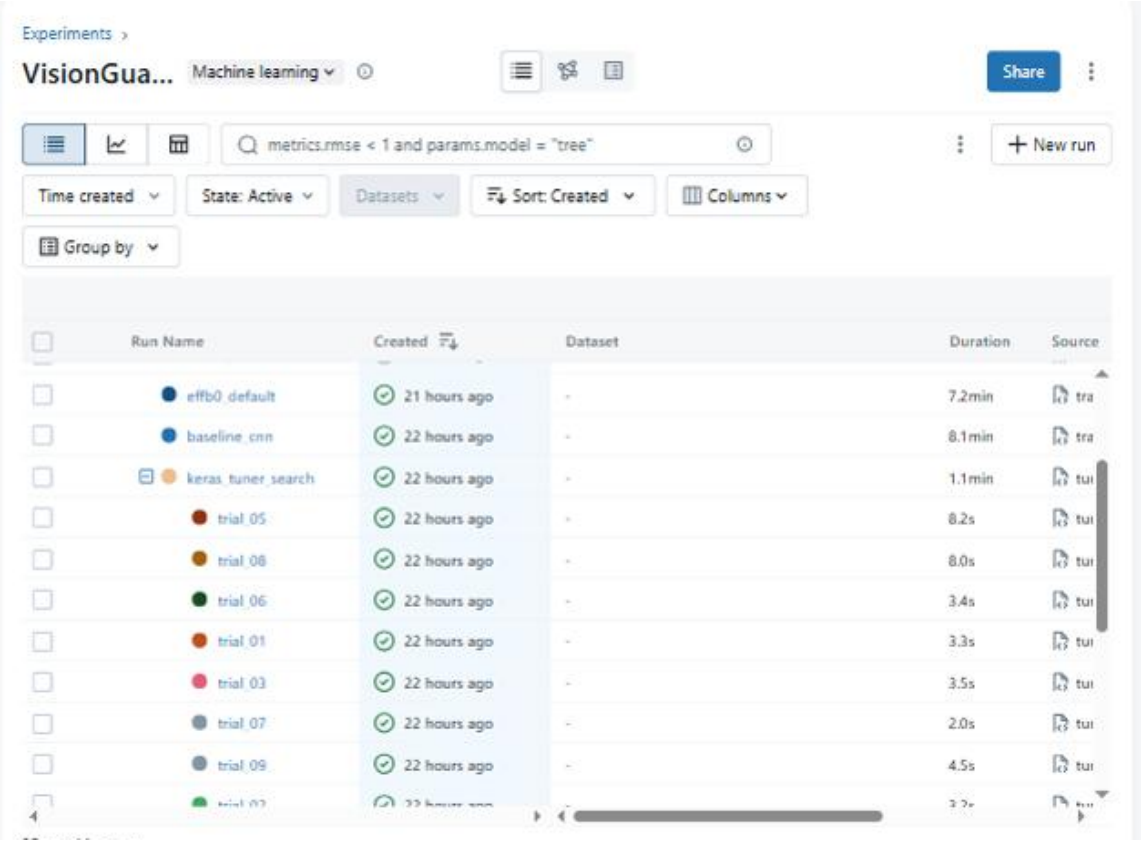


Figure 2.13: Tuner search output for VisionGuard.

Best trial: learning rate 2.24e-03, dropout 0.1, 60 unfrozen layers (validation loss 0.4773). The hyperparameter with the largest effect was **unfrozen layers**.

Why. How much of the backbone is unfrozen decides how far the ImageNet features can adapt to generator artefacts (which ImageNet never had to detect): with 0 layers the network can only re-weight generic object features, while 60 layers with a small learning rate let the mid-level texture filters change; it also has the largest effect on over-fitting, which is why its levels differ most in validation loss.

Why the best validation score can over-fit the validation set. The tuner looks at the validation loss of every trial and keeps the minimum. With many trials, some configurations score well partly by luck on this particular, small validation set (a "winner's curse"): selection itself fits noise in the validation data, so the best score is an optimistic estimate. That is why the final numbers come from the untouched test set, used once, and why a clearly simpler setting within noise of the best is a reasonable choice.

Bonus: Grad-CAM

Figure 2.14: Grad-CAM heat maps: top row correct predictions, bottom row errors. Red = regions that pushed $p(\text{AI})$ up.

Grad-CAM takes the gradient of the output with respect to the last convolutional feature map, averages it per channel to get channel weights, and shows the weighted sum of the feature maps as a heat map.

What the model looks at. For the correct AI predictions the heat map covers the dense, swirling decorative texture (the ornamental "G" and the psychedelic face), i.e. the high-frequency patterns generators produce. For real images it focuses on a small region (the tortoise's shell, beside the Mona Lisa's face) and $p(\text{AI})$ stays below 0.5. In the errors the focus is on the colourful headscarf and flowers (painted portrait $\rightarrow$ AI), the bright faces of the stock photo, and on only the left, painted half of the composite (AI $\rightarrow$ real), which supports the error analysis: colour/texture style drives the decision more than true generation artefacts.

3 | Experiment Tracking with MLflow and DagsHub

3.1 Tool lab: MLflow locally

`labs/mlflow_toy.py` trains a one-hidden-layer network on 10 000 Fashion-MNIST images and logs: parameters (`lr`, `epochs`, `units`, `optimizer`), per-epoch metrics through a small Keras callback (`mlflow.log_metrics(..., step=epoch)`), a loss plot (`mlflow.log_figure`) and the model (`mlflow.tensorflow.log_model`). I ran it with `lr=0.001` and `lr=0.01` against a local store (`-local` writes to `./mlruns_toy`) and opened `mlflow ui -backend-store-uri ./mlruns_toy`. In the comparison view the two `val_accuracy` curves can be overlaid and the parameter that differs is highlighted. These two local runs were executed on CPU in a separate Linux workspace (TensorFlow 2.20, MLflow 2.22.5), with the UI served on `127.0.0.1`. Result: after 5 epochs the higher learning rate fits the training data faster (training accuracy 0.862 vs 0.849, loss 0.373 vs 0.437) but does not generalise better (validation accuracy 0.842 vs 0.846), so `lr=0.001` is the safer choice.

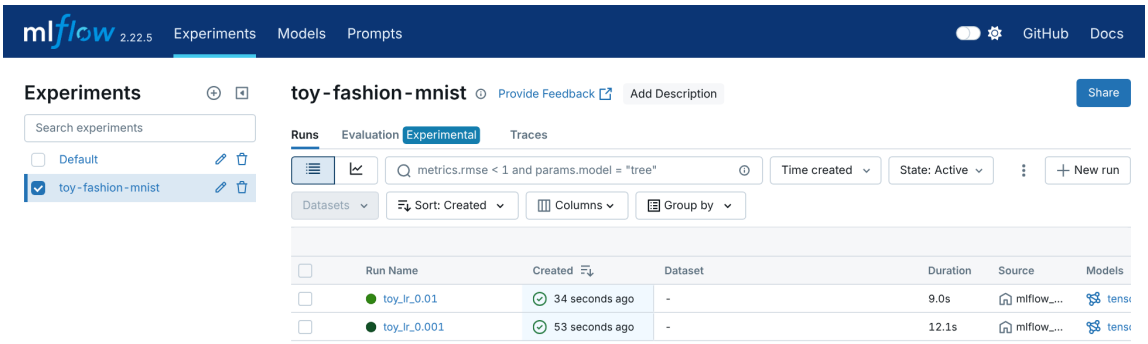


Figure 3.1: Local MLflow UI: the toy experiment with both runs.

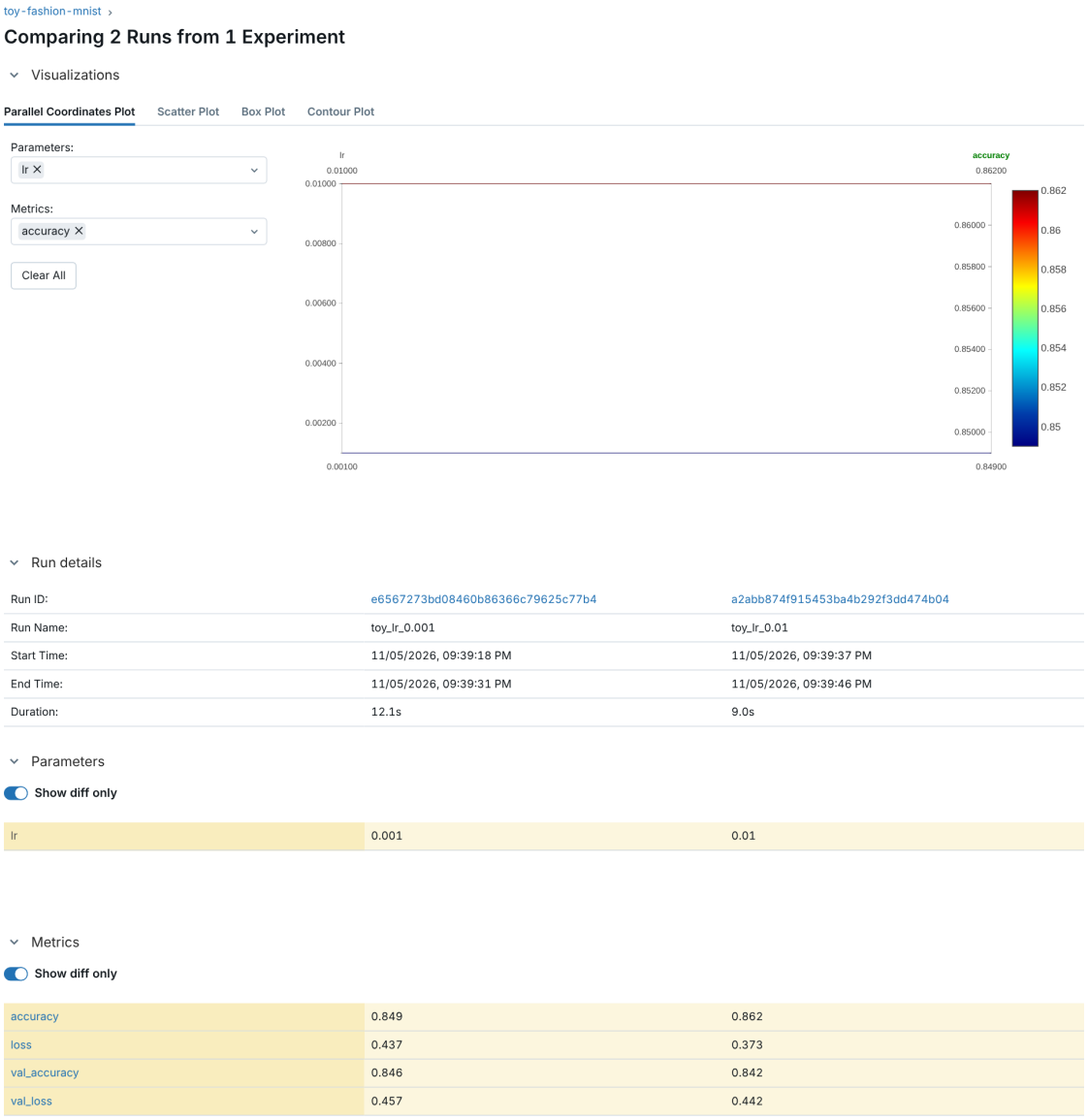


Figure 3.2: Comparison of the two learning rates.

Definitions in my own words.

Experiment A named folder of runs that try to solve the same problem, so they can be compared side by side (here `VisionGuard` and `toy-fashion-mnist`).

Run One execution of training code, with a unique id, start/end time and status; everything logged during it belongs to it.

Parameter An input setting that is fixed for the run and written once, e.g. the learning rate. Parameters are immutable: logging a different value for the same key is an error.

Metric A numeric result that may be logged many times with a step, e.g. `val_loss` per epoch, which is how MLflow draws curves.

Artifact Any output file stored with the run: plots, CSVs, the requirements list, the model itself.

Tag Free-form key/value metadata describing the run rather than configuring it, e.g. the git commit, the data version or who ran it; tags can be changed later and are used for search.

3.2 DagsHub setup

The DagsHub project `23L-0764_A2_Ops` was created with *Connect a repository*, so it mirrors my GitHub repository and adds DVC storage and a hosted MLflow server. Its tracking address is `https://dagshub.com/Anasnaveed69/23L-0764_A2_Ops.mlflow`. The code never contains it: `tracking.py` calls `load_dotenv()` and reads `MLFLOW_TRACKING_URI`; MLflow itself reads `MLFLOW_TRACKING_USERNAME` and `MLFLOW_TRACKING_PASSWORD` (the DagsHub token) from the environment. Those live in `.env`, which is listed in `.gitignore` and `.dockerignore`; `.env.example` documents the variable names with dummy values. If `MLFLOW_TRACKING_URI` is missing the code falls back to a local `mlruns/` folder, so nothing breaks offline.

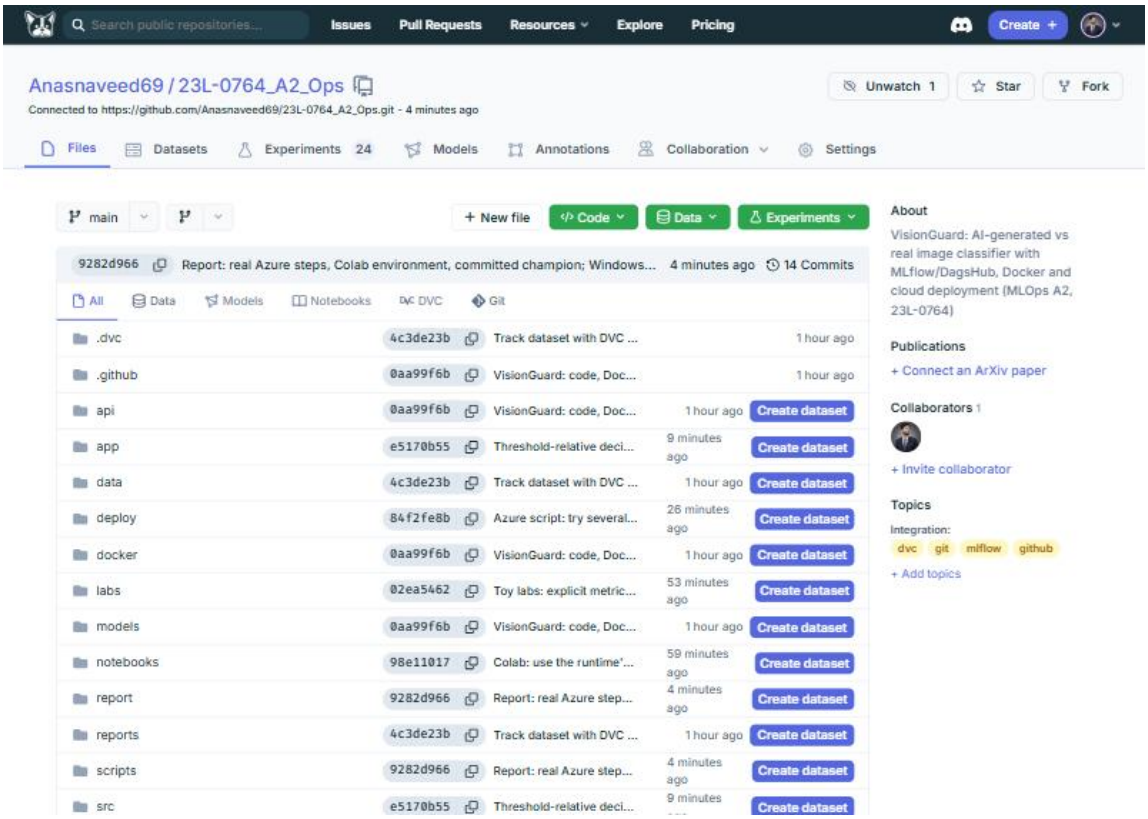


Figure 3.3: The DagsHub project connected to GitHub.

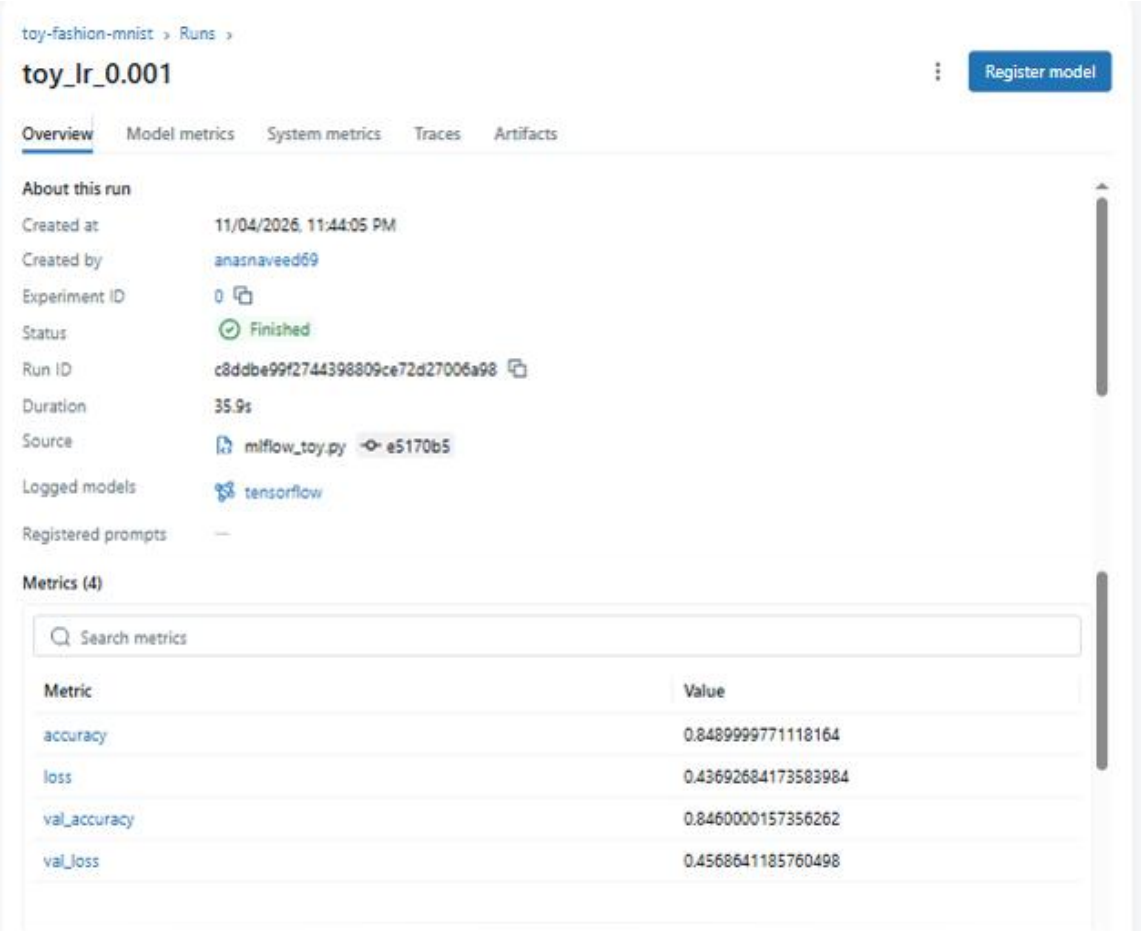


Figure 3.4: The toy run logged to DagsHub's MLflow server.

```
23L-0764_A2_Ops — .env is never committed

$ git check-ignore -v .env
.gitignore:2:.env□.env
$ git status --short --ignored | grep -E "\.env|mlruns|data/raw"
!! .env
!! mlruns_toy/
$ git ls-files | grep -c "^\.env$"
0
$ grep -n "env" .gitignore
2:.env
3:.env.*
4:!.env.example
26:.venv/
27:venv/
```

Figure 3.5: Proof that `.env` is git-ignored: `git check-ignore -v .env` names the rule and `git status` does not list it.

3.3 Instrumenting VisionGuard training

Every run of `scripts/train.py` logs:

- **Parameters**: every value of `params.yaml` actually used (flattened, e.g. `train.lr_head`), plus split sizes and class weights.
- **Per-epoch metrics**: loss, accuracy, AUC, precision, recall for train and validation, and the learning

rate (so ReduceLROnPlateau is visible).

- **Final metrics**: `best_epoch`, the chosen `val_threshold`, all validation and test metrics at that threshold (`test_accuracy`, `test_f1`, `test_roc_auc`, `test_false_positive_rate`, confusion-matrix cells) and test accuracy/F1 at the naive 0.5 threshold for comparison.
- **Tags**: `git_commit`, `git_dirty`, `dvc_data_version` (the md5 from `data/raw.dvc`), `seed`, Python/TensorFlow/Keras versions, platform, model type, backbone and tracking mode.

Manual approach (`-tracking manual`, e.g. run `mnv2_default`): my callback `MLflowEpochLogger` sends `logs` to `mlflow.log_metrics` at the end of every epoch with a global step, so the two training stages form one continuous curve.

Autologging (`-tracking autolog`, run `mnv2_autolog`): `mlflow.tensorflow.autolog()` patches `model.fit()`.

	Manual callback	Autologging
Captures	exactly what I choose: per-epoch metrics + LR, my params, my tags	every `fit()` argument (epochs, batch size, class weight…), all optimizer settings (`opt_*`), per-epoch metrics, model summary, EarlyStopping info, with no code
Misses	anything I forget to log	test metrics, chosen threshold, git/DVC tags, the confusion matrix, data splits: it only sees `fit()`, not evaluation or provenance
Multi-stage training	one continuous run	a second `fit()` tries to re-log `epochs` with a new value and params are immutable, so I put stage 2 in a nested child run
Noise	concise	many parameters irrelevant to the experiment

Table 3.1: Manual logging versus autologging.

In practice the two complement each other: autolog for the training internals, explicit logging for provenance and evaluation. Both kinds of run were produced from the same configuration.

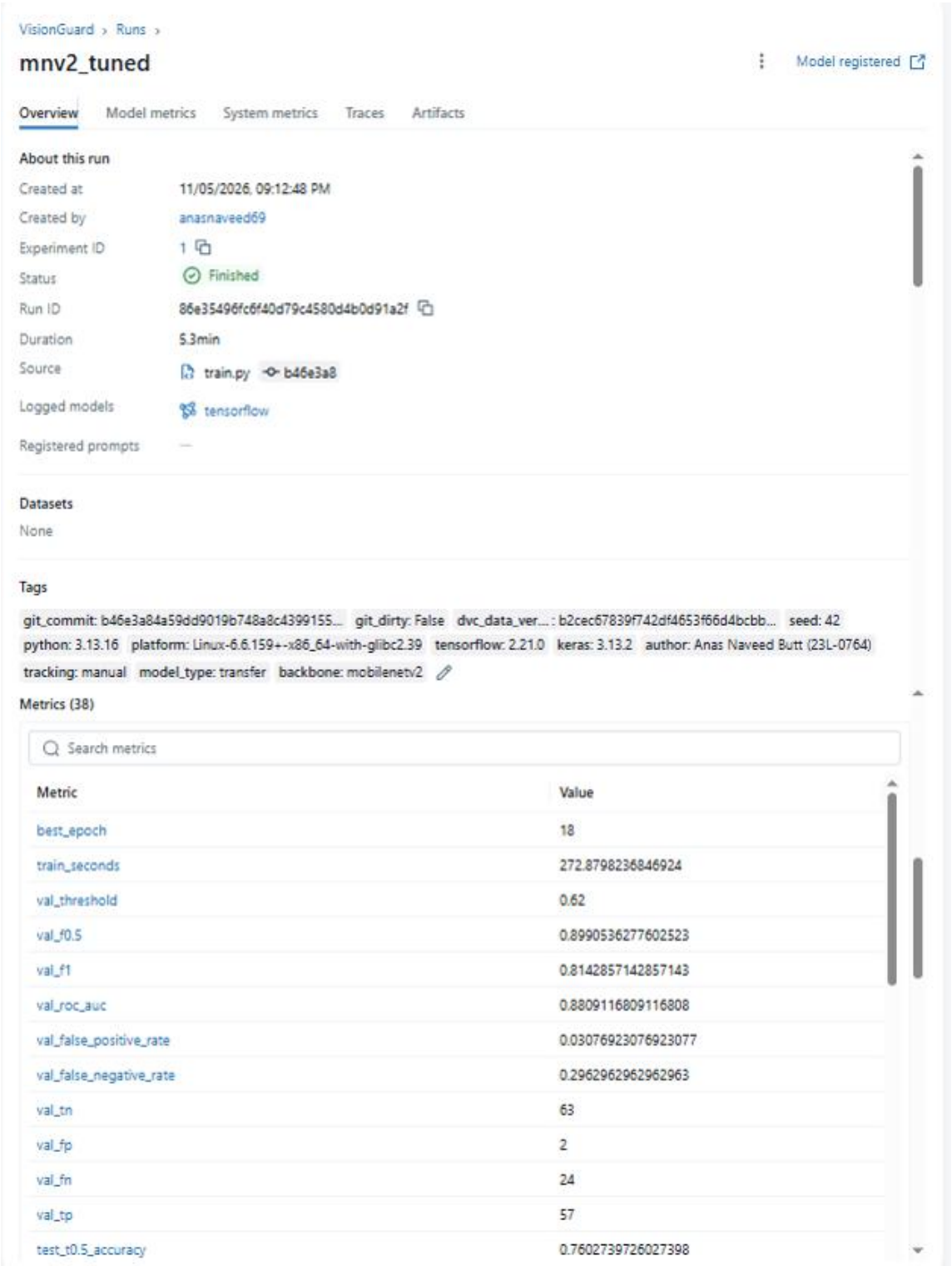


Figure 3.6: Manual run: parameters, metric curves and provenance tags.

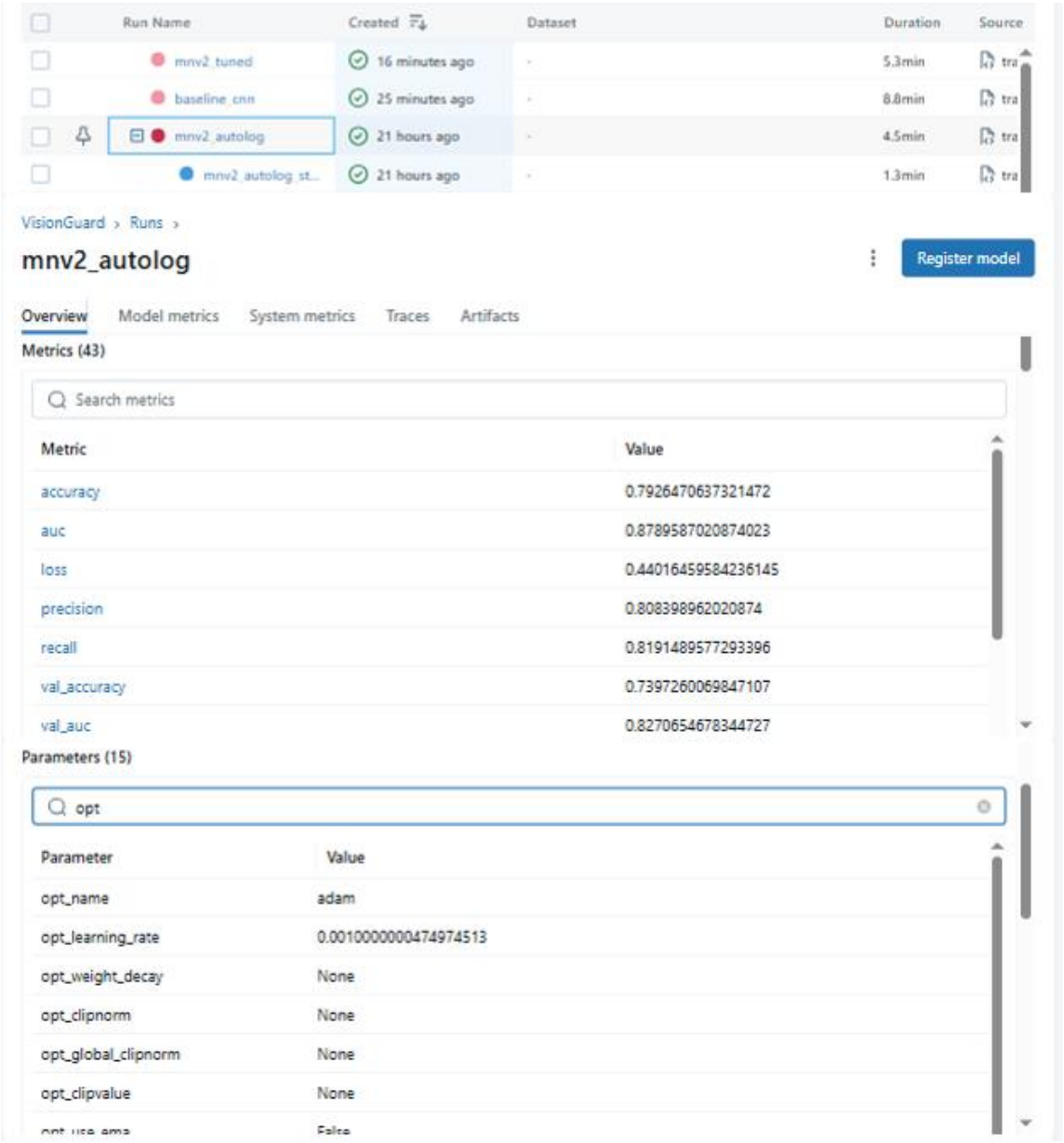


Figure 3.7: Autologged run: the automatically captured `fit()` and optimizer parameters.

What went wrong

(1) My first metric name `test@0.5_accuracy` was rejected: MLflow metric names may only contain letters, digits, `_ - . / :` and spaces; renamed to `test_t0.5_accuracy`. (2) The first autolog run logged *no* epoch metrics: autolog tried to log `class_weight`, which I had already logged under the same key with a different string format, and since parameters are immutable the autolog hook aborted. I renamed my parameter to `class_weights` and moved the second `fit()` into a nested run.

3.4 Artifacts

Each run stores: `plots/` (training curves, confusion matrix, ROC curve), `splits/` (the exact train/val/test CSVs used), `environment/` (`requirements-freeze.txt` from `pip freeze` and `params.yaml`), `evaluation/test_predictions.csv`, `model_info/` (model summary, trainable layers per stage), `keras_model/` (the `model.keras` file and `model_meta.json` with image size, resize mode and threshold) and `model/` (the MLflow TensorFlow-flavour model with an input/output signature, used by the registry).

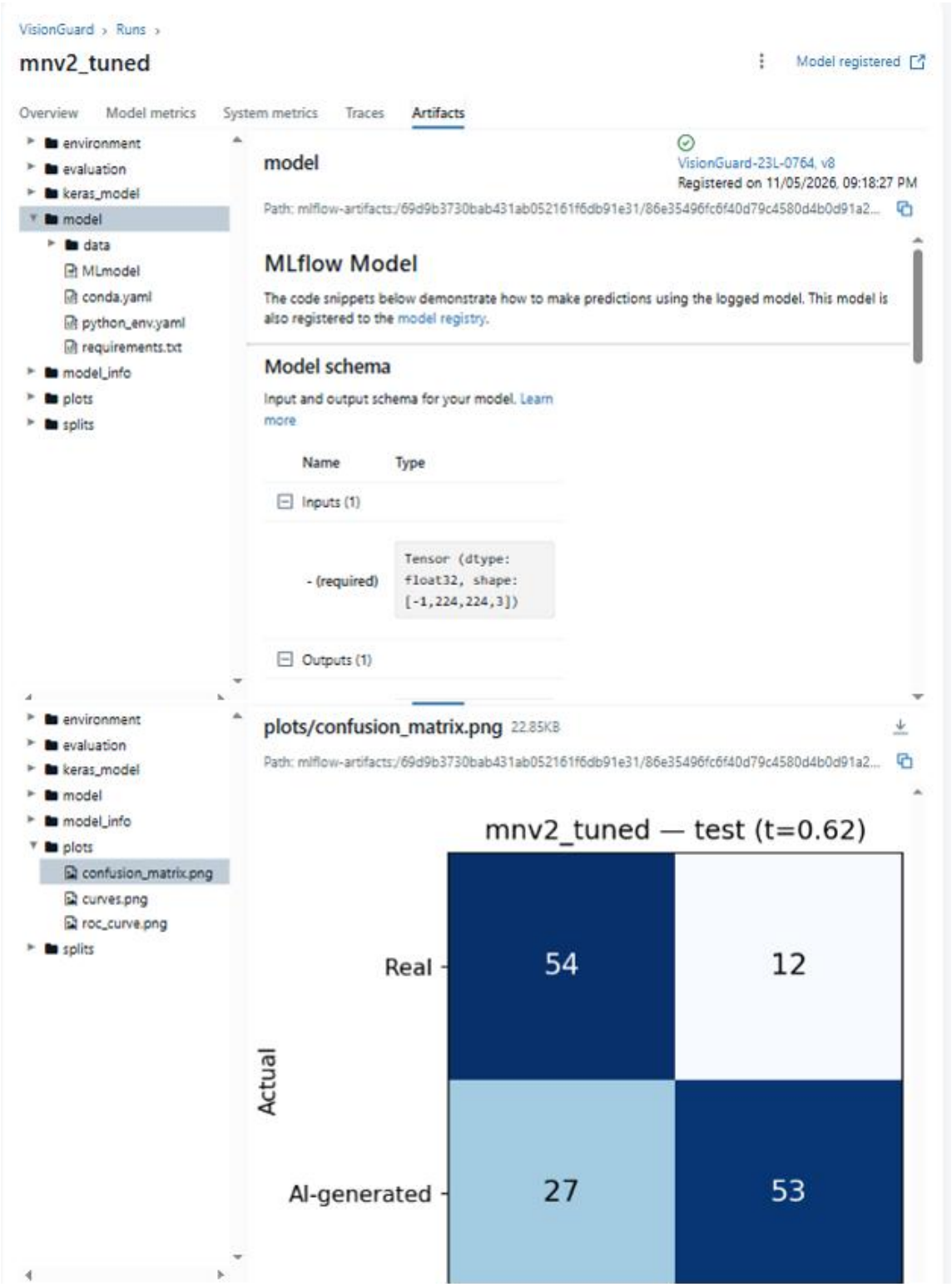


Figure 3.8: Artifacts tab of the champion run.

3.5 Experiment comparison

`scripts/run_experiments.py` launches the reference run and variants that each change *one* thing relative to it, so differences can be attributed to that change. `scripts/compare_runs.py` queries all finished runs from MLflow and builds the table and chart.

Run	Change vs reference	val AUC	val FPR	test acc	test F1	test AUC	best ep.
mnv2_tuned	tuner's best: lr 2.2e-3, dropout 0.1, 60 layers unfrozen	0.881	0.031	0.733	0.731	0.844	18
mnv2_no_aug	augmentation off	0.859	0.031	0.678	0.618	0.814	18
mnv2_lr_1e-4	head lr 1e-3 -> 1e-4	0.835	0.046	0.651	0.598	0.760	22
mnv2_default	reference (MobileNetV2, lr 1e-3, aug, 224, last block)	0.830	0.092	0.685	0.676	0.754	13
mnv2_autolog	same as reference, autolog tracking	0.830	0.123	0.678	0.671	0.764	13
mnv2_frozen	no fine-tuning (all frozen)	0.822	0.062	0.678	0.647	0.733	7
effb0_default	backbone -> EfficientNetB0	0.805	0.062	0.685	0.629	0.852	22
mnv2_img160	image size 224 -> 160	0.764	0.292	0.747	0.758	0.793	18
baseline_cnn	model: CNN from scratch	0.757	0.123	0.651	0.598	0.743	38

Table 3.2: All VisionGuard runs (sorted by validation ROC-AUC).

Figure 3.9: Validation ROC-AUC and test F1 per run.

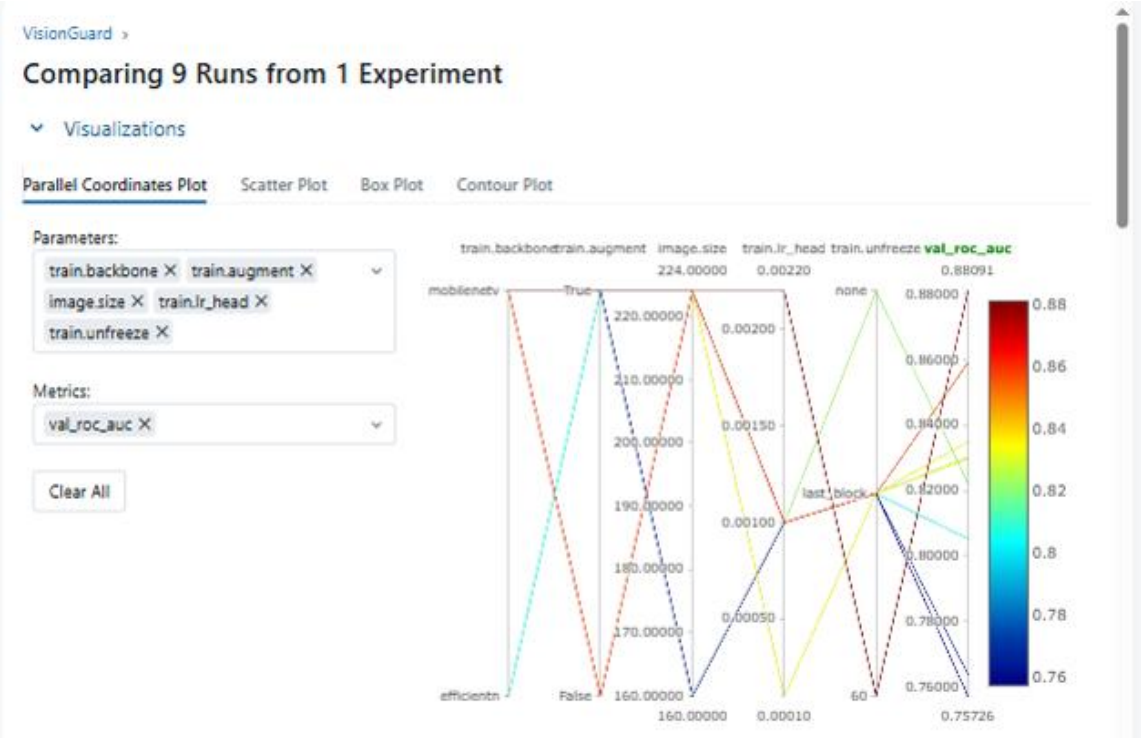


Figure 3.10: MLflow/DagsHub run comparison view.

Selection rule. I choose by **validation ROC-AUC** (threshold-free: it measures how well the model ranks AI above real, before any threshold), breaking ties by validation false-positive rate because false accusations are the costly error. Test metrics are shown but never used to choose, otherwise the test set would become a second validation set. **Chosen:** `mnv2_tuned` (validation AUC 0.881, test AUC 0.844, test accuracy 0.733, test F1 0.731).

Interpretation. Starting the transfer stage with the tuner's settings helped most (validation AUC 0.830 → 0.881, test F1 0.676 → 0.731). Turning augmentation *off* also improved validation AUC (0.859): on this small dataset the random flips, rotations and zoom mostly slowed learning, and the fine generator textures the model relies on are changed by zoom and rotation. The smaller input (160 px) hurt most on the selection metric (validation AUC 0.764, validation FPR 0.29), because down-scaling erases fine texture; its higher test accuracy is noise on 146 images, and the selection rule ignores it on purpose. EfficientNetB0 had the best test AUC (0.852) but a lower validation AUC, so it was not chosen. The autolog run matches the manual reference (validation AUC 0.830 for both; test AUC 0.764 vs 0.754 because GPU training is not bit-exact).

What went wrong

My first comparison ranked runs by the metric `val_auc`, which is the Keras AUC of the *last* epoch logged by the callback, not of the best checkpoint that is actually evaluated. It also did not exist in autologged runs under the same meaning. I switched to `val_roc_auc`, computed by my evaluation code on the reloaded best model for every run.

3.6 Model registry

`scripts/register_model.py` makes sure the registered model `VisionGuard-23L-0764` has at least two versions (the reference transfer run and the baseline CNN are registered at training time with `-register`; the selected run is registered if it is not already), gives the best version the alias `champion` and the runner-up the alias `challenger`, and writes a description and a `val_auc` tag. An alias is a movable name: promoting a new model means moving the alias, and every consumer that loads `@champion` follows it without code

changes.

Version	Run	Alias	val AUC	test acc
1	mnv2_default		0.830	0.685
2	baseline_cnn		0.393	0.548
3	mnv2_no_aug		0.859	0.678
4	mnv2_lr_1e-4		0.835	0.651
5	baseline_cnn		0.751	0.664
6	mnv2_tuned		0.852	0.705
7	baseline_cnn		0.757	0.651
8	mnv2_tuned		0.881	0.733

Table 3.3: Registered versions and aliases.

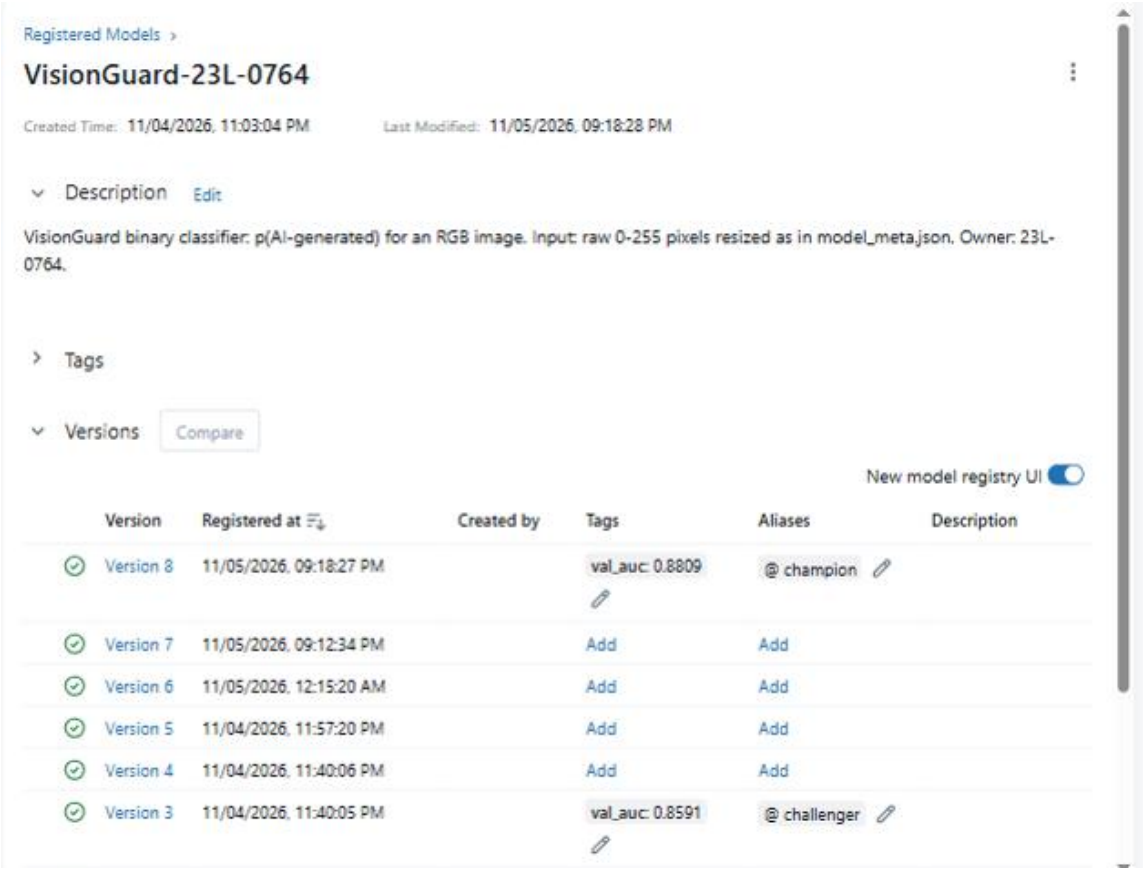


Figure 3.11: Model registry on DagsHub with two versions and the `champion` alias.

`scripts/load_champion.py` is a separate script that resolves the alias with `MlflowClient().get_model_version_by_alias()`, loads that version with `mlflow.tensorflow.load_model("models:/VisionGuard-23L-0764/<v>")`, reads the threshold and image size from the version's `model_meta.json`, and classifies one image.

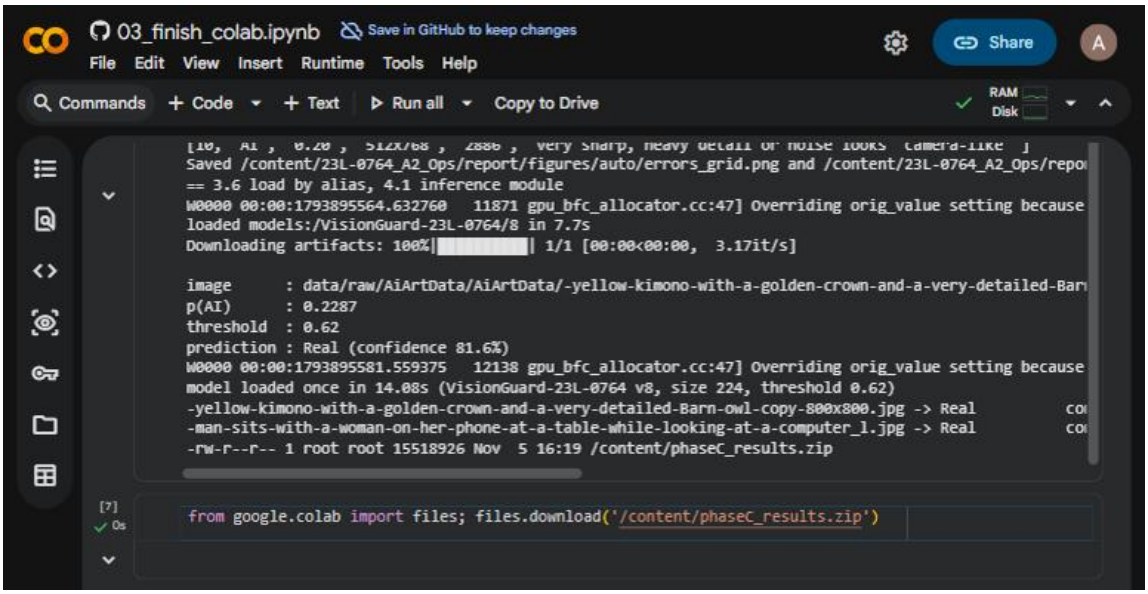


Figure 3.12: Loading the model by alias and classifying one image.

How the champion reaches the server. The serving processes must not call DagsHub per request. `scripts/export_champion.py` resolves `@champion` *once*, downloads `model.keras` and writes `models/champion.keras` plus `champion.json` (version, threshold, image size). It re-saves the model without optimizer state (serving never trains), which makes it 9.6 MB. That exported copy is committed to Git (it is small, inference-only and regenerable from the registry, which stays the source of truth), so CI and the cloud VM can build the API image without registry credentials. Locally the `models/` folder is mounted read-only into the web container; for the cloud API the file is copied into the image at build time (Chapters 6 and 7).

4 | Flask Web Application

4.1 Inference module

`src/visionguard/inference.py` defines `Predictor`. Its constructor runs once per process: it loads `champion.keras` with `keras.models.load_model`, reads `champion.json` (image size, resize mode, threshold, version), and runs one dummy prediction to warm up the TensorFlow graph. `predict_bytes(data)` decodes the bytes, resizes them, predicts $p(\text{AI})$, applies the threshold and returns label, confidence (probability of the predicted label), $p(\text{AI})$, threshold, model name/version and latency.

Why not load per request. Loading deserialises millions of weights and builds the graph: it takes seconds and hundreds of MB. Per request, every user would wait for it, memory would spike with concurrent requests, and the first predict of a fresh model is the slowest one. Loading once and keeping the model in memory makes each prediction tens of milliseconds.

Why training and serving preprocessing must match. The model only learned the mapping from *the inputs it saw*. If serving resized with a different interpolation, kept the alpha channel, scaled to a different range or preserved the aspect ratio when training did not, the inputs would come from a different distribution and predictions would silently degrade, with no error raised (*training/serving skew*). I removed the possibility by construction: training, audit and serving call the same `decode_and_resize` function, the image size and resize mode travel with the model in `champion.json`, and backbone-specific scaling is a layer inside the saved model.

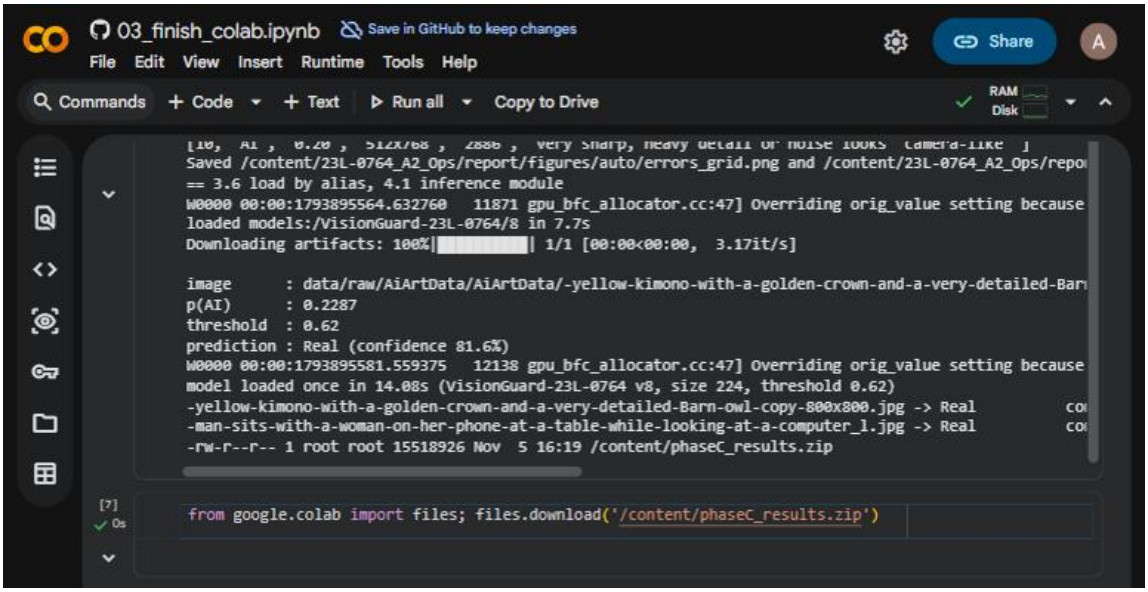


Figure 4.1: `scripts/infer_cli.py`: model loaded once, then two known test images (one AI, one real).

4.2 Routes and validation

Route	Method	Purpose
/	GET	upload page (drag & drop, client-side preview, last 5 checks)
/predict	POST	validate, predict, write DB row, redirect to the result (Post/Redirect/Get)
/result/<id>	GET	verdict, confidence, $p(\text{AI})$ vs threshold, caveat, audit record
/history	GET	paginated database trail, filter by verdict, show rejected uploads
/api/history	GET	the same trail as JSON
/health	GET	JSON health check: DB reachable, model backend reachable, model version

Validation (`src/visionguard/validation.py`), cheapest check first:

1. No file → 400.
2. Extension not in {jpg, jpeg, png, webp, bmp, gif} → **415 Unsupported file type**.
3. Size: Flask's `MAX_CONTENT_LENGTH` rejects oversized bodies before my code runs, and the stream is read with a hard limit of 5 MB + 1 byte → **413 File too large**.
4. Full decode with Pillow (verify + load) → **422 Not a readable image** for renamed non-images or truncated files. Pillow's decompression-bomb limit is also set.

File names go through `werkzeug.utils.secure_filename` (e.g. `../../etc/passwd.png` becomes `etc_passwd.png`) and are only stored as text. The upload is read into memory and **never written to disk**: only its SHA-256 hash, size and dimensions are kept. Every error renders a friendly page with the status code, a plain explanation and a way back; unexpected exceptions are logged with a traceback but the user only sees a generic 500 page.

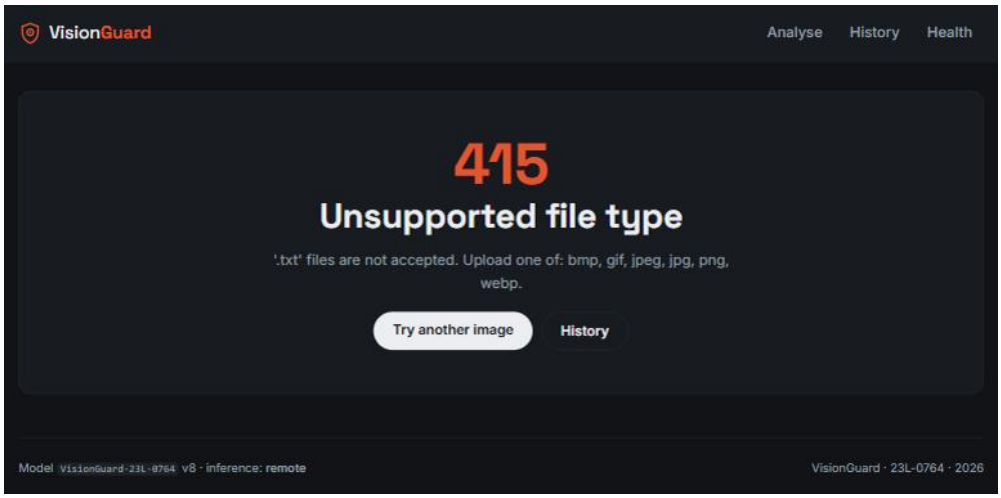


Figure 4.2: Failure case 1: wrong file type (415).

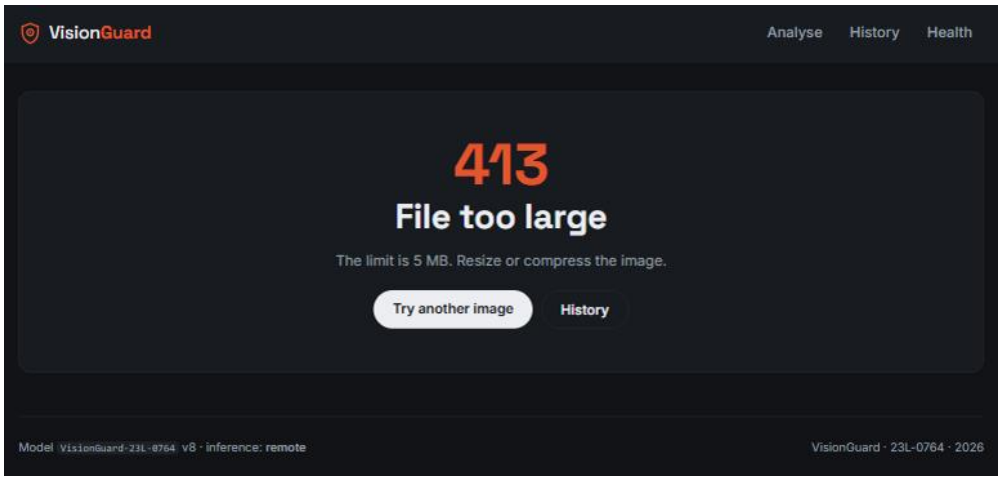


Figure 4.3: Failure case 2: file over the size limit (413).

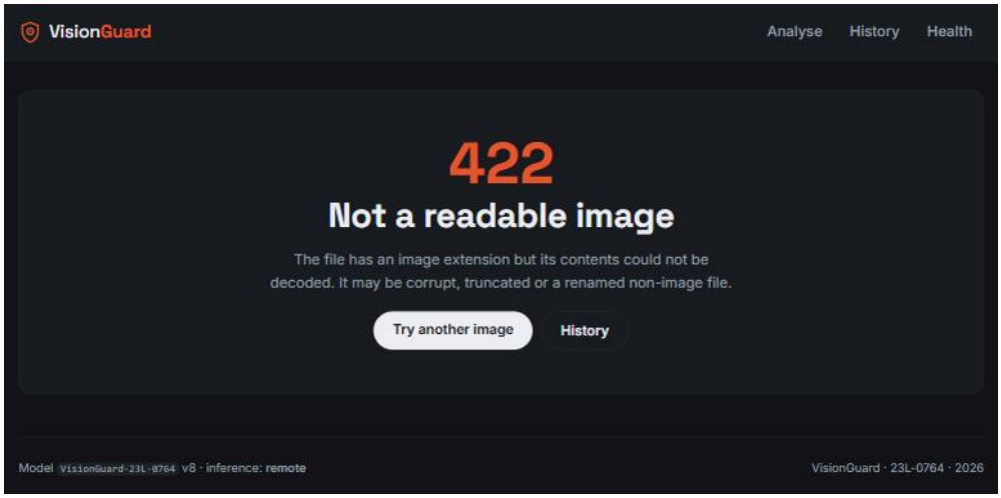


Figure 4.4: Failure case 3: a text file renamed to `.jpg` (422).

4.3 HTML and CSS

Templates use Jinja inheritance (`base.html` holds the header, navigation with the History menu, and the footer showing the model version and inference mode). The result page shows the verdict in the class colour, a confidence bar, a scale with the decision threshold and the model's $p(\text{AI})$, a "low certainty" badge under 65%, and an honest caveat box: the result is a statistical estimate, weakest on screenshots, heavy compression and newer generators, tuned to avoid false accusations, and never sufficient on its own. The stylesheet is plain CSS with a dark mode and a mobile layout.

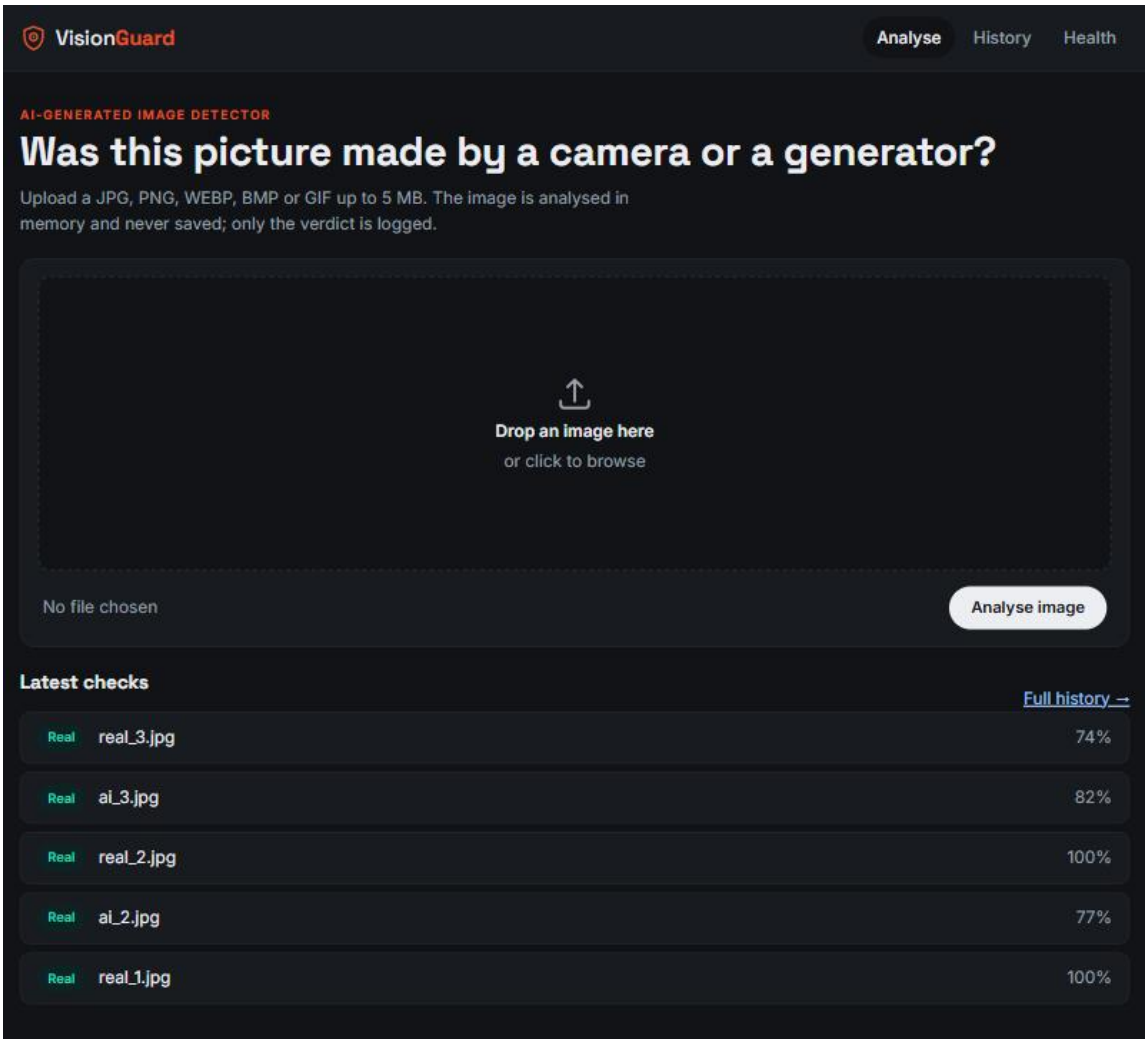


Figure 4.5: Upload page.

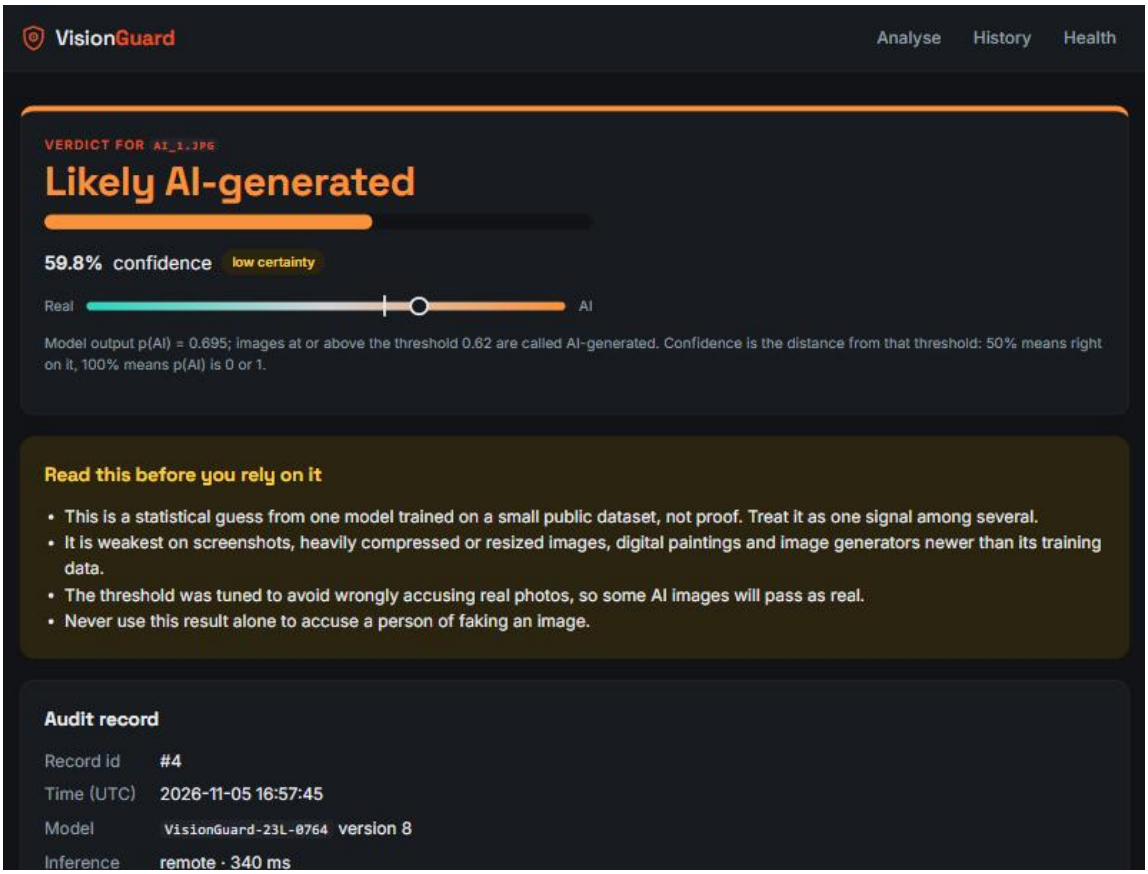


Figure 4.6: Result page with verdict, confidence, threshold scale and caveat.

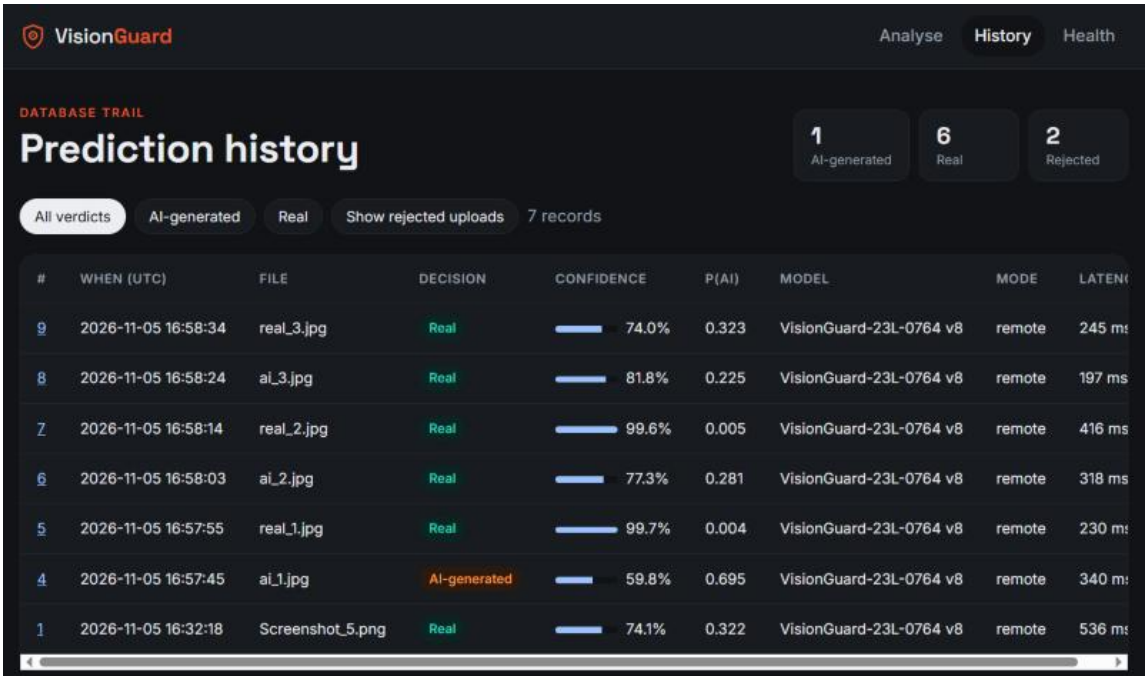


Figure 4.7: History menu: the database trail.

What went wrong

With the threshold chosen on validation data below 0.5 (e.g. 0.47), an image with $p(\text{AI}) = 0.49$ is labelled AI-generated with a "confidence" of 49%, which looks contradictory. The number is honest (it is the model's probability for that label), so instead of hiding it the page now shows the threshold scale and a "low certainty" badge.

4.4 Configuration and production server

All settings are environment variables read in `app/config.py`: `MODEL_PATH`, `THRESHOLD` (optional override), `DATABASE_URL`, `INFERENCE_MODE` (`local`/`remote`), `MODEL_API_URL`, `MODEL_API_KEY`, `MAX_UPLOAD_MB`, `SECRET_KEY`, `FLASK_DEBUG`. The same image therefore runs on my laptop, in Compose and against the cloud API without code changes, and no secret is in the code or the image.

In the container the app is served by **gunicorn** (1 worker, 4 threads), not by `flask run`: Flask's built-in server is single-process, not hardened, and meant for development. Debug mode is **off** (`FLASK_DEBUG=0`, visible as `"debug": false` in `/health`) because the Werkzeug debugger lets anyone who triggers an error execute arbitrary Python in the browser, and debug pages leak source code and configuration. One worker keeps one copy of the model in RAM; threads overlap waiting on the database and the network.

Browser — http://127.0.0.1:5000/health

GET http://127.0.0.1:5000/health (Flask app on my laptop)


```
{
  "database": "ok",
  "debug": false,
  "inference_mode": "remote",
  "model": {
    "api": "https://20-219-10-43.sslip.io",
    "model_name": "VisionGuard-23L-0764",
    "model_version": "8",
    "threshold": 0.62
  },
  "model_backend": "ok",
  "status": "ok",
  "time": "2026-11-05T16:59:49+00:00"
}
```


Figure 4.8: `/health` response: database ok, model backend ok, debug false.

5 | Database Trail

What and why. Every request leaves a permanent row answering: *what* was decided, *when*, *by which model*, and *how confidently*. I use **PostgreSQL** in the Compose stack (durable, concurrent writes, a named volume) and SQLite for local runs and unit tests; the code is the same because it goes through SQLAlchemy and only the `DATABASE_URL` changes.

Column	Type	Meaning
`id`	integer PK	record number, shown on the result page
`created_at`	timestamptz (indexed)	UTC time of the request (*when*)
`status`	varchar	`ok`, `rejected` (failed validation) or `error` (model unreachable)
`filename`	varchar	sanitised upload name
`image_sha256`	char(64)	identifies the image without storing it; repeated uploads are detectable
`width, height, size_kb`	int/float	basic properties of the upload
`label`	varchar (indexed)	the decision: AI-generated / Real (*what*)
`confidence, prob_ai, threshold`	float	*how confidently*, and the rule that produced the label
`model_name, model_version`	varchar	*which model* (registry name and version)
`inference_mode`	varchar	`local` (in-process) or `remote` (cloud API)
`latency_ms`	float	prediction or round-trip time
`error`	varchar	reason for a rejected or failed request

Table 5.1: Table `predictions`.

How it is attached to the views. `app/db.py` creates the engine once at start-up (with a retry loop in case the database is still starting) and creates the table if needed. In `/predict`, after the model returns, one row is inserted in its own transaction (`Session.begin()` commits or rolls back automatically) and the browser is redirected to `/result/<id>`, which reads that row back, so the result page itself proves the record exists. Rejected uploads and model failures are recorded too, so the trail is complete, not just the successes.

History menu. `/history` shows the newest records first, 20 per page, with totals per verdict, filters (AI-generated / Real) and a toggle to show rejected uploads. `/api/history` returns the same rows as JSON.

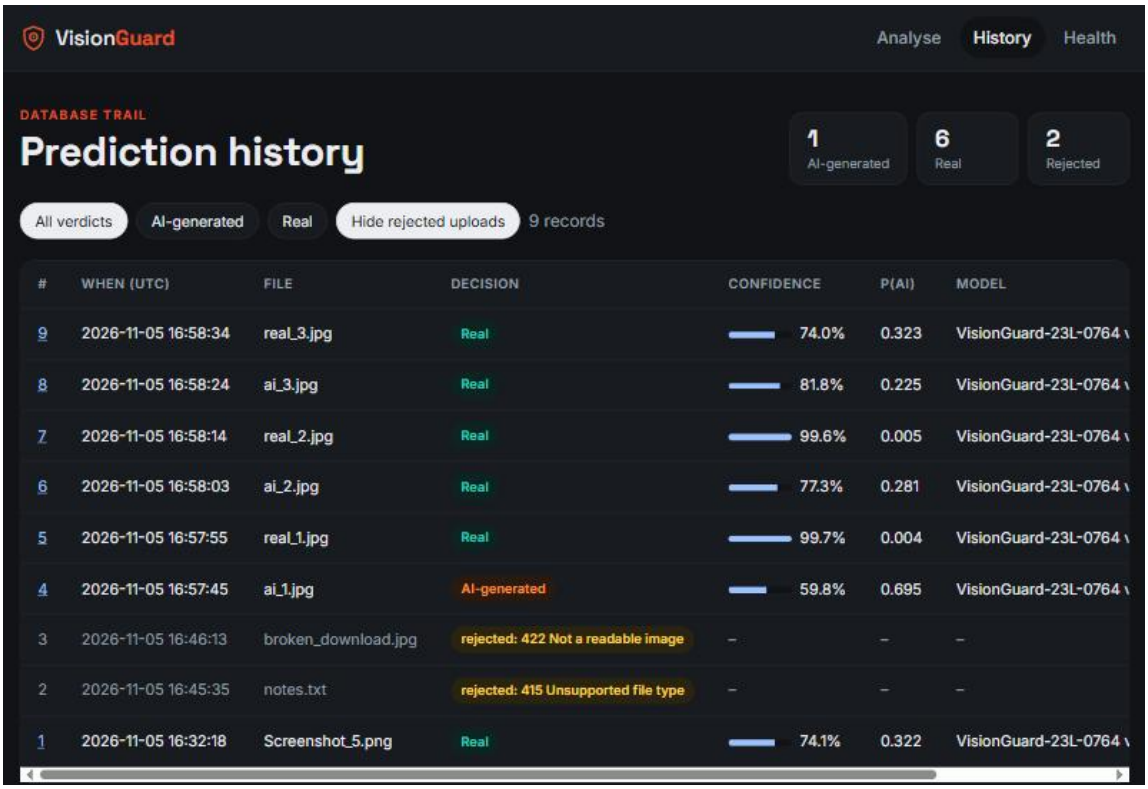


Figure 5.1: History page backed by PostgreSQL.

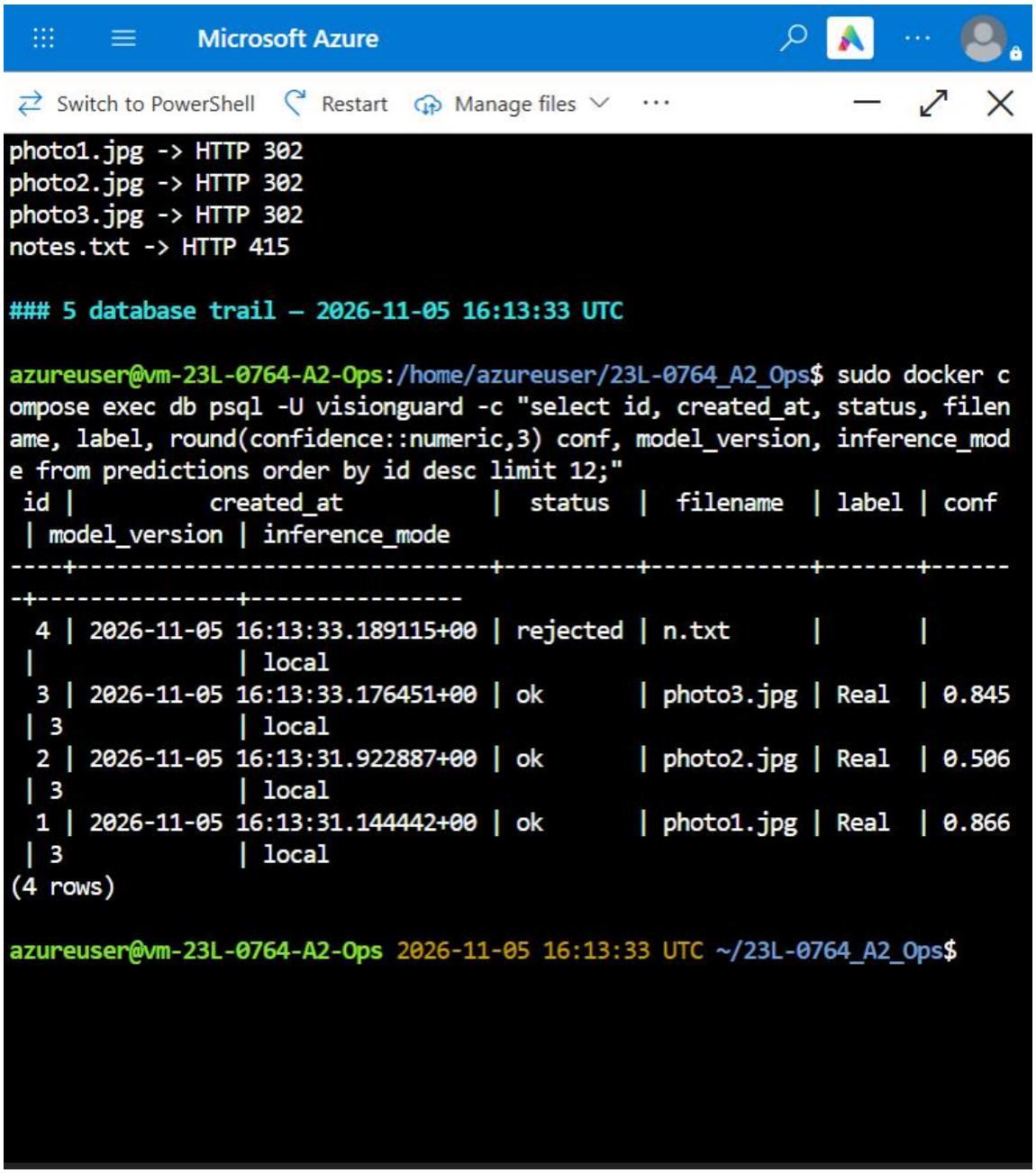


Figure 5.2: The same records queried directly in PostgreSQL inside the `db` container.

6 | Containerization with Docker and Docker Compose

Where this ran. All Docker work in this chapter was done on the project's Ubuntu 24.04 VM (`vm-23L-0764-A2-Ops`, Docker Engine 29.9, Compose v5.7) over SSH from Azure Cloud Shell; every screenshot shows the `azureuser@vm-23L-0764-A2-Ops` prompt with date and time. The same files work unchanged with Docker Desktop on Windows. GitHub Actions additionally builds and smoke-tests both images on every push (Section 6.6).

6.1 Docker warm-up

Before containerising my project I practised with the ready-made `nginx:1.27-alpine` image (`labs/docker_warmup.md`): `docker run -d -name warmup -p 8081:80 -v .../labs/warmup-site:/usr/share/nginx/html:ro nginx:1.27-alpine` runs it in the background (`-d`), maps host port 8081 to container port 80 (`-p`) and bind-mounts a folder with my own `index.html` read-only (`-v ...:ro`); the page appears at `http://localhost:8081`. `docker logs` shows nginx's access log for each refresh, `docker exec -it warmup sh` opens a shell inside the running container, `docker ps -a` lists containers, `docker stop`/`docker rm` remove the container and `docker images`/`docker rmi` the image.

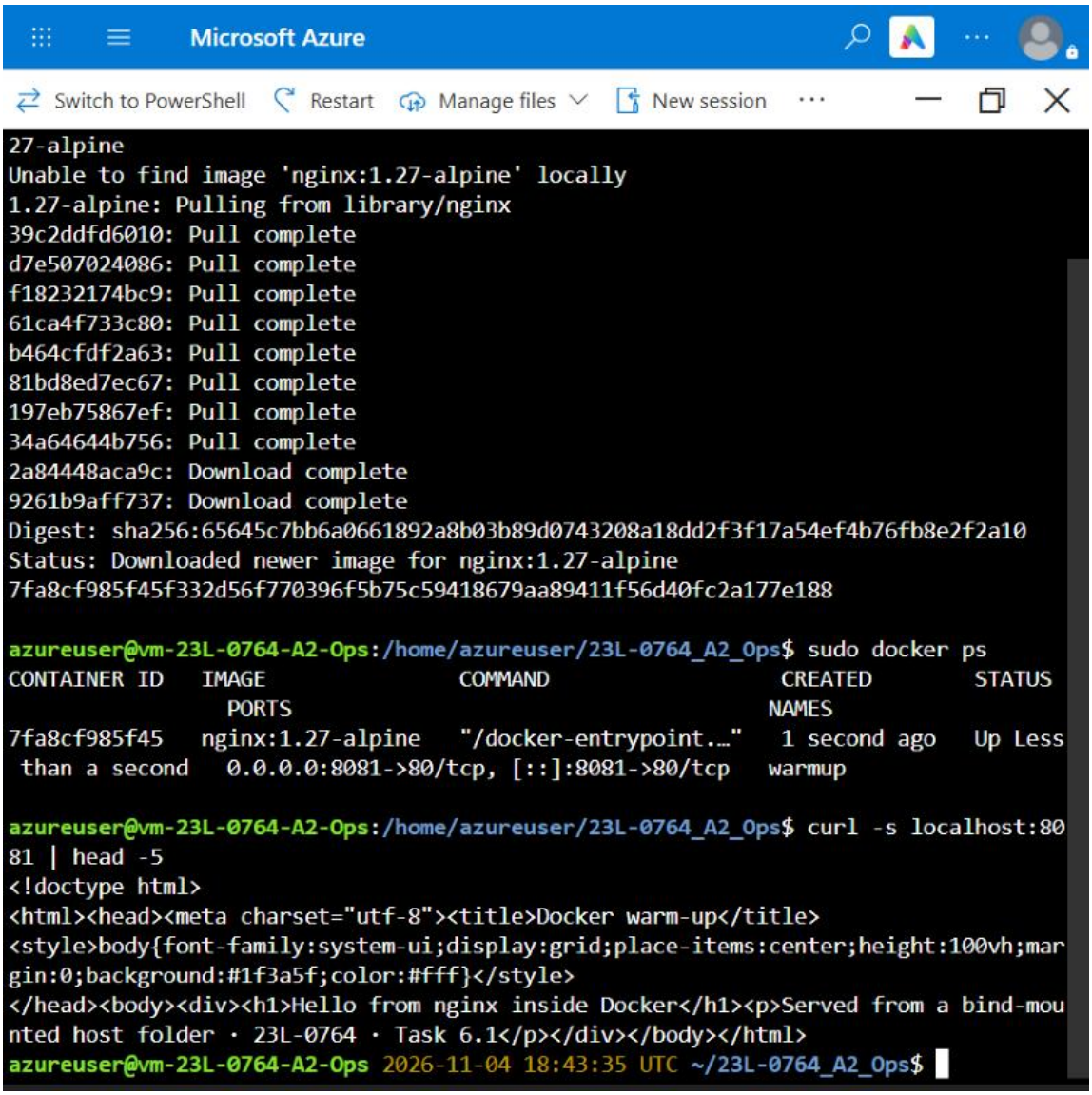


Figure 6.1: Run with port mapping and volume; page served in the browser.

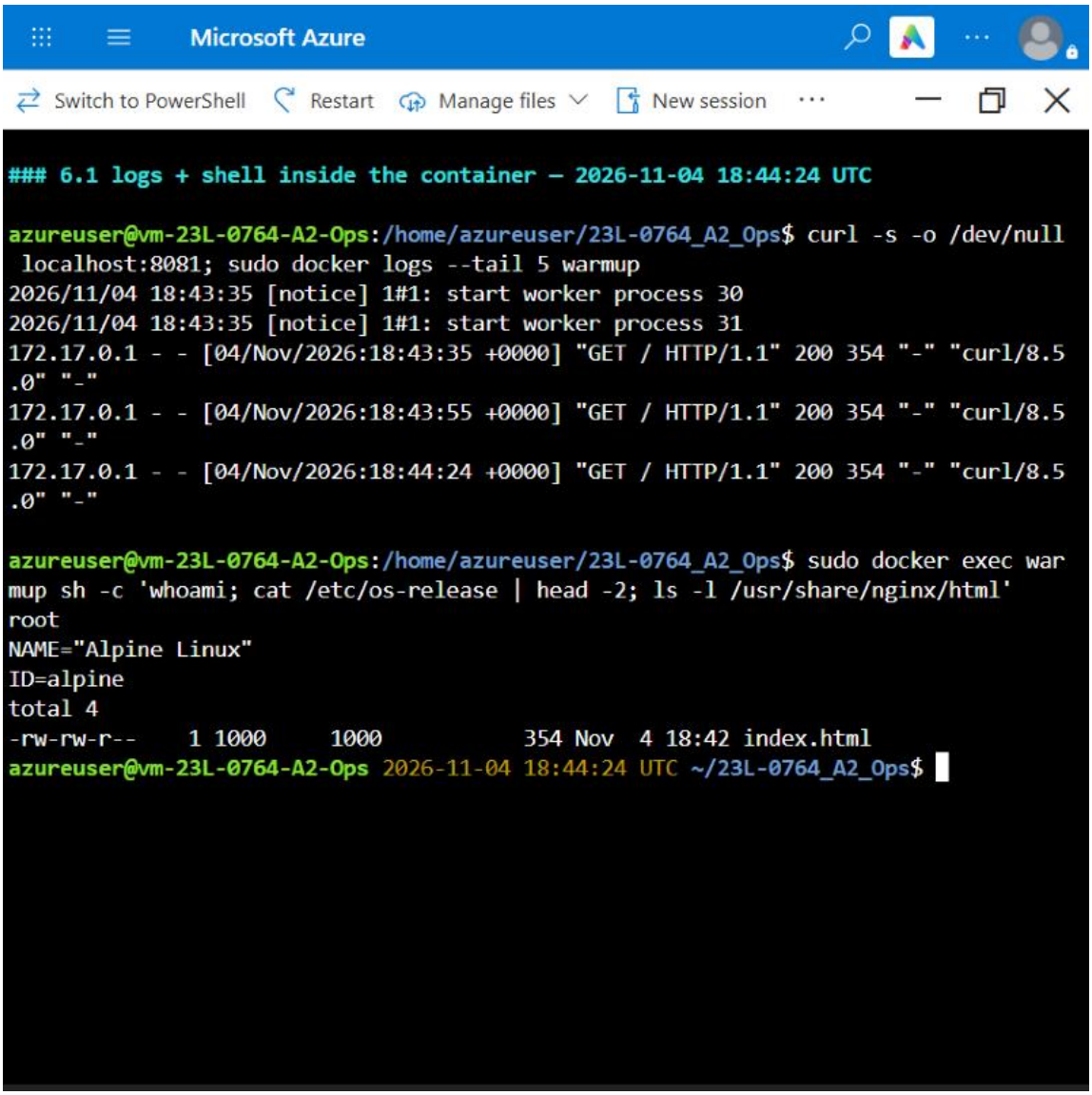


Figure 6.2: Logs and a shell inside the container.

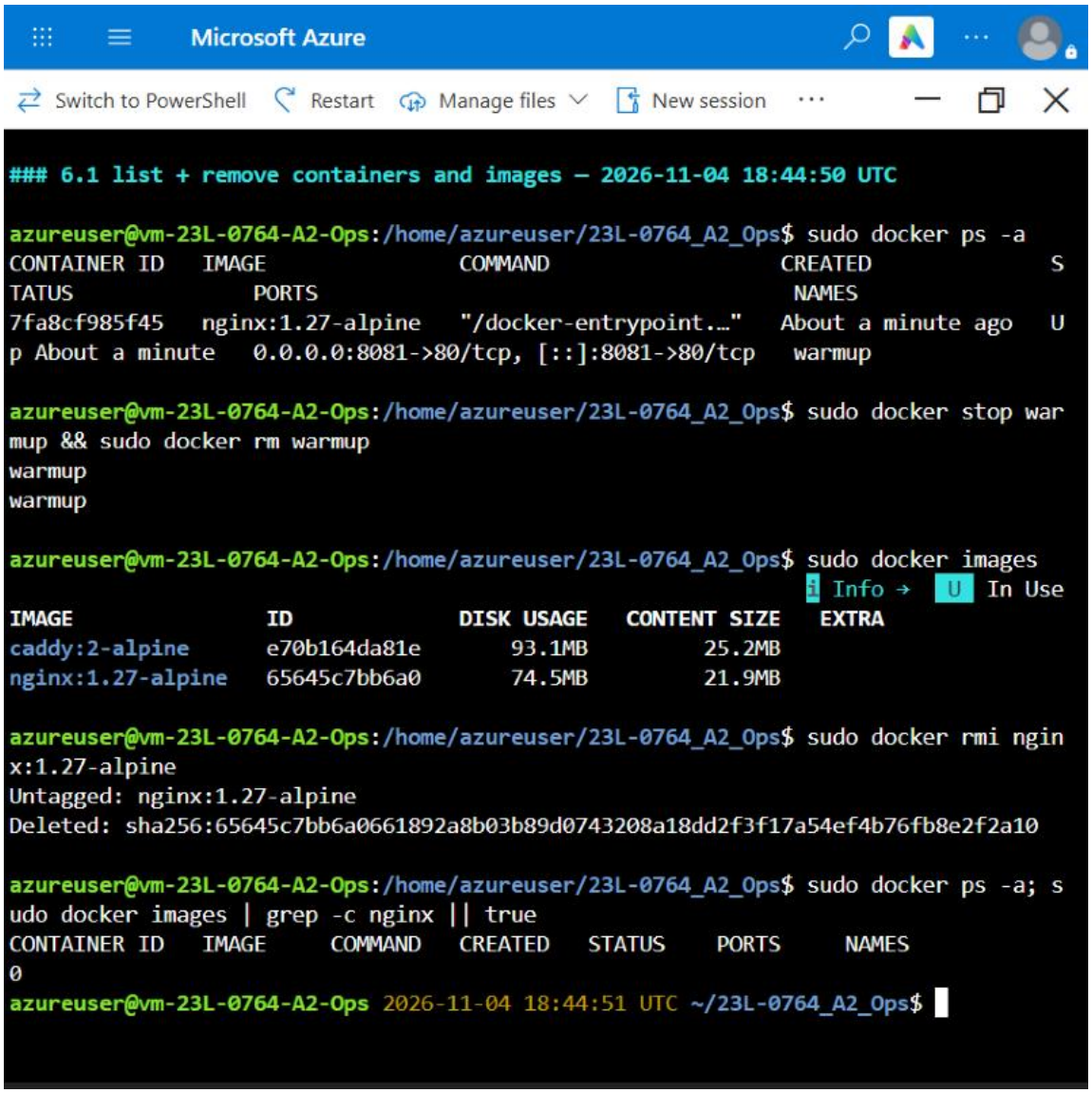


Figure 6.3: Listing and removing containers and images.

Image A read-only template: a file system plus metadata (command, environment, exposed ports) built from a Dockerfile. It does not run; it is the blueprint.

Container A running (or stopped) instance of an image: an isolated process with its own file system view, network and process space, plus a thin writable layer that disappears when the container is removed.

Layer Each Dockerfile instruction that changes the file system produces one immutable layer; an image is a stack of them. Layers are cached and shared between images, so an unchanged instruction is not rebuilt.

Volume Storage managed outside the container's writable layer that survives container removal; a bind mount maps a host folder instead.

Port mapping Forwarding a port on the host to a port inside the container's private network (`host:container`); without it the service is unreachable from outside.

6.2 Dockerfile for the web app

(`Dockerfile`) Order of instructions and why:

1. `FROM python:3.11-slim`: Debian slim without compilers or docs (∼130 MB instead of ∼1 GB for `python:3.11`); all my dependencies ship as wheels.
2. `ENV`: no `.pyc` files, unbuffered logs (they appear immediately in `docker logs`), `PIP_NO_CACHE_DIR=1` so the pip cache is not baked into a layer.
3. `COPY requirements-*.txt` then `RUN pip install`: **dependencies before source**. Docker reuses a cached layer as long as the instruction and the files it copies are unchanged. Requirements rarely change; code changes constantly. With this order, editing a template or a route reuses the cached ∼1 GB TensorFlow layer and rebuilds in seconds; copying the source first would invalidate the cache and reinstall everything on every edit.
4. `useradd vg` (uid 10001) and later `USER vg`: the app does not run as root, so a compromised process cannot modify the image's files or install software.
5. `COPY src app`: the code, owned by `vg`. The model is *not* copied (it is mounted, Section 6.3).
6. `HEALTHCHECK`: every 30 s Python requests `/health` (slim has no `curl`); Docker marks the container *unhealthy* after 3 failures.
7. `CMD gunicorn ...`: production WSGI server, exec form so gunicorn is PID 1 and receives `SIGTERM` for a clean shutdown.

Ignore file. `.dockerignore` excludes `.git`, `.env`, `data/`, `mlruns/`, `outputs/`, notebooks, tests and the report from the build context: the upload to the Docker daemon is small, and secrets and datasets can never end up in a layer. **Secrets** are only passed at run time as environment variables from the untracked `.env`; `docker history` of the image shows none.

TensorFlow package. I install `tensorflow-cpu==2.20.0`: the default `tensorflow` wheel on Linux bundles CUDA/cuDNN libraries (several GB) that a CPU-only server never uses, while a single MobileNetV2 prediction takes tens of milliseconds on a CPU. A build argument `WITH_TF=false` builds the web image without TensorFlow at all for `INFERENCE_MODE=remote`, where the model runs in the cloud API.

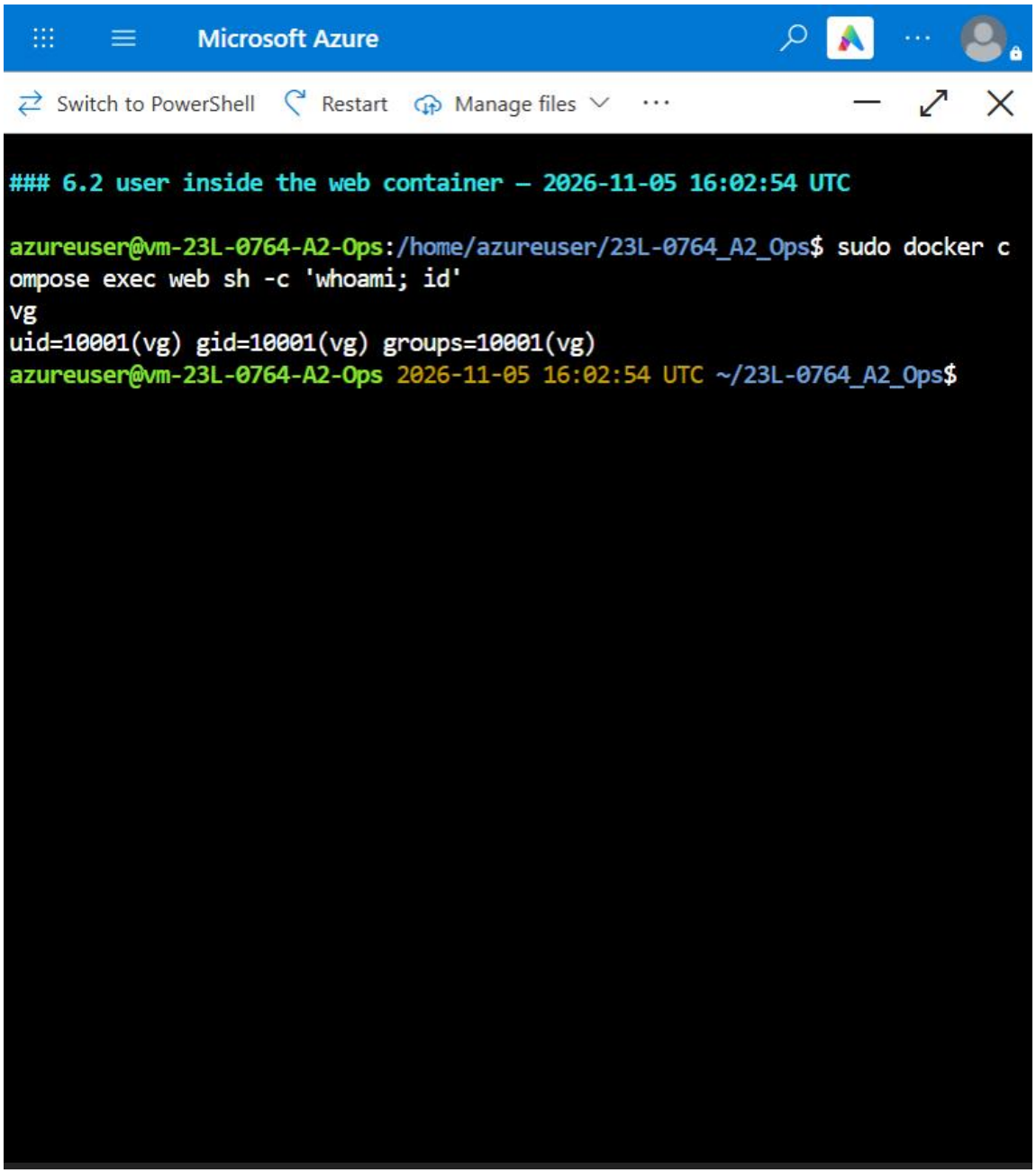


Figure 6.4: The process inside the container runs as the unprivileged user `vg` (uid 10001).

What went wrong

My first container started gunicorn with `-preload` (load the app in the master process before forking workers, so start-up errors appear immediately). Health checks passed, but the first prediction hung until gunicorn's timeout killed the worker. TensorFlow starts thread pools when it initialises; `fork()` copies only the calling thread, so the child inherited locks held by threads that no longer exist and deadlocked. Removing `-preload` makes the worker load the model itself after the fork. It is still loaded once per process.

6.3 Compose stack

(`docker-compose.yml`)

- **Two services**, `web` and `db` (`postgres:16-alpine`), on a user network `backend`. Compose's internal DNS resolves service names, so the web app connects to

`postgresql+psycopg2://...@db:5432/visionguard`: `db` is the service name, not an IP.

- **Named volume** `pgdata` holds the database files, so the history survives `docker compose down` and image upgrades (only `down -v` deletes it).
- **Readiness**: `db` has a health check (`pg_isready`) and `web` declares `depends_on: db: condition: service_healthy`, so the app starts only when PostgreSQL accepts connections (plain `depends_on` only waits for the container to *start*, which is too early). The app also retries its first connection.
- **No published database port**: `db` has no `ports:` entry, so it is reachable only from containers on `backend`, not from my machine or the network.
- **Secrets**: passwords come from the untracked `.env` (`${POSTGRES_PASSWORD:?}` makes Compose refuse to start if it is missing); `.env.example` is committed with placeholders.

How the model reaches the container: mounted. `./models` is bind-mounted read-only at `/app/models`. Promoting a new champion is then `python scripts/export_champion.py` and `docker compose restart web`; no rebuild, and the 9.6 MB model is not duplicated into every image version. `:ro` means the app cannot modify it. For the cloud API I made the opposite choice (built in, Chapter 7) because there the image is shipped to another machine and should be self-contained and immutable.

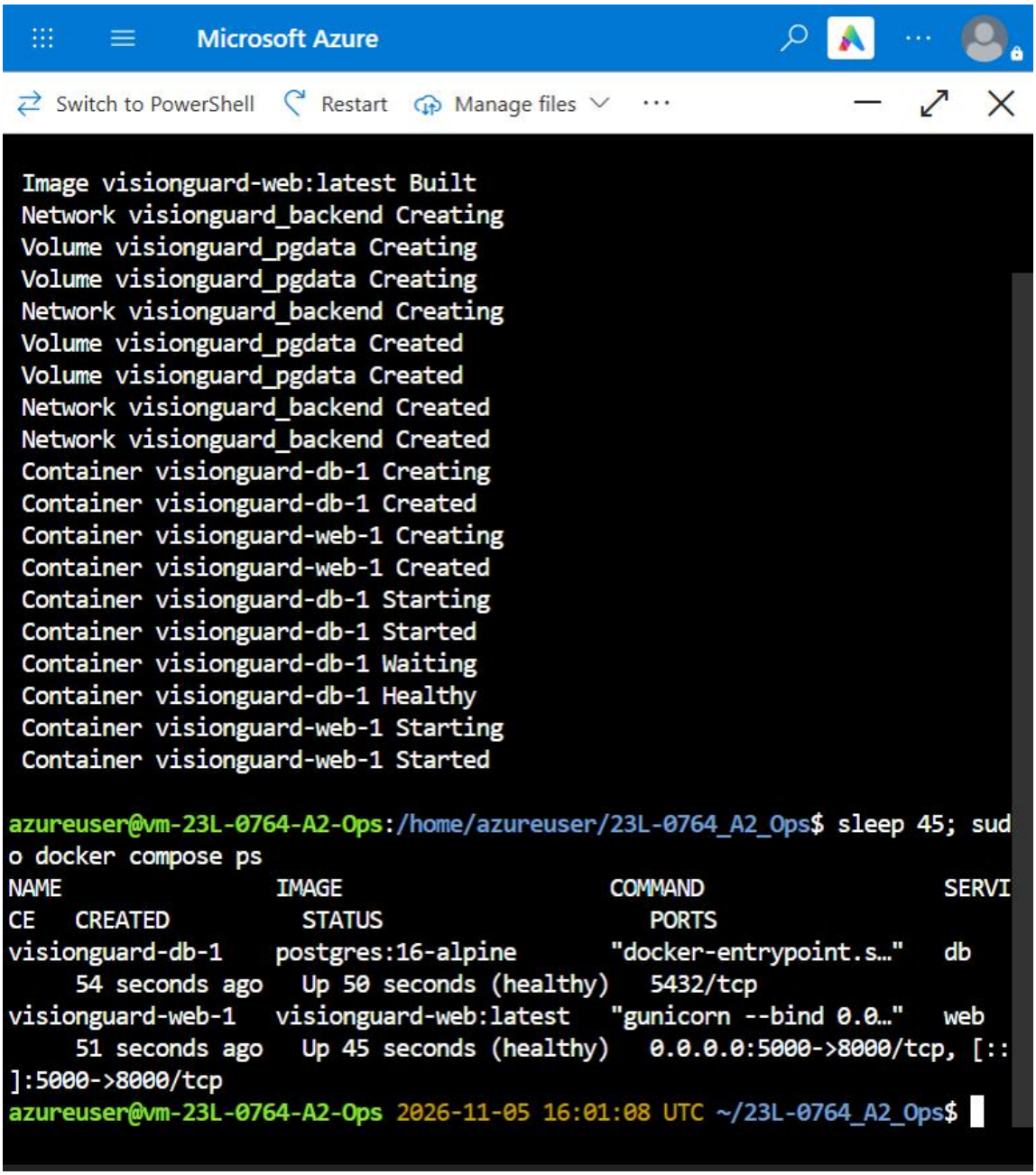


Figure 6.5: `docker compose ps`: both services healthy.

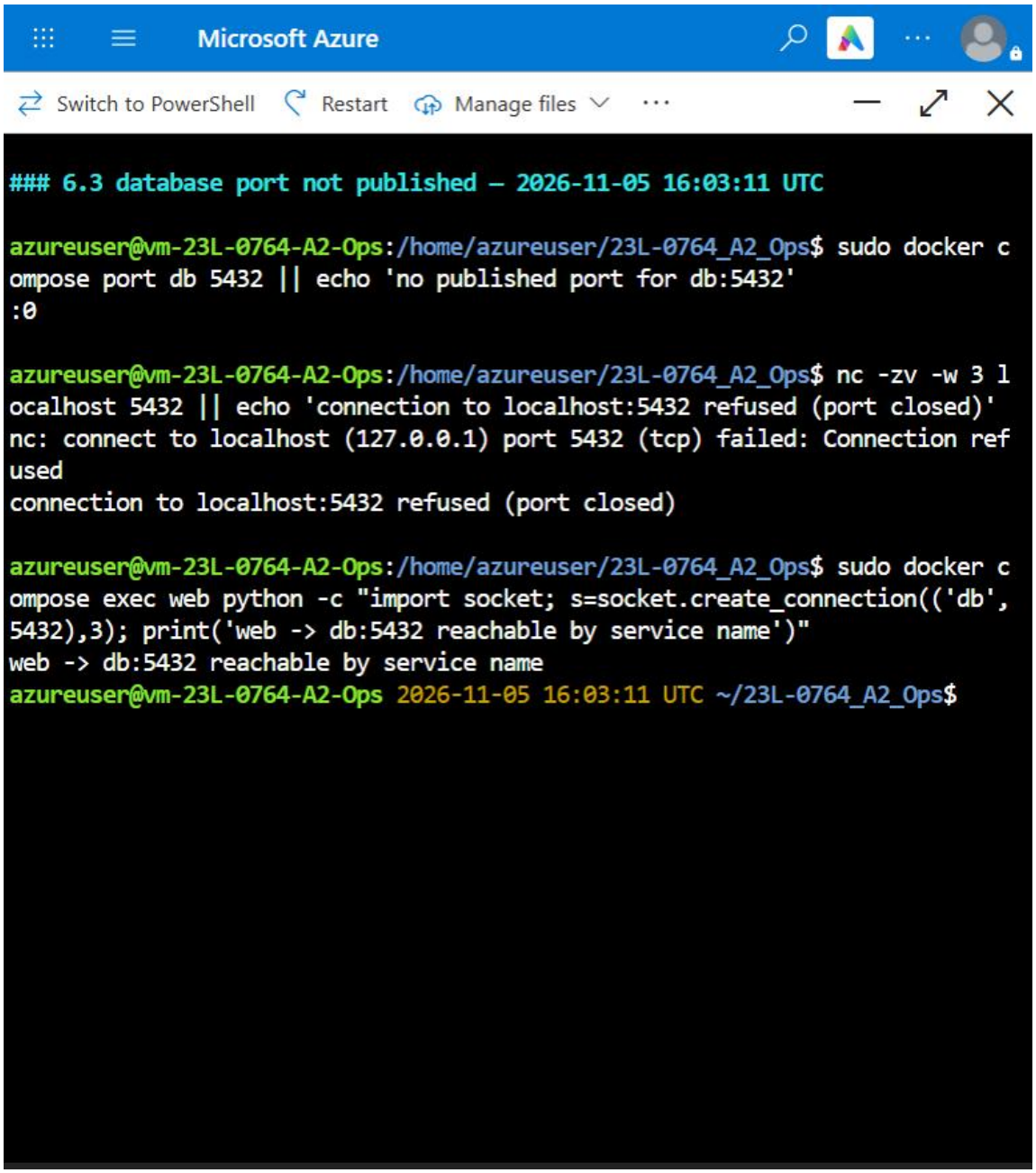


Figure 6.6: The database port is closed from outside (`docker compose port db 5432` is empty; a connection to localhost:5432 is refused).

6.4 Image analysis

Image	Size
`visionguard-web:naive` (`docker/Dockerfile.naive`)	6.06 GB (1.87 GB compressed content)
`visionguard-web:latest` (optimised)	2.16 GB (422 MB)
`visionguard-web:remote` (`WITH_TF=false`)	385 MB (90.8 MB)

Table 6.1: Image sizes from `docker images`.

The naive image uses the full `python:3.11` base, the GPU `tensorflow` wheel with its CUDA libraries, keeps the pip cache and runs as root. `docker history` shows that the largest layer by far is the `pip install` of TensorFlow and the serving libraries (1.59 GB), followed by the base image; my own code is about 150 KB (`app` 82 KB, `src` 66 KB) and the user/metadata layers are a few KB. The optimisations applied were: slim base, `tensorflow-cpu`, `PIP_NO_CACHE_DIR`, and the ignore file.

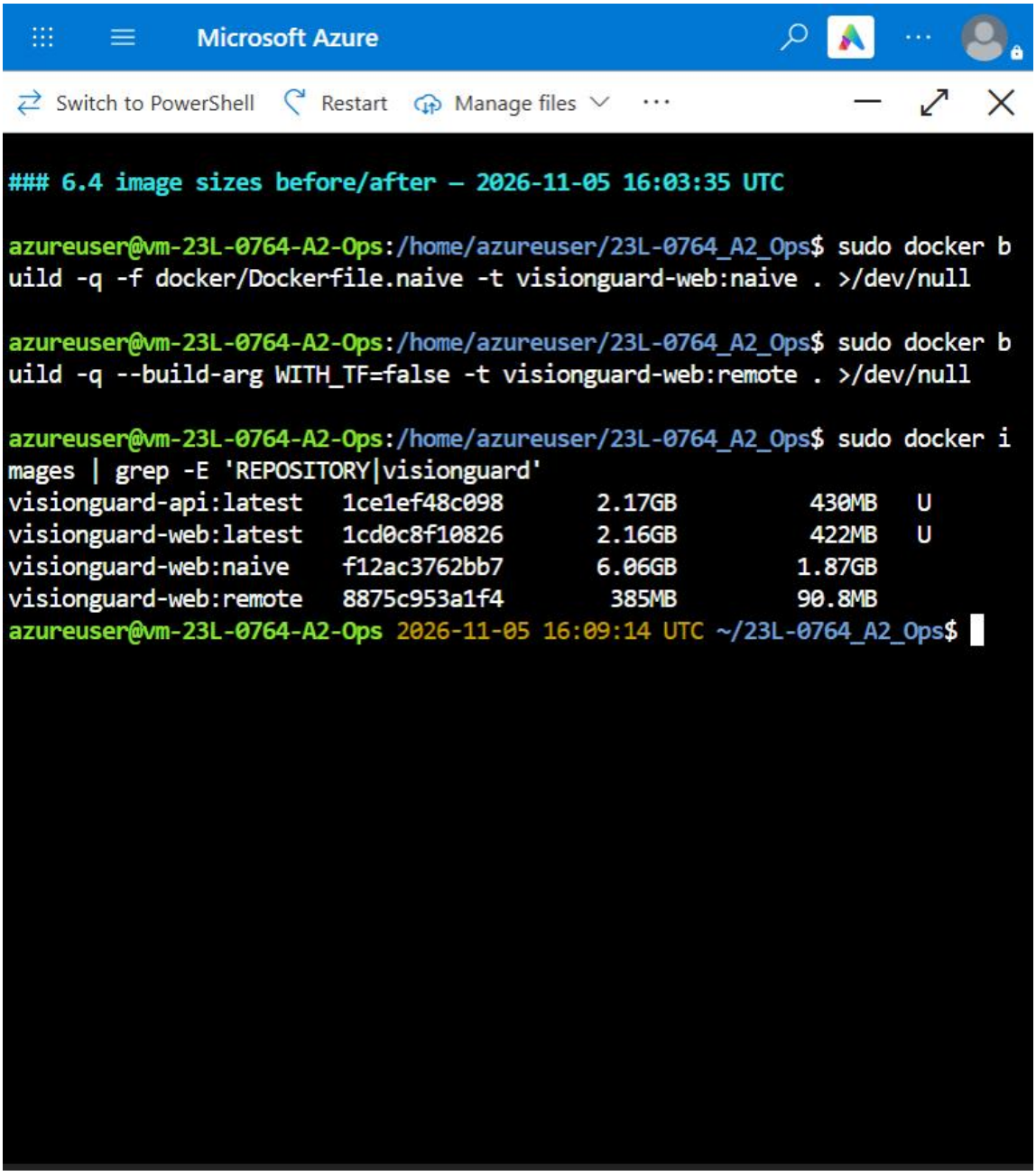


Figure 6.7: `docker images`: before and after.

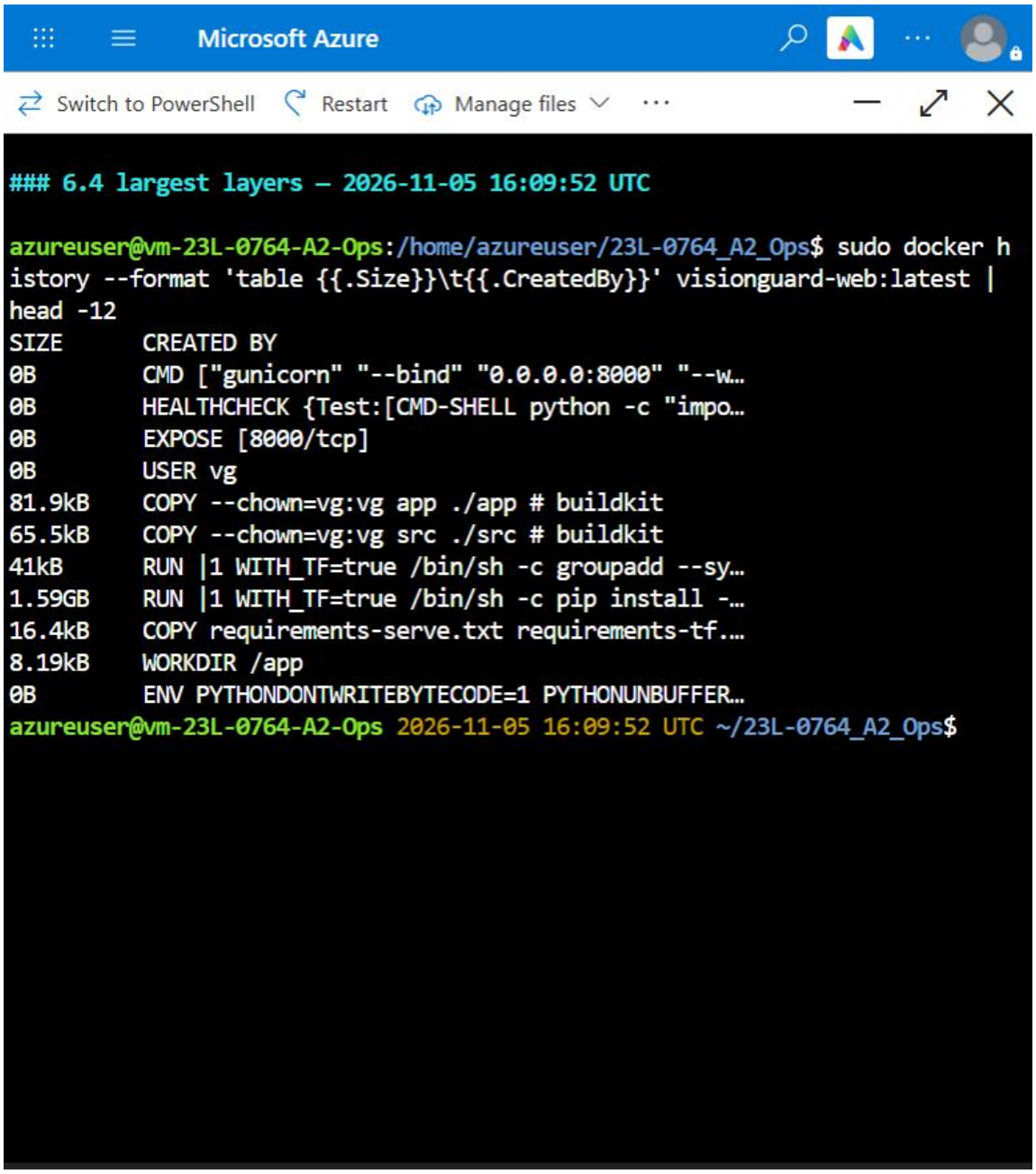


Figure 6.8: `docker history`: the TensorFlow layer dominates.

Conclusion. The slim base plus `tensorflow-cpu` and no pip cache cut the web image from 6.06 GB to 2.16 GB (−**64%**); almost all of the saving is the CUDA libraries that the GPU wheel pulls in and a CPU-only server never uses. Building without TensorFlow for `INFERENCE_MODE=remote`, where the model runs in the cloud API, brings it down to 385 MB (−94% versus naive).

6.5 Operations and troubleshooting

Routine operations: `docker compose stop web`, `start`, `restart db`, `docker compose logs -tail 30 web`, `docker compose exec web sh` and `docker compose exec db psql -U visionguard`.

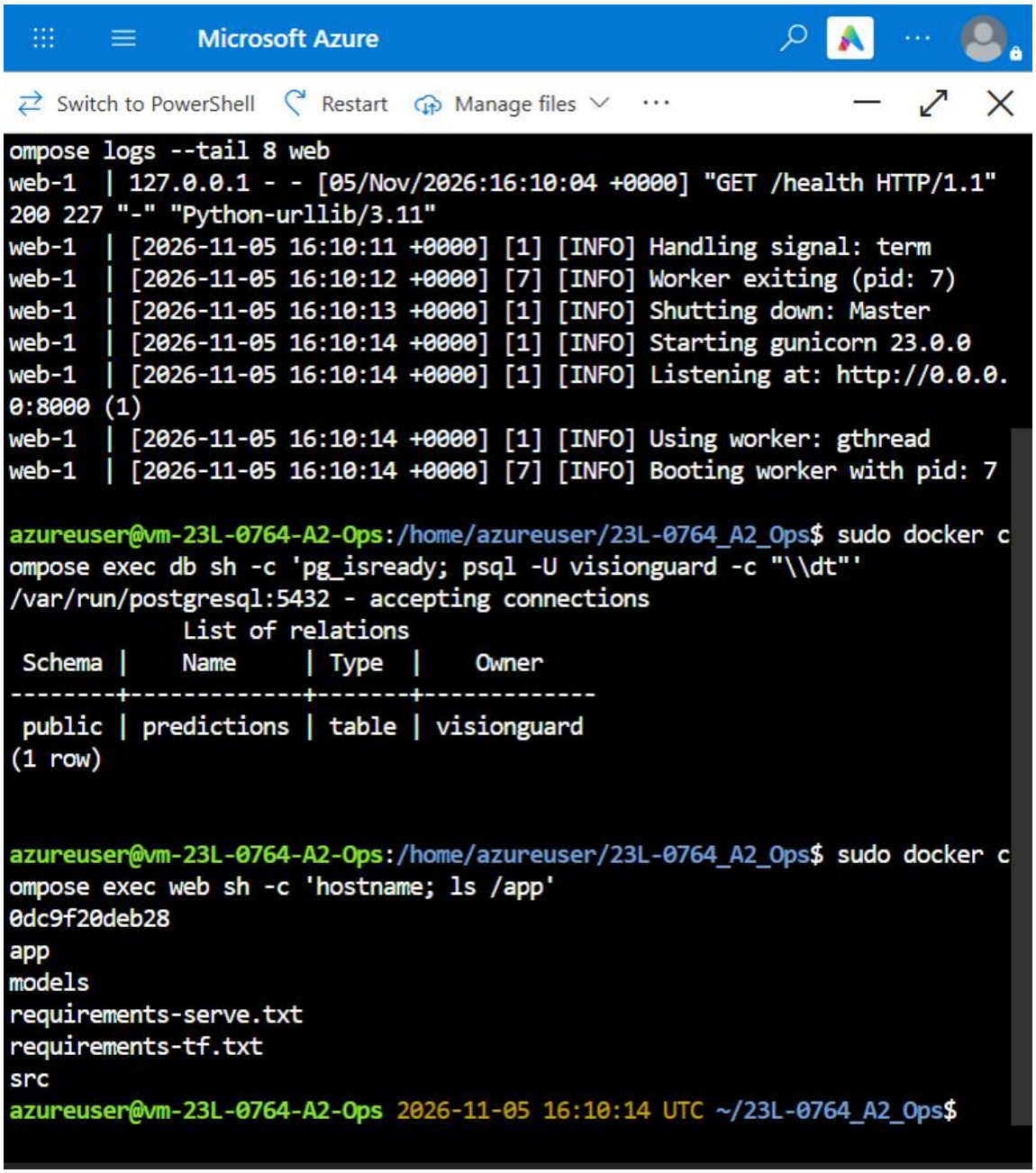


Figure 6.9: Start/stop/restart, logs and shells in both services.

Deliberate failure: wrong database password. I changed `POSTGRES_PASSWORD` in `.env` and recreated only the web container. `docker compose ps` showed it restarting. **Diagnosis from the logs only:** `docker compose logs web` showed my retry loop (`database not ready (OperationalError), retry n/10`) and then `FATAL: password authentication failed for user "visionguard"`. So the network and the database were fine (it answered) and the credentials were wrong. A key detail: the PostgreSQL image applies `POSTGRES_PASSWORD` only when the volume is *first* initialised, so the database kept the old password and only the client side changed. **Fix:** restore the value in `.env` and `docker compose up -d -force-recreate web`; the service returns to healthy and the history is intact.

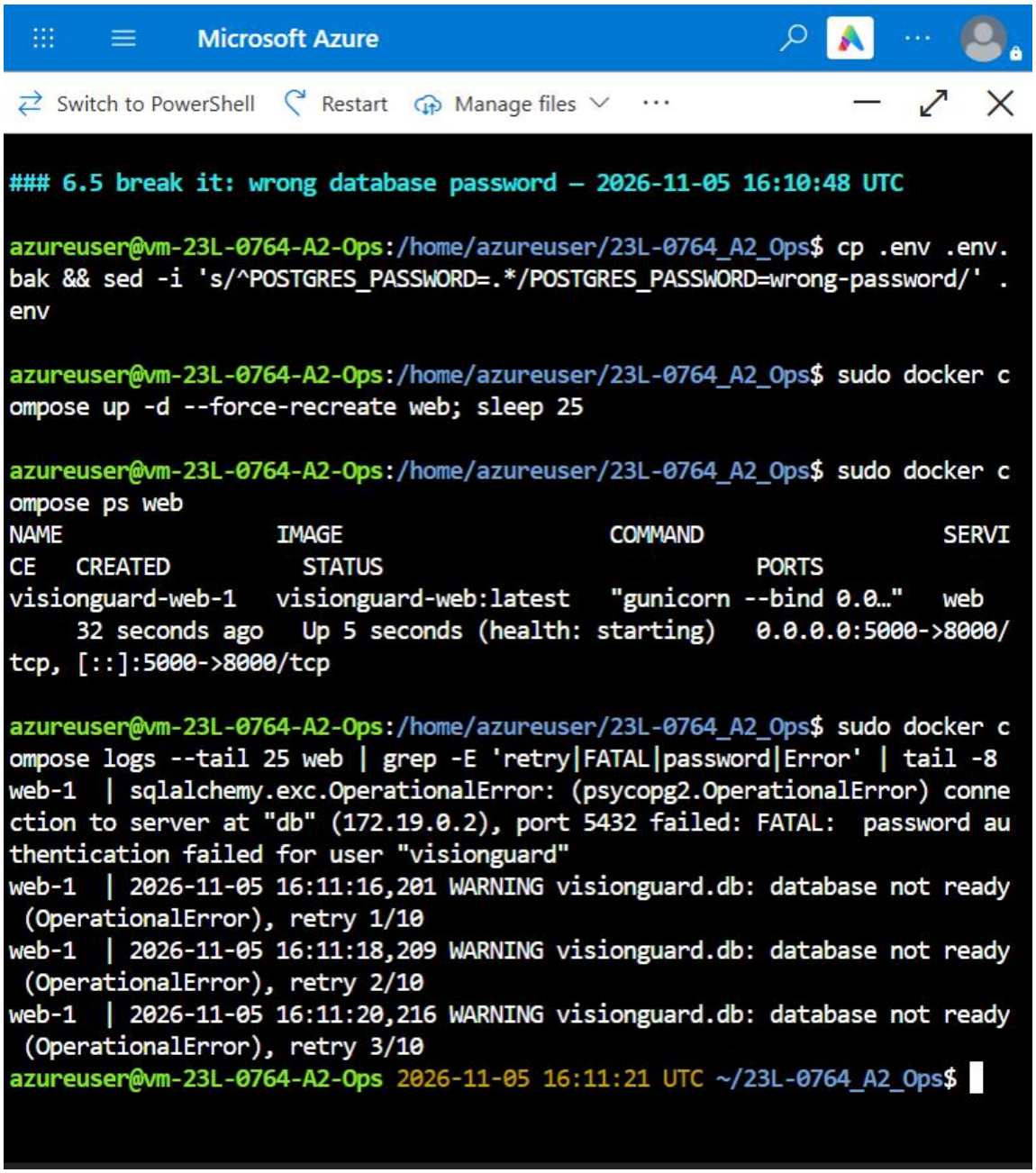


Figure 6.10: Log excerpt of the failure.

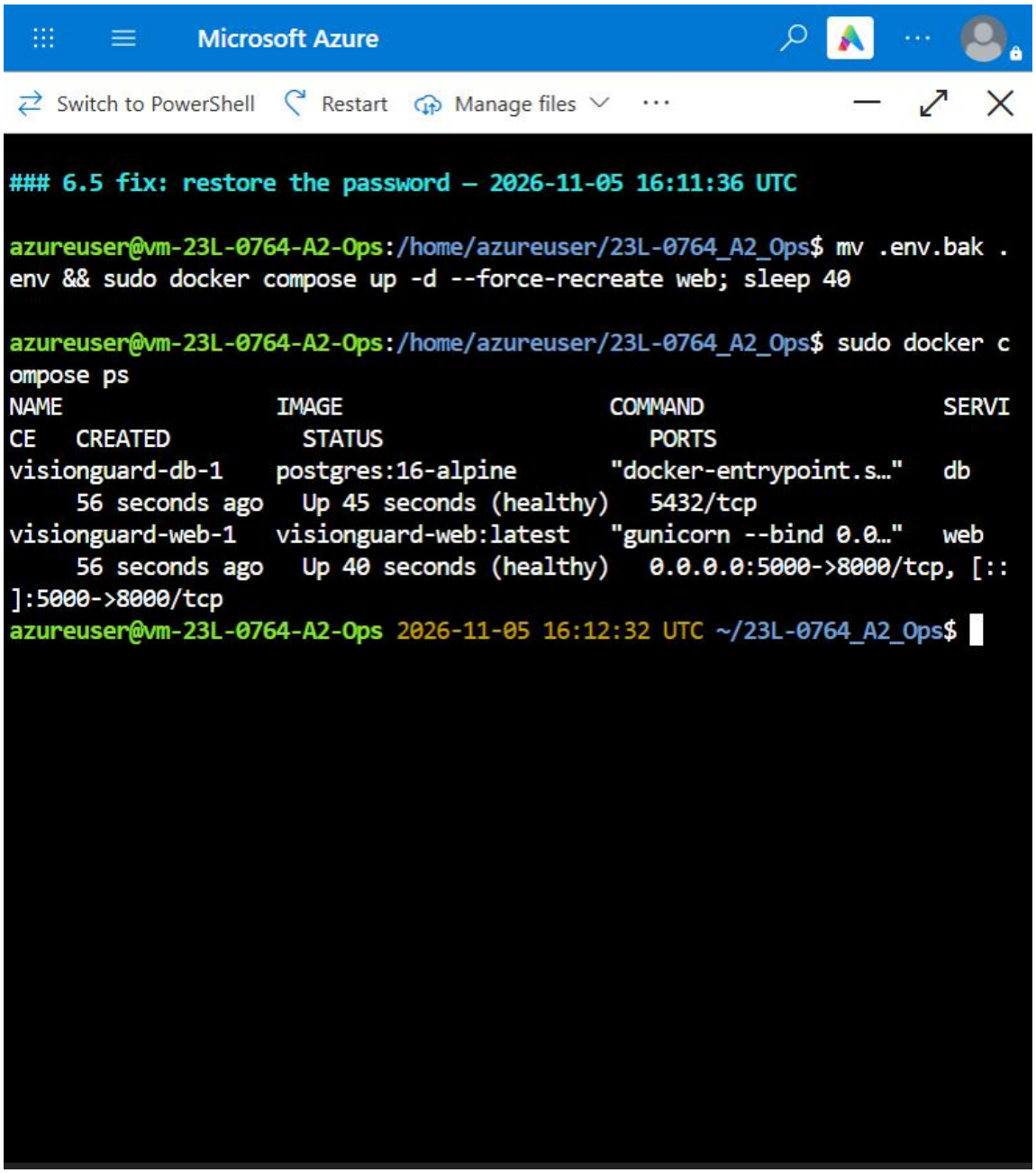


Figure 6.11: After the fix: both services healthy.

6.6 Tool lab: GitHub Actions (CI)

Toy part. On a separate orphan branch `ci-toy` of the repository (it shares no history or files with `main`; its content is `labs/ci_toy/`) a workflow runs on every push with a **matrix** of Python 3.11 and 3.12, so the same job runs twice in parallel, once per version. I changed an assertion to a wrong value (red run), then reverted it (green run).

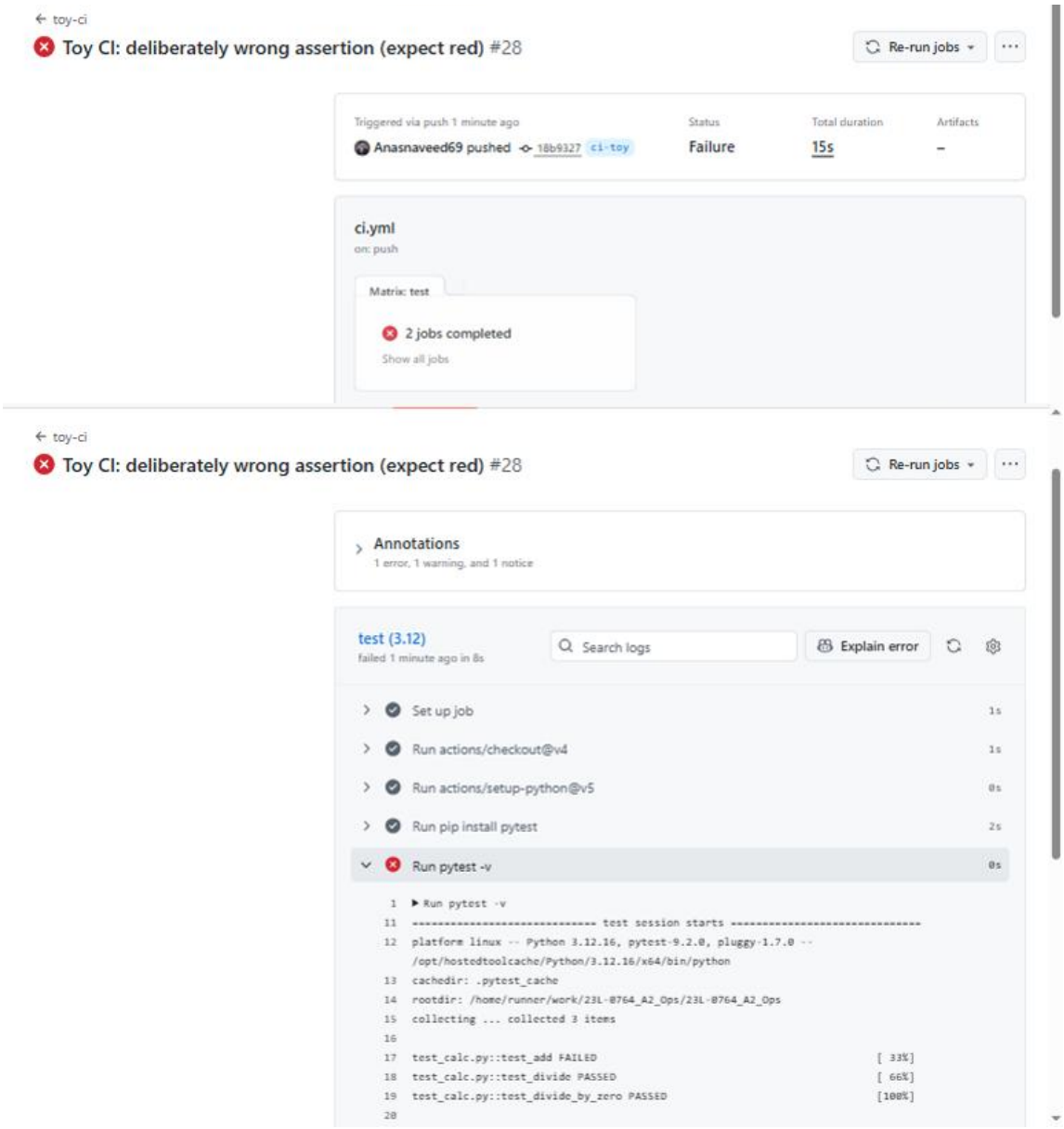


Figure 6.12: Toy workflow: failing test, red run.

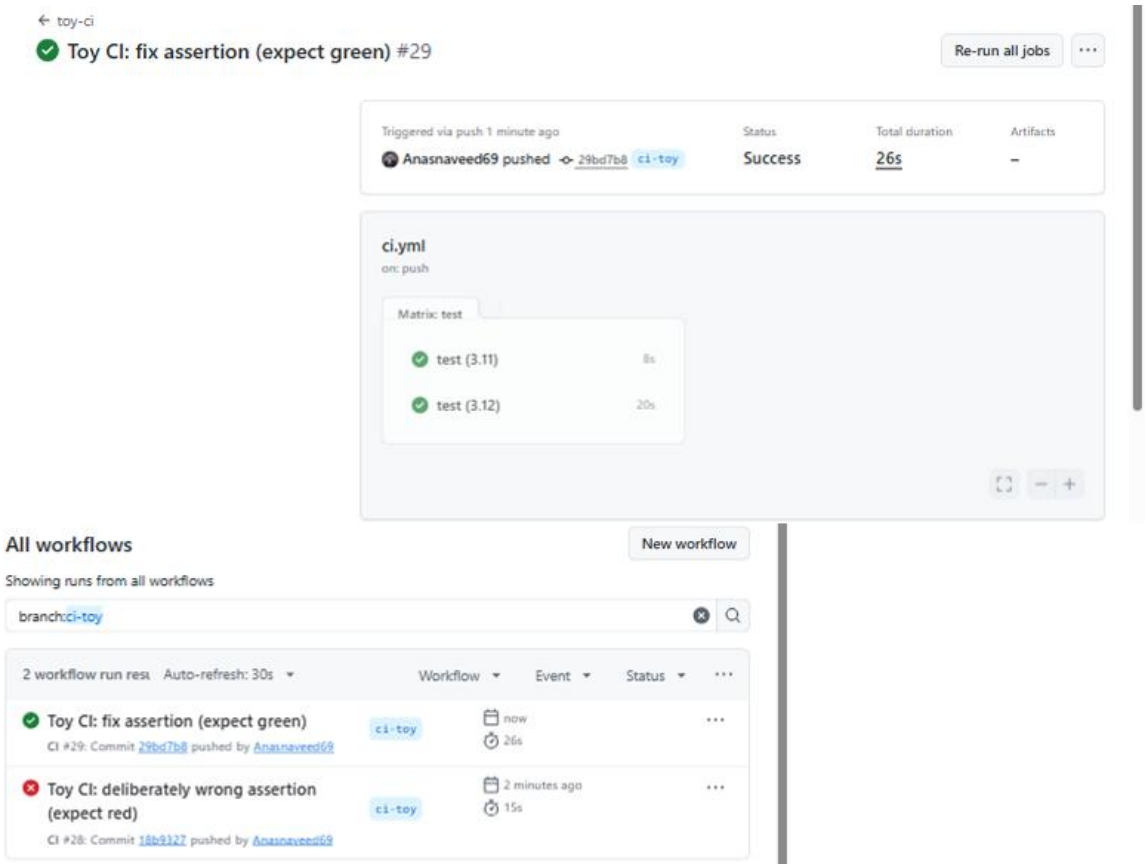


Figure 6.13: Toy workflow: fixed, green run on both Python versions.

Applied to VisionGuard (`.github/workflows/ci.yml`), on every push:

1. **lint + tests** on Python 3.11 and 3.12: `ruff check .`, then `pytest` (27 tests: upload validation, all routes and the DB trail with a fake predictor, the inference module and the API's key check with a tiny model).
2. **tests against PostgreSQL**: a `postgres:16` service container, so the trail is tested on the same database engine as production.
3. **docker**: builds the web and API images (with layer caching) and starts them, waits for `/health`, asserts the process user is `vg` and that the API returns 401 without a key. A tiny dummy model is generated for this step because the real champion lives in the registry, not in Git.

The badge in `README.md` shows the status of the latest run.

Where secrets go. Secrets (e.g. a DagsHub token, a deploy key) are stored in the repository's *Settings* → *Secrets and variables* → *Actions*, encrypted, and referenced as `${{ secrets.NAME }}`; GitHub injects them at run time and masks them in logs. Writing them in the workflow file would publish them to anyone who can read the repository, forever in Git history, and forks would copy them.

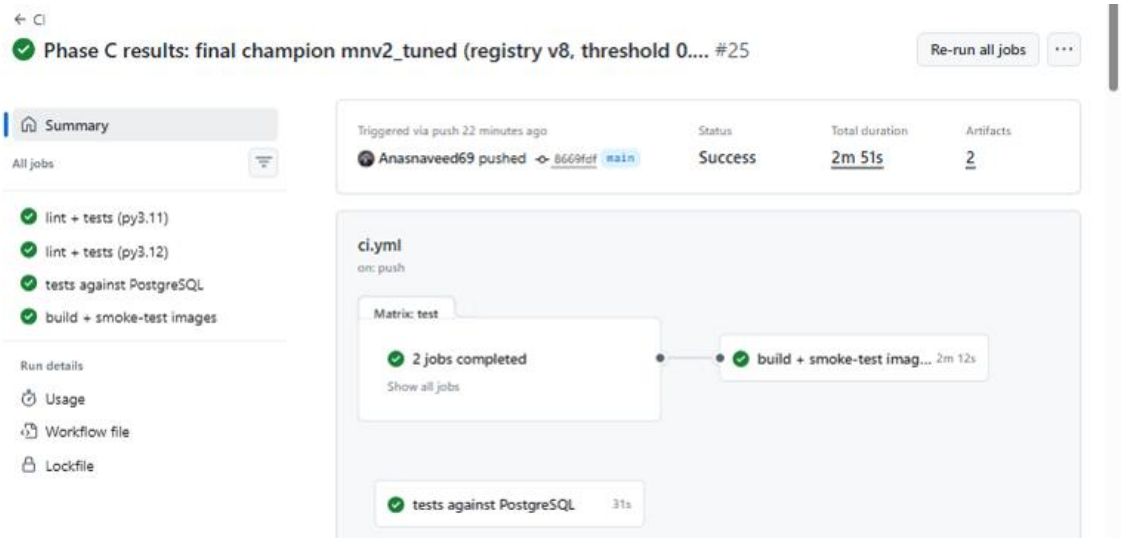


Figure 6.14: VisionGuard CI: all jobs green.

Figure 6.15: CI status badge in the README.

7 | API Deployment

Design. Following the assignment, the *model* goes to the cloud as an HTTP API and the Flask web app runs on my machine and calls it. I chose **pathway C**: one Ubuntu VM on **Azure** (Azure for Students: $100 credit with a university e-mail, no credit card) running a small Compose stack.

API URL: `https://20-219-10-43.sslip.io`

Compose / local element	Azure equivalent used
Host running Docker	VM `vm-23L-0764-A2-Ops` in resource group `rg-23L-0764-A2-Ops`, region Central India, Ubuntu 24.04 x64, size `Standard_B2s_v2` (2 vCPU, 8 GB)
Published ports 80/443	Network Security Group inbound rules + `ufw` on the VM
`api` service	`api/Dockerfile`: gunicorn + Flask API + TensorFlow-CPU, model built in
HTTPS	Caddy reverse proxy with an automatic Let's Encrypt certificate for `<ip>.sslip.io`
Secrets (`.env`)	`deploy/.env` created on the VM only (`chmod 600`), never committed
Database	stays with the web app on my machine (Task 8 design)

Table 7.1: Mapping the stack to Azure.

Things researched first.

- *Free terms*: Azure for Students gives $100 of credit valid for 12 months with no credit card; a 2-vCPU burstable VM costs a few cents per hour, so a week of uptime is a small fraction of the credit. I stop (deallocate) the VM when it is not needed.
- *Allowed regions*: the subscription carries an Azure Policy *"Allowed resource deployment regions"*; I read it with `az policy assignment list`: only Malaysia West, Central India, Austria East, UAE North and Southeast Asia are allowed.
- *Size*: TensorFlow plus the model needs roughly 1 GB of RAM at start-up; a 1 GB VM would be killed by the OOM killer, so I needed a 2-vCPU size with $\geq$4 GB and added 2 GB swap.
- *x86 vs Arm*: x86-64 image, so the same `tensorflow-cpu` wheel as on my laptop and in CI is used (Arm needs different wheels).
- *Two firewalls*: the cloud-level Network Security Group must allow 80/443 *and* the VM's own firewall (`ufw`) must too; either one alone blocks traffic.
- *Protecting the API*: the prediction endpoint requires a secret `X-API-Key` header, compared in constant time (`hmac.compare_digest`) so response timing does not leak the key; HTTPS means the key is never sent in clear text; request bodies are capped at 6 MB by Caddy and 5 MB by the app; `/health` is public but reveals only the model version.

Step-by-step deployment

Everything was done from **Azure Cloud Shell** (a browser terminal already logged in to my subscription) with two scripts from the repository, so the deployment is reproducible: `deploy/azure_cloudshell.sh` (cloud resources) and `deploy/vm_bootstrap.sh` (inside the VM).

1. **Register resource providers.** A brand-new subscription has `Microsoft.Compute`, `Microsoft.Network` and `Microsoft.Storage` in state `NotRegistered`, so every VM de-

ployment fails. `az provider register -n Microsoft.Compute -wait` (and the same for Network and Storage) enables them.

2. **Create the VM.** `az vm create -g rg-23L-0764-A2-Ops -n vm-23L-0764-A2-Ops -image Ubuntu2404 -size Standard_B2s_v2 -admin-username azureuser -generate-ssh-keys -public-ip-sku Standard -nsg-rule SSH` creates the VM, its disk, virtual network, public IP and a Network Security Group (NSG) that allows only SSH. `-generate-ssh-keys` creates a key pair, so no password login exists.

3. **Open the cloud firewall.** `az vm open-port -port 80 -priority 1010` and `-port 443 -priority 1020` add NSG inbound rules for HTTP (needed for the certificate challenge) and HTTPS.

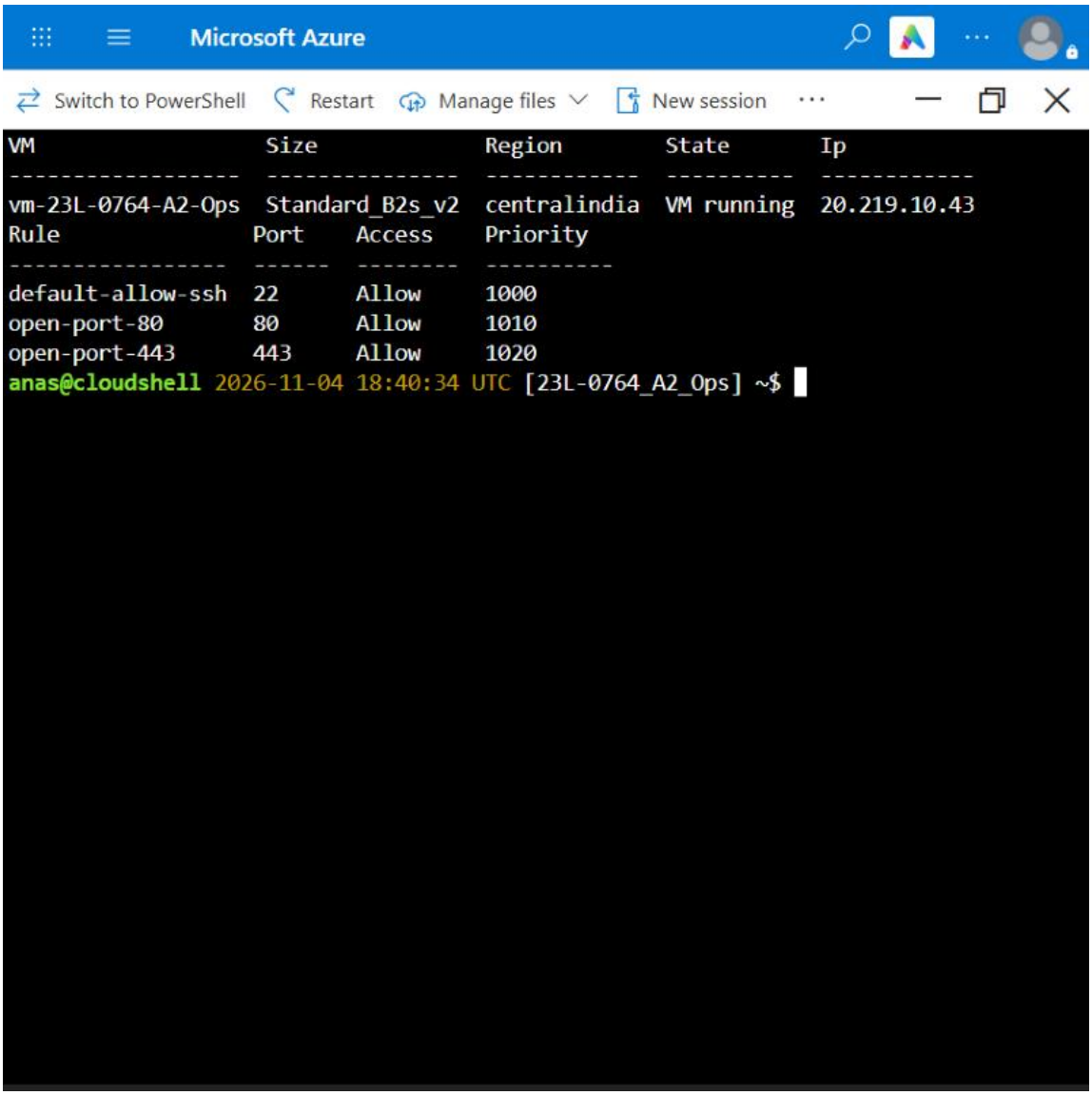


Figure 7.1: The VM (size, region, state, public IP) and the NSG inbound rules 22/80/443, listed from Cloud Shell.

4. **Prepare the VM** (over SSH, `deploy/vm_bootstrap.sh`): installs Docker Engine and the compose plugin, enables `ufw` allowing only SSH/80/443 (the second firewall), creates 2 GB swap, clones the repository (which contains the exported champion model, see Chapter 6), writes `deploy/.env` with a random 256-bit `API_KEY` (`secrets.token_urlsafe(32)`) and `API_DOMAIN=<ip-with-dashes>.sslip.io`; the free sslip.io DNS resolves that name to the IP, so Let's Encrypt can issue a certificate without buying a domain.

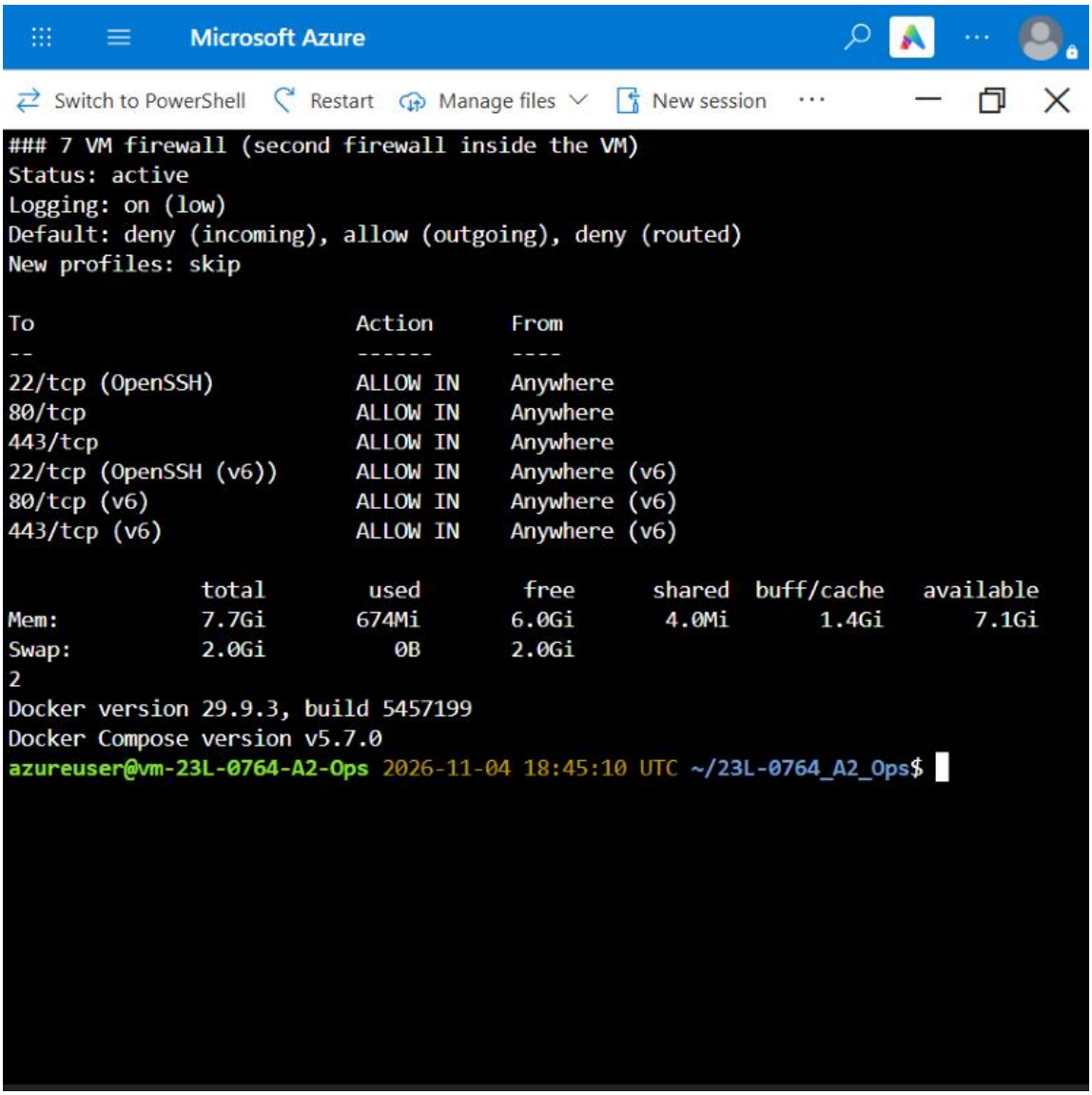


Figure 7.2: The VM's own firewall (`sudo ufw status verbose`), memory incl. swap, Docker versions.

5. **Start**: `docker compose -env-file deploy/.env -f deploy/docker-compose.cloud.yml up -d -build`. The API image is built with the model inside; Caddy starts only after the API is healthy, obtains the certificate and proxies `https://API_DOMAIN` to `api:8080`. Only Caddy publishes ports; the API container is reachable only on the internal network.

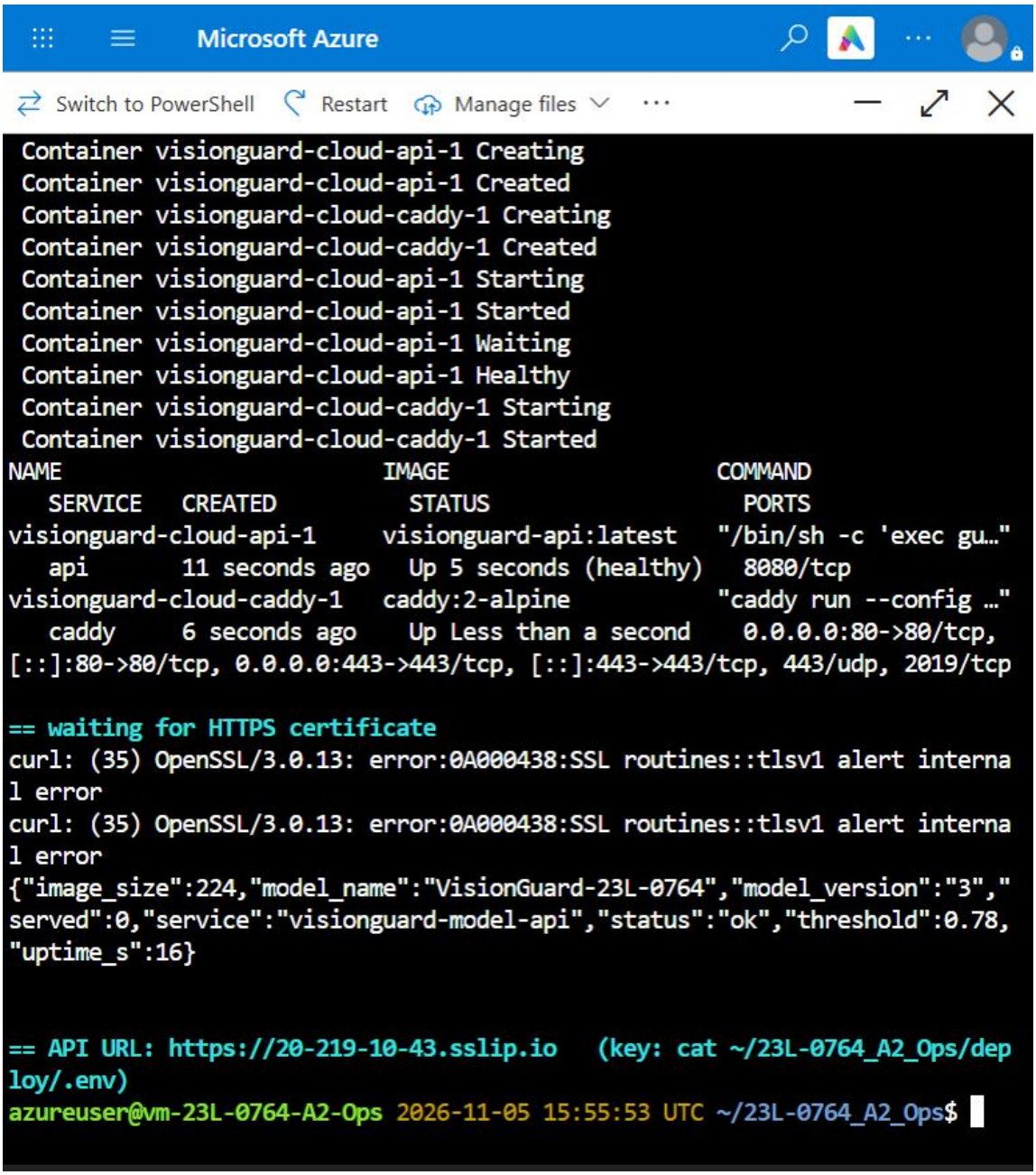


Figure 7.3: Cloud stack running: API healthy, Caddy up, HTTPS health check answered.

6. **Verify from another network**: `/health` opened from a different network.

Figure 7.4: `https://20-219-10-43.sslip.io/health` opened from outside the VM's network.

What went wrong

(1) The first `az vm create` failed in every region with an unreadable Python traceback (*"The content for this response was already consumed"*, an Azure CLI bug that hides the real error). Checking the state showed the cause: the resource providers were `NotRegistered`; registering them fixed it. **(2)** Next, `Standard_B2s` failed with `SkuNotAvailable`: *"currently not available in location CentralIndia"* (and in all other allowed regions) because of capacity restrictions. I looped over equivalent small sizes; `Standard_B2s_v2` (2 vCPU, 8 GB) was available in Central India. **(3)** The first API image build on the VM failed with `"/models/champion.keras": not found` because I started it before the champion had been trained and exported; it built once the model was committed.

Testing with Postman

The collection `deploy/VisionGuard.postman_collection.json` has variables `base_url` and `api_key` and five requests with test scripts that assert the status code and the response fields:

1. `GET /health` → 200 and the model name/version;
2. `POST /v1/predict` with an AI image (form-data field `image`) → 200, label, confidence, latency;
3. the same with a real photo;
4. without the `X-API-Key` header → 401;
5. with a non-image file (I used a `.md` file) → 415.

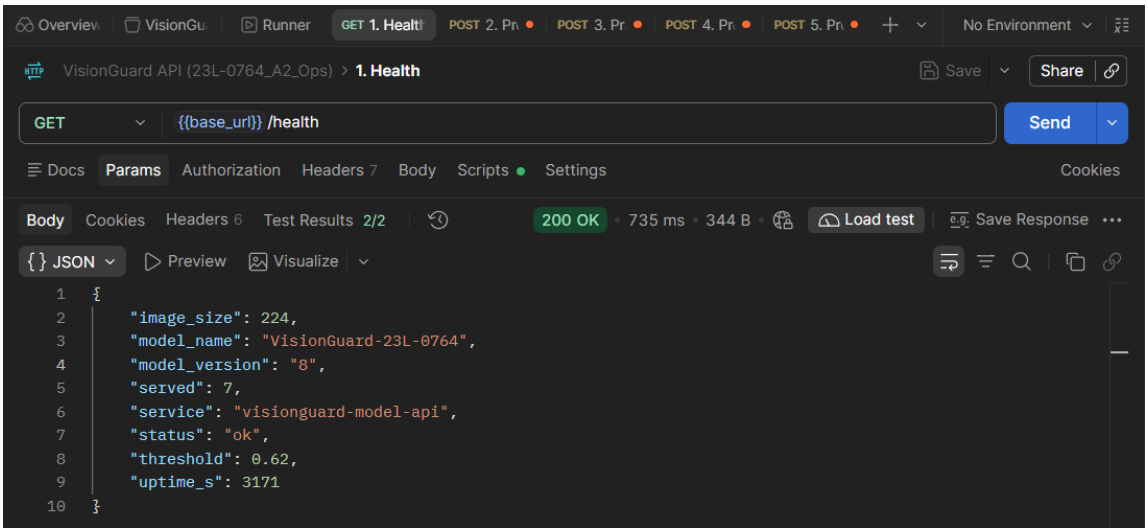


Figure 7.5: Postman: health.

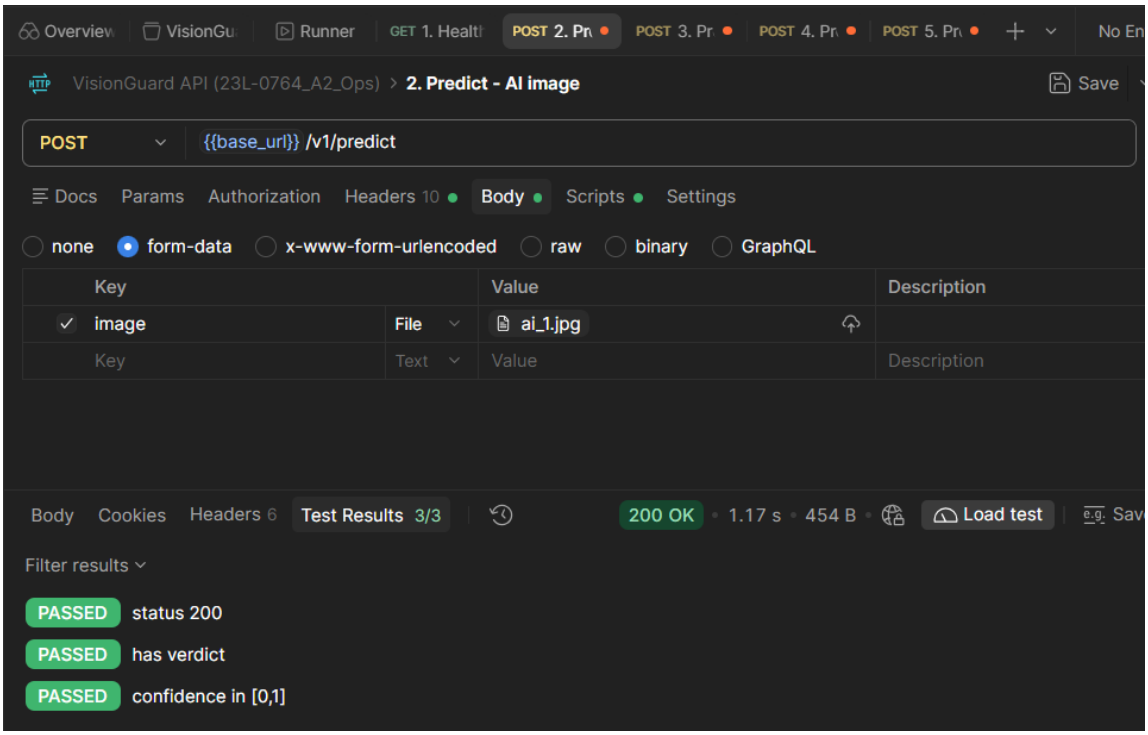


Figure 7.6: Postman: AI image (`ai_1.jpg`) → 200, all three test assertions pass.

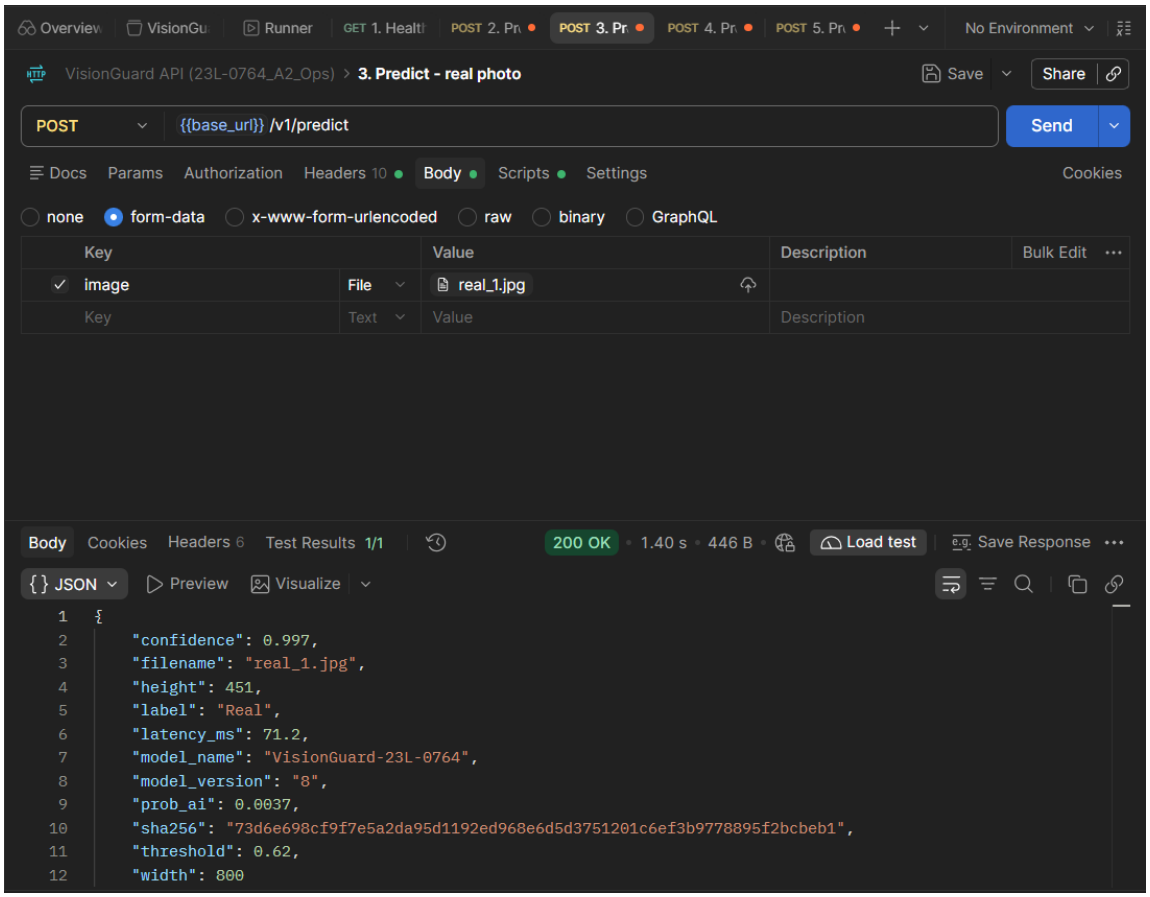


Figure 7.7: Postman: real photo → verdict JSON.

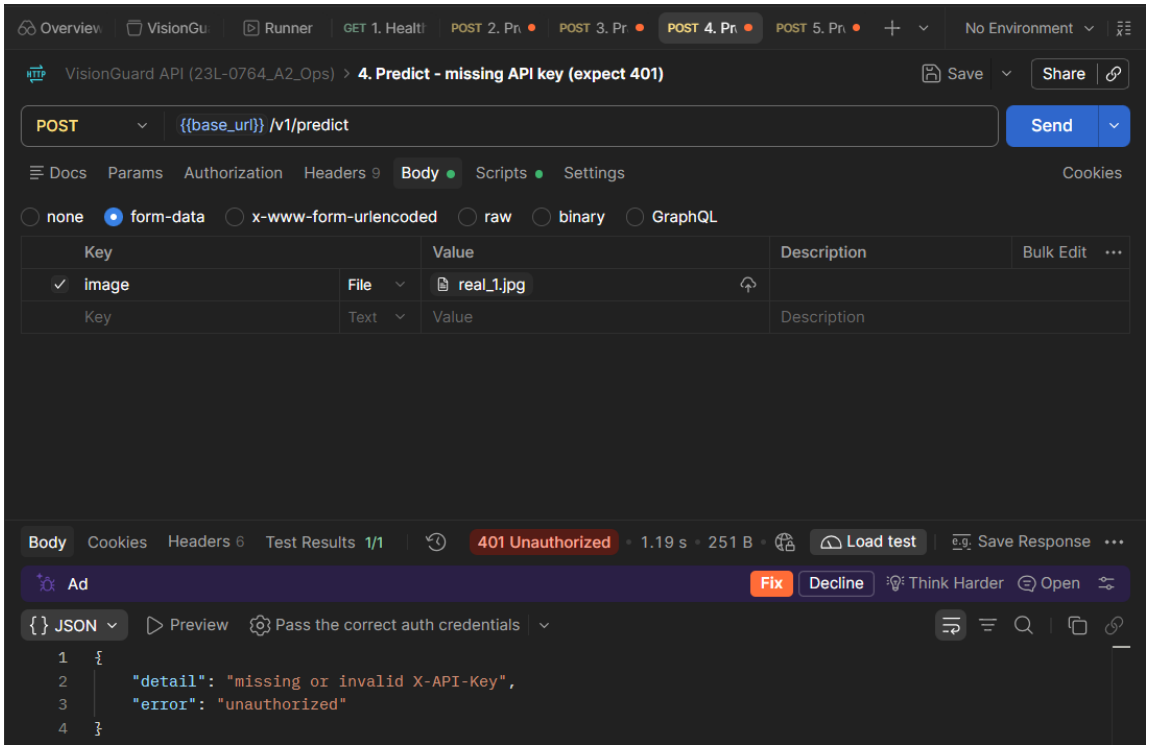


Figure 7.8: Postman: missing API key rejected (401).

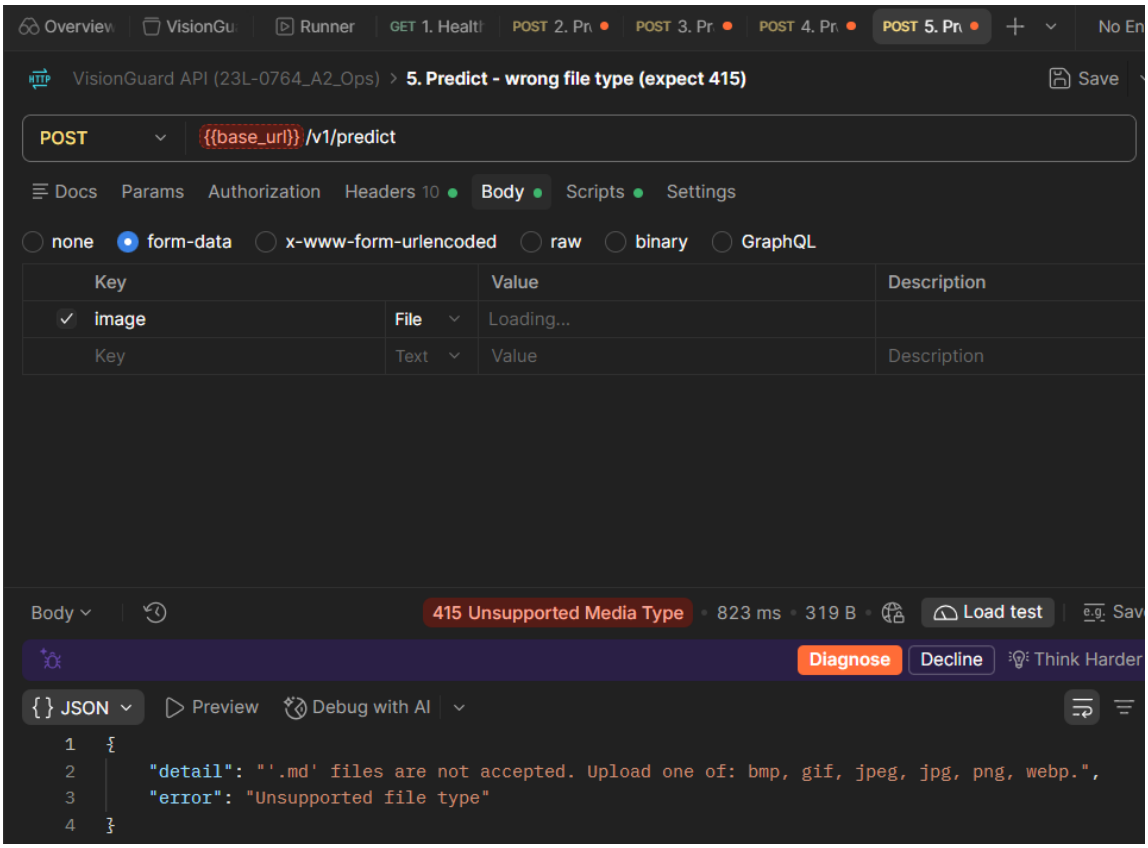


Figure 7.9: Postman: wrong file type (`.md`) rejected (415).

Example response (`real_1.jpg`, from Postman):


```
{"confidence": 0.997, "filename": "real_1.jpg", "height": 451, "label": "Real",
 "latency_ms": 71.2, "model_name": "VisionGuard-23L-0764", "model_version": "8",
 "prob_ai": 0.0037, "sha256": "73d6e698...f2bcbeb1", "threshold": 0.62, "width": 800}
```


Latency. The model itself needs about 70 ms per image on the 2-vCPU VM (`latency_ms`, 69–77 ms over all requests in the API log). The round trip seen from Lahore is much larger and varies with the client: 0.2–0.4 s from the Flask app on my laptop (re-used HTTPS connection) and 0.7–1.4 s from Postman, which opens a new TLS connection per request and uploads through its agent. Nearly all of the time is network, not model.

8 | Integrating Flask and the Cloud API

How it works. The web app has two interchangeable back-ends behind one interface (`app/predictors.py`): `LocalPredictor` wraps the in-process `Predictor`; `RemotePredictor` sends the validated image bytes as multipart form data to `MODEL_API_URL/v1/predict` with the `X-API-Key` header over HTTPS, through one reused `requests.Session` (connection pooling) with a 30 s timeout. Switching is one environment variable, `INFERENCE_MODE=remote`; routes, templates and the database code are unchanged. In remote mode the web image can be built without TensorFlow (`WITH_TF=false`), because the model lives only in the cloud.

The flow per upload: the browser posts to `/predict` → the web app validates the file locally (wrong types never leave my machine) → calls the API → stores the decision, the model version reported by the API, `inference_mode = remote` and the round-trip latency in the database → shows the result page. For this test the web app ran directly on my Windows laptop (`scripts/run_local.ps1`: virtual environment, `FLASK_DEBUG=0`, history in a local SQLite file through the same SQLAlchemy code); in the Compose stack the same code writes to PostgreSQL (Chapter 5). If the API is unreachable or answers with an error, the user gets a friendly 503 page and the attempt is still recorded with `status = error`.

Browser — http://127.0.0.1:5000/health

GET http://127.0.0.1:5000/health (Flask app on my laptop)


```
{
  "database": "ok",
  "debug": false,
  "inference_mode": "remote",
  "model": {
    "api": "https://20-219-10-43.sslip.io",
    "model_name": "VisionGuard-23L-0764",
    "model_version": "8",
    "threshold": 0.62
  },
  "model_backend": "ok",
  "status": "ok",
  "time": "2026-11-05T16:59:49+00:00"
}
```


Figure 8.1: Web app health in remote mode: model backend = the cloud API.

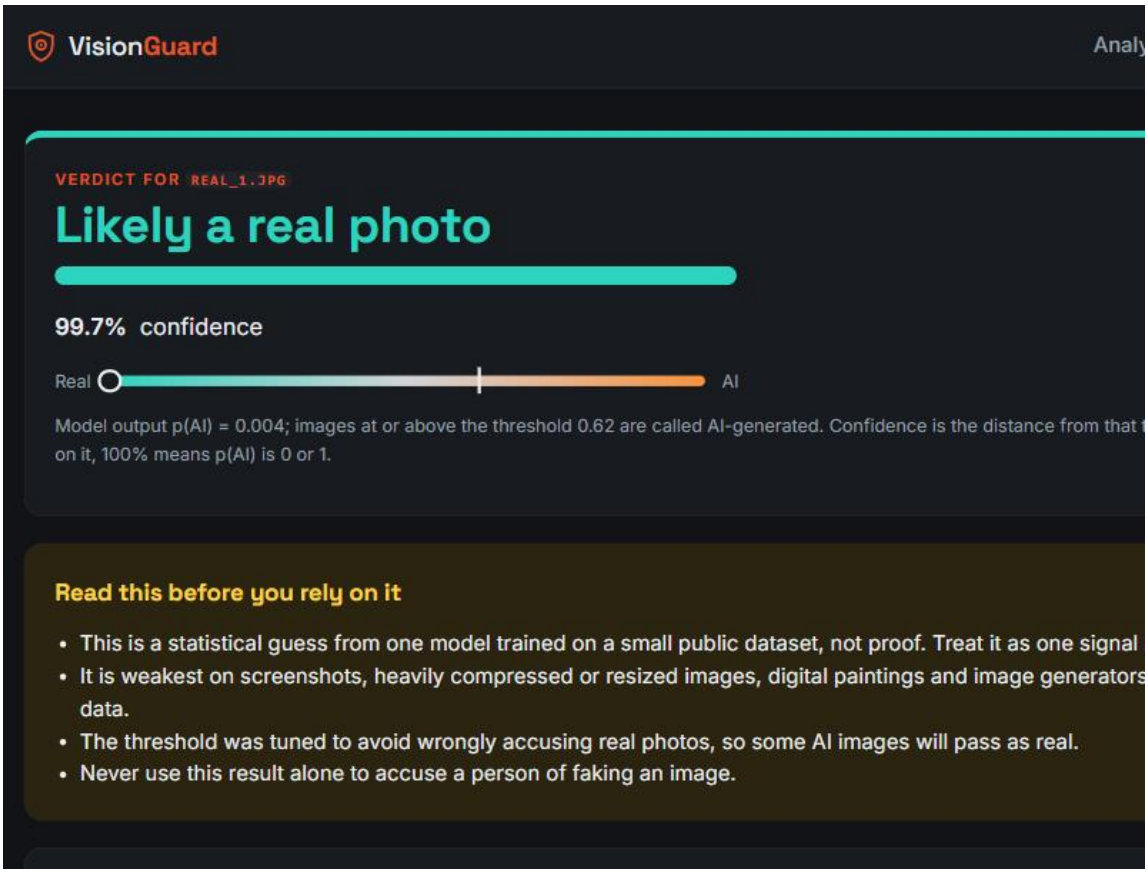


Figure 8.2: Upload of `real_1.jpg` through the UI, answered by the cloud model (record #5, 230 ms).

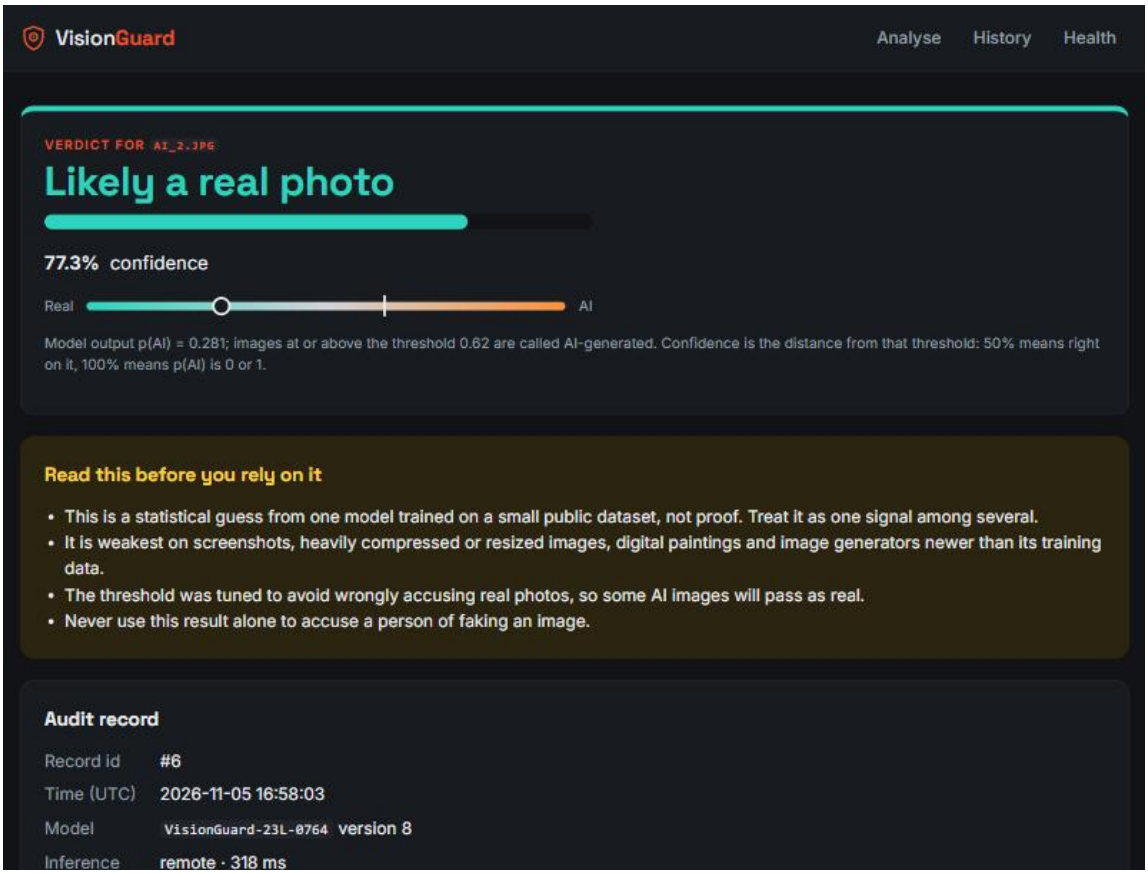


Figure 8.3: Upload of `ai_2.jpg`: an AI image that the model misses ($p(\text{AI}) = 0.28 < 0.62$).

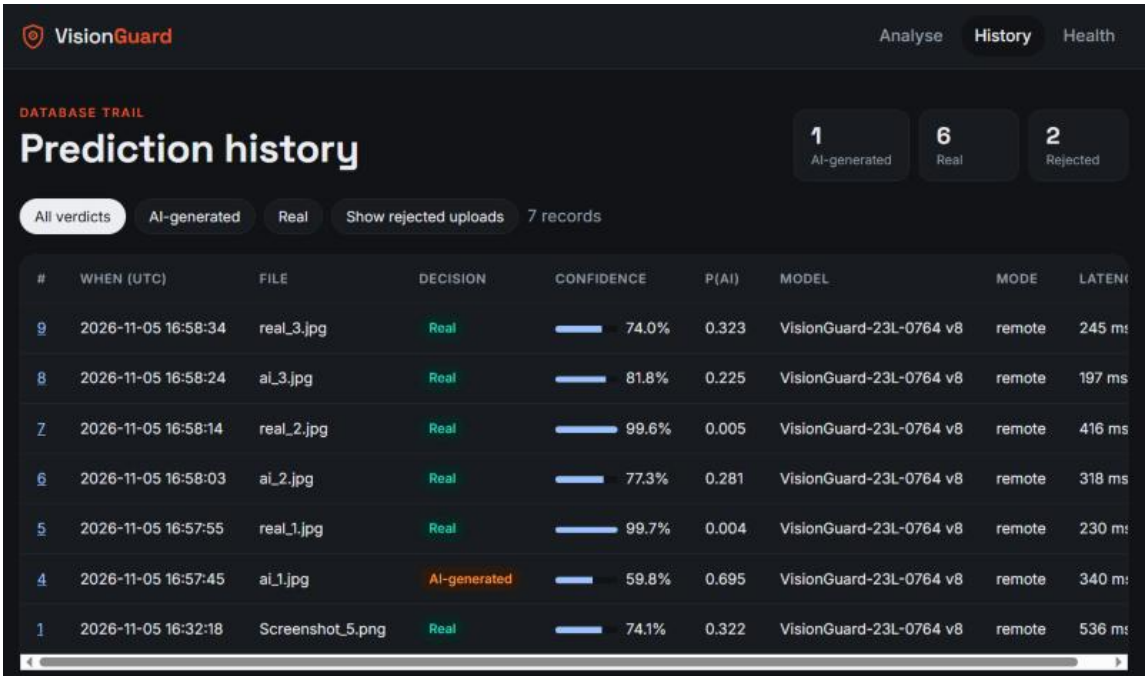


Figure 8.4: History menu: the database log, with `mode = remote` rows.

```
2026-11-05 16:57:45,399 INFO visionguard.api: predict ai_1.jpg -> AI-genera
ted (0.598) in 70 ms from 39.52.100.128
172.18.0.3 - - [05/Nov/2026:16:57:45 +0000] "POST /v1/predict HTTP/1.1" 200
 277 "-" "VisionGuard-web/1.0"
2026-11-05 16:57:55,724 INFO visionguard.api: predict real_1.jpg -> Real (0
.997) in 71 ms from 39.52.100.128
172.18.0.3 - - [05/Nov/2026:16:57:55 +0000] "POST /v1/predict HTTP/1.1" 200
 269 "-" "VisionGuard-web/1.0"
2026-11-05 16:58:03,840 INFO visionguard.api: predict ai_2.jpg -> Real (0.7
73) in 73 ms from 39.52.100.128
172.18.0.3 - - [05/Nov/2026:16:58:03 +0000] "POST /v1/predict HTTP/1.1" 200
 269 "-" "VisionGuard-web/1.0"
2026-11-05 16:58:14,872 INFO visionguard.api: predict real_2.jpg -> Real (0
.996) in 77 ms from 39.52.100.128
172.18.0.3 - - [05/Nov/2026:16:58:14 +0000] "POST /v1/predict HTTP/1.1" 200
 271 "-" "VisionGuard-web/1.0"
2026-11-05 16:58:24,446 INFO visionguard.api: predict ai_3.jpg -> Real (0.8
18) in 69 ms from 39.52.100.128
172.18.0.3 - - [05/Nov/2026:16:58:24 +0000] "POST /v1/predict HTTP/1.1" 200
 268 "-" "VisionGuard-web/1.0"
2026-11-05 16:58:33,952 INFO visionguard.api: predict real_3.jpg -> Real (0
.740) in 71 ms from 39.52.100.128
172.18.0.3 - - [05/Nov/2026:16:58:33 +0000] "POST /v1/predict HTTP/1.1" 200
 270 "-" "VisionGuard-web/1.0"
azureuser@vm-23L-0764-A2-Ops 2026-11-05 17:01:45 UTC ~/23L-0764_A2_Ops$
```

Figure 8.5: The same requests arriving in the API logs on the Azure VM.

Multi-input test. I uploaded six held-out test images from my laptop through the UI (three AI-generated, three real). All six were answered by the cloud model (version 8) and logged with `mode = remote`. All three real photos were classified correctly with high confidence ($p(\text{AI})$ 0.004, 0.005 and 0.32), and one of the three AI images was caught ($p(\text{AI}) = 0.69$, just above the 0.62 threshold). The other two AI images (0.28 and 0.23) were called real, so 4 of 6 were correct. This matches the operating point I chose on purpose in Section 2.5: the threshold favours precision (no real photo was accused) at the cost of recall on the AI class (test recall 0.66), and the two misses are photorealistic renders, the same error type as in Section 2.6. The API logs show about 70 ms of model time per image on the VM, while the round trip seen by the web app (Lahore → Central India over HTTPS) was 197–416 ms. The earlier rejected uploads (415, 422) never reached the API, because validation happens locally first.

9 | Conclusion and Limitations

VisionGuard is reproducible end to end: the data version is pinned by DVC, every run records its commit, data version, seed, parameters, metrics and artifacts in MLflow on DagsHub, the served model is chosen by a written rule on validation data and promoted with a registry alias, and the whole system starts with one `docker compose up`. The model API runs on Azure behind HTTPS and an API key, and every decision the web app makes is stored with its time, model version and confidence.

Limitations.

- The dataset is small and scraped; "AI" images come from a few generators and "real" images include edited and stock photos, classical paintings and clip-art. The error analysis and Grad-CAM (Section 2.6) show that the model partly learned *style* (painterly $\Rightarrow$ AI, photographic $\Rightarrow$ real) instead of generation artefacts. Accuracy on new generators, screenshots or images re-compressed by social media will be lower than the test numbers suggest.
- Size, format and compression differ between classes (Chapter 1); despite resizing, some of the model's skill may come from these shortcuts rather than generation artefacts.
- A single train/val/test split of a few hundred images gives noisy metrics; cross-validation or a larger, more diverse dataset would make the comparison in Section 3.5 more reliable.
- One VM is a single point of failure; there is no autoscaling, rate limiting or monitoring of prediction drift yet.

Next steps. Train on a larger multi-generator dataset, add JPEG re-compression as a *test-time* robustness check, log production predictions back to MLflow to monitor drift, and deploy the API image from CI on every tagged release.